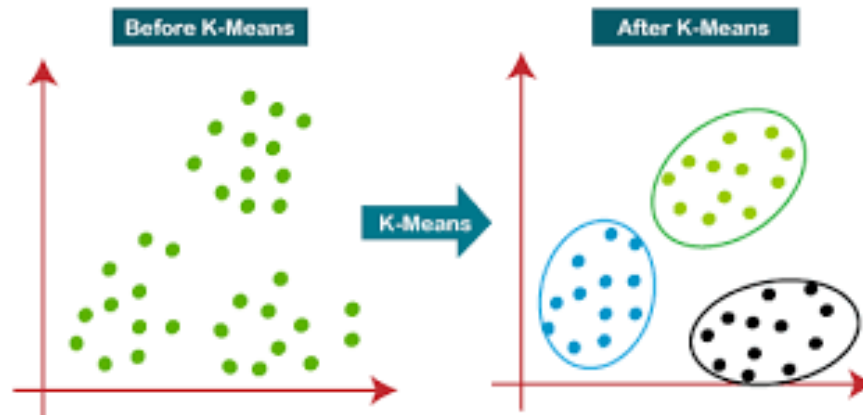


Clustering

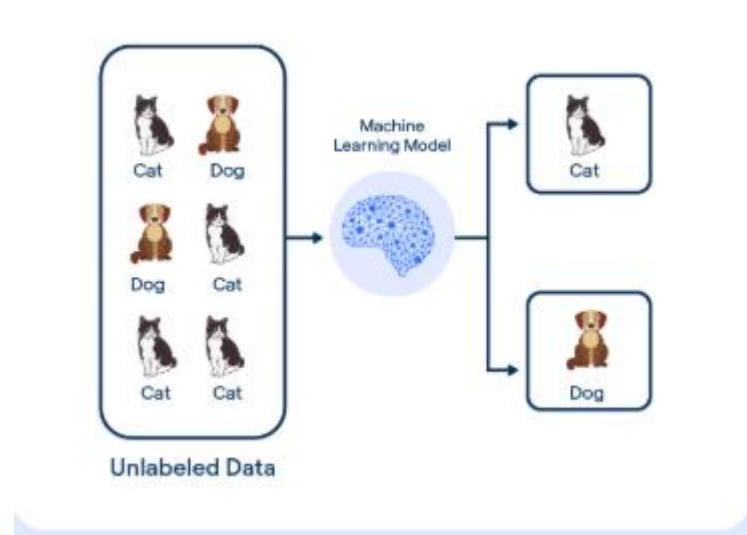


Supervised learning vs. unsupervised learning

- **Supervised learning:** discover patterns in the data that relate data attributes with a target (class) attribute.
 - These patterns are then utilized to predict the values of the target attribute in future data instances.
- **Unsupervised learning:** The data have no target attribute.
 - We want to explore the data to find some intrinsic structures in them.

What is Unsupervised Learning?

- Unsupervised learning is a machine learning paradigm where the algorithm **learns patterns and structures from input data without explicit supervision.**
- Unlike supervised learning, there are **no labeled target variables.** The algorithm must find **inherent patterns** on its own.
- The primary goal is to **explore and uncover underlying relationships** and groupings in the data.



Cont.

- It's about learning interesting/useful structures in the data (unsupervisedly!)
- There is no supervision (no labels/responses), only inputs $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$
- Some examples of unsupervised learning
 - **Clustering**: Grouping similar inputs together (and dissimilar ones far apart)
 - **Dimensionality Reduction**: Reducing the data dimensionality
 - **Estimating the probability density of data** (which distribution “generated” the data)
- Most unsup. learning algos can also be seen as learning a **new representation** of data
 - For example, hard clustering can be used to get a **one-hot representation**

Each point belongs deterministically to a single cluster

K = 6 clusters

z_n

0	0	1	0	0	0
---	---	---	---	---	---

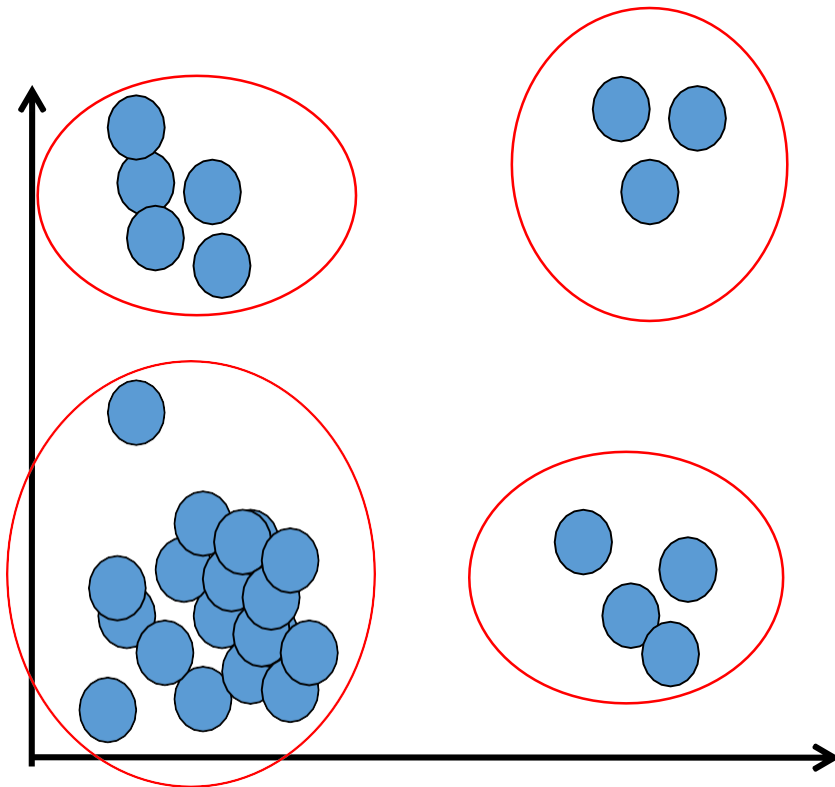
 $\leftarrow x_n$ belongs to cluster 3

A one-hot (quantized) rep.

Clustering

- Clustering is a technique for finding **similarity groups** in data, called **clusters**. I.e.,
 - it groups data instances that are similar to (near) each other in one cluster and data instances that are very different (far away) from each other into different clusters.
- Clustering is often called an **unsupervised learning** task as no class values denoting an *a priori* grouping of the data instances are given, which is the case in supervised learning.
- Due to historical reasons, clustering is often considered synonymous with unsupervised learning.
 - In fact, association rule mining is also unsupervised

Clustering



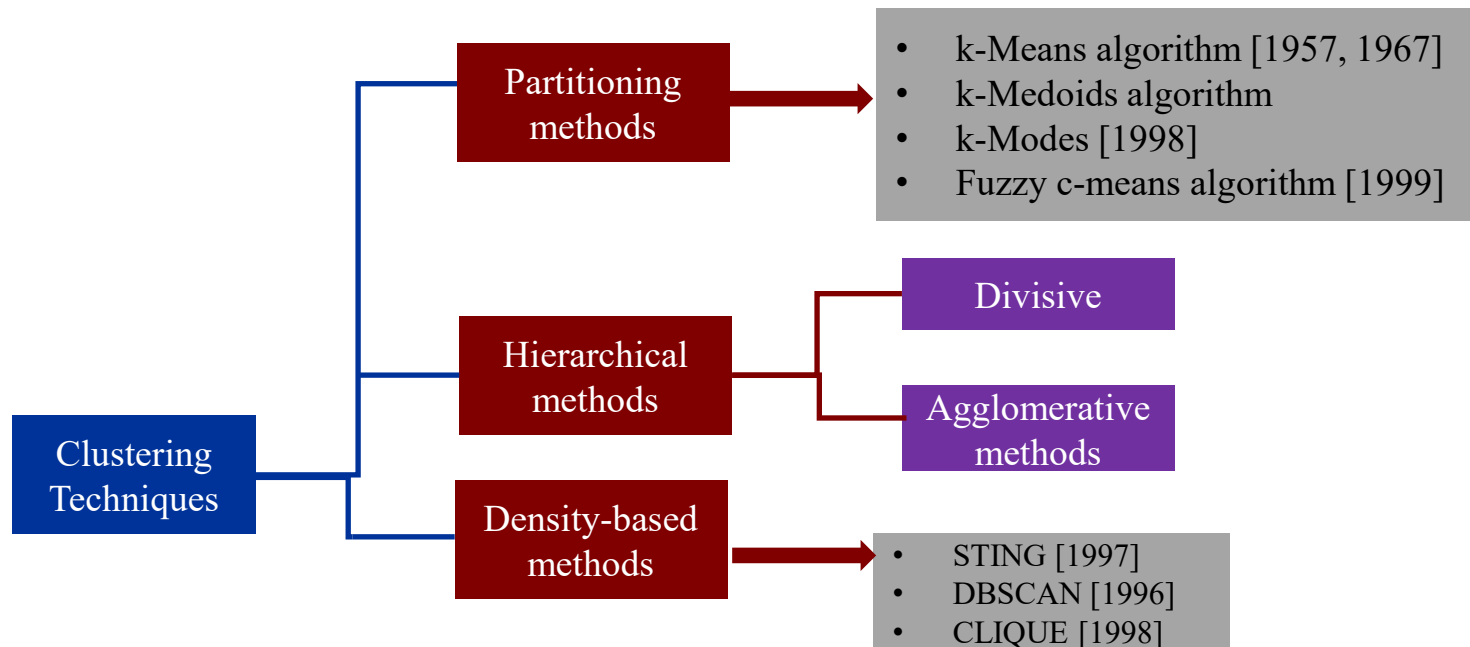
The goal of clustering is to

- group data points that are close (or similar) to each other
- identify such groupings (or clusters) in an unsupervised manner

How to define similarity ?

Clustering techniques

- **Clustering has been studied extensively for more than 40 years and across many disciplines due to its broad applications.**
- we shall cover the following clustering techniques.



Common Distance measures:

- *Distance measure* will determine how the *similarity* of two elements is calculated and it will influence the shape of the clusters.

They include:

1. The [Euclidean distance](#) (also called L2-norm distance) is given by:

$$d(x, C) = \sqrt{(x_H - C_H)^2 + (x_W - C_W)^2}$$

2. The [Manhattan distance](#) (also called taxicab norm or L1-norm) is given by:

$$d(x, y) = \sum_{i=1}^p |x_i - y_i|$$

3. The maximum norm is given by:

$$d(x, y) = \max_{1 \leq i \leq p} |x_i - y_i|$$

4. The Mahalanobis distance corrects data for different scales and correlations in the variables.
5. Inner product space: The angle between two vectors can be used as a distance measure when clustering high dimensional data.
6. Hamming distance (sometimes edit distance) measures the minimum number of substitutions required to change one member into another.

Applications

Example 1: groups people of similar sizes together to make “small”, “medium” and “large” T-Shirts.

- Tailor-made for each person: too expensive
- One-size-fits-all: does not fit all.

Example 2: In marketing, segment customers according to their similarities

- To do targeted marketing.

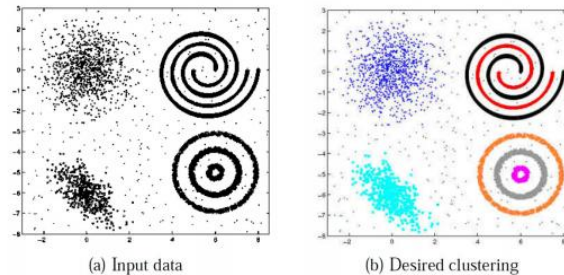
Example 3: Given a collection of text documents, we want to organize them according to their content similarities,

- To produce a topic hierarchy

Let's dive in...

In some cases, we may not know the right number of clusters in the data and may want to learn that (technique exists for doing this but beyond the scope)

- Given: N unlabeled examples $x_1, x_2, \dots, x_N$; desired no. of partitions K
- Goal: Group the examples into K “homogeneous” partitions



Picture courtesy: “Data Clustering: 50 Years Beyond K-Means”, A.K. Jain (2008)

- Loosely speaking, it is classification without ground truth labels
- A good clustering is one that achieves
 - High within-cluster similarity
 - Low inter-cluster similarity

Similarity can be Subjective

- Clustering only looks at similarities b/w inputs, since no labels are given
- Without labels, similarity can be hard to define



- Thus using the right distance/similarity is very important in clustering
- In some sense, related to asking: “Clustering based on what”?

Clustering: Some Examples

- Document/Image/Webpage Clustering
- Image Segmentation (clustering pixels)



- Clustering web-search results
- Clustering (people) nodes in (social) networks/graphs
- .. and many more..

Initializing K-means.

- Initializing K-means is the initial step of the K-means clustering algorithm, which is an unsupervised machine learning technique used for partitioning data points into K clusters.
- The goal of K-means is to minimize the sum of squared distances between data points and their corresponding cluster centers, often referred to as centroids.

General process for initializing K-means

1. **Choose the number of clusters (K):** Decide the number of clusters you want to partition your data into. This is a user-specified parameter and requires domain knowledge or experimentation to determine the optimal value.
2. **Select initial centroids:** Randomly select K data points from your dataset as the initial centroids. These data points will serve as the starting positions for the cluster centers.
3. **Assign data points to clusters:** Calculate the distance between each data point and all the centroids. Assign each data point to the cluster with the nearest centroid. This step forms the initial clustering.

Cont.

4. **Update centroids:** Calculate the mean (center) of the data points within each cluster and move the centroids to these new positions.

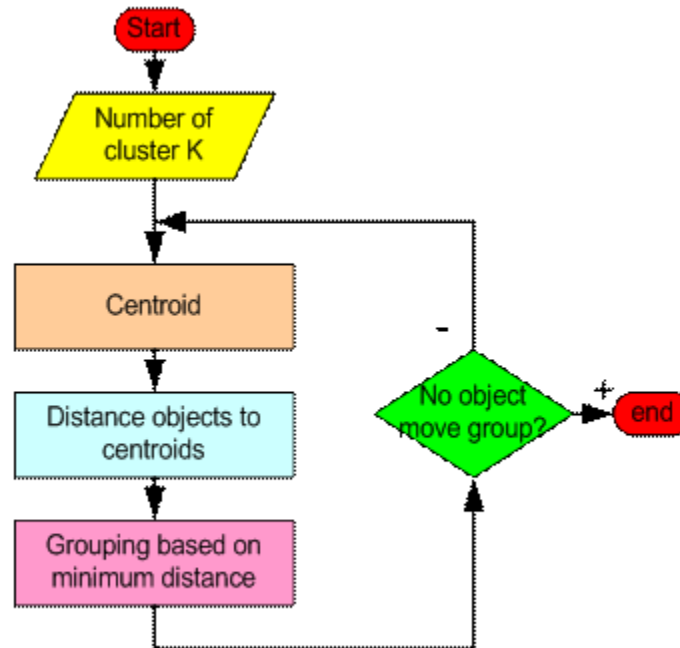
5. Repeat steps 3 and 4: Continue steps 3 and 4 iteratively until convergence or until a specified number of iterations is reached. Convergence happens when the cluster assignments stop changing significantly or when the centroids stop moving substantially.

6. Final clustering: Once **convergence is reached**, the algorithm stops, and you have your final clustering result.

K-Means Algorithm

- **K-Means clustering algorithm proposed by J. Hartigan and M. A. Wong [1979].**
- **Given a set of n distinct objects, the k-Means clustering algorithm partitions the objects into k number of clusters such that **intracluster similarity is high but the intercluster similarity is low.****
- **In this algorithm, user has to specify k , the number of clusters and consider the objects are defined with numeric attributes and thus using any one of the distance metric to demarcate the clusters.**

What have we learned so far...



- **Step 1:** Begin with a decision on the value of k = number of clusters .
- **Step 2:** Put any initial partition that classifies the data into k clusters. You may assign the training samples randomly, or systematically as the following:
 1. Take the first k training sample as single-element clusters
 2. Assign each of the remaining $(N-k)$ training sample to the cluster with the nearest centroid. After each assignment, recompute the centroid of the gaining cluster.

- **Step 3:** Take each sample in sequence and compute its [distance](#) from the centroid of each of the clusters. If a sample is not currently in the cluster with the closest centroid, switch this sample to that cluster and update the centroid of the cluster gaining the new sample and the cluster losing the sample.
- **Step 4:** Repeat step 3 until convergence is achieved, that is until a pass through the training sample causes no new assignments.

Apply Kmeans clustering to given set of data –
Assume C1 and C2 as initial centroids

SR_no	Age	Amount
C1	20	500
C2	40	1000
C3	30	800
C4	18	300
C5	28	1200
C6	35	1400
C7	45	1800

K-means clustering algorithm workflow and starts an example calculation.

The data provided in the table is:

Let's use the initial cluster centroids ,assuming we are targeting K=2 clusters.

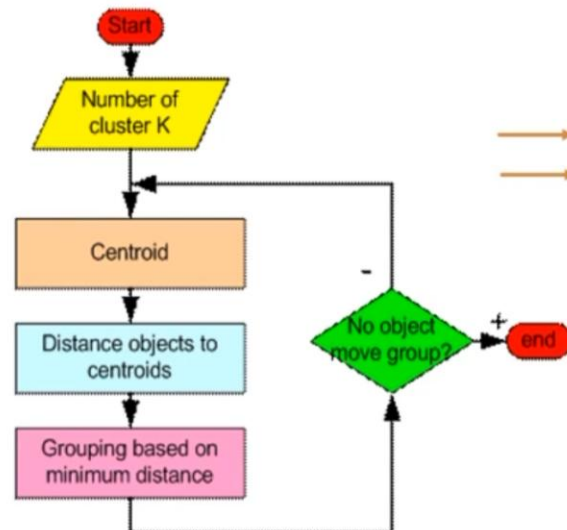
K-means Mathematical Example (First Iteration)

1. Initialization (K=2)

Initial Centroids:

- Cluster 1 Centroid (C1): **(185, 72)** (The first data point)
- Cluster 2 Centroid (C2): **(170, 56)** (The second data point)

Height (H)	Weight (W)
185	72
170	56
168	60
179	68
182	72
188	77
180	71
180	70
183	84
180	88
180	67
177	76



Height	Weight
185	72
170	56
168	60
179	68
182	72
188	77
180	71
180	70
183	84
180	88
180	67
177	76

Cluster 1 (185, 72)

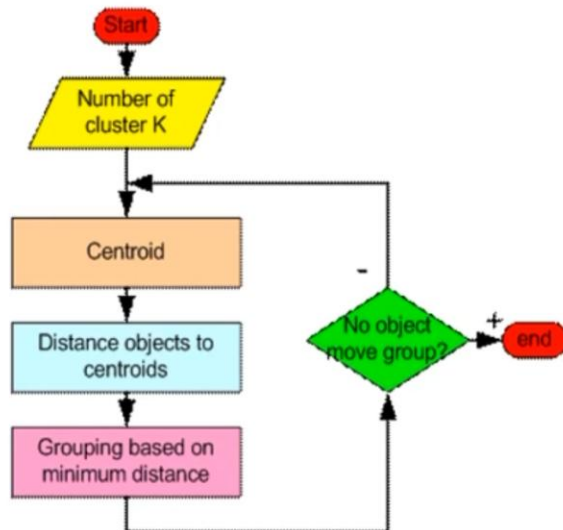
Cluster 2 (170, 56)

Euclidean Distance

$$\sqrt{(x_H - C_1)^2 + (x_w - C_2)^2}$$

$$\sqrt{(168 - 185)^2 + (60 - 72)^2} = 20.80$$

$$\sqrt{(168 - 170)^2 + (60 - 56)^2} = 4.48$$



Height	Weight
185	72
170	56
168	60
179	68
182	72
188	77
180	71
180	70
183	84
180	88
180	67
177	76

Cluster 1 $\left(\left(\frac{185 + 179}{2}, \frac{72 + 68}{2} \right) \right)$

Cluster 2 $\left(\left(\frac{170 + 168}{2}, \frac{56 + 60}{2} \right) \right) \{2, 3\}$
 (169, 58)

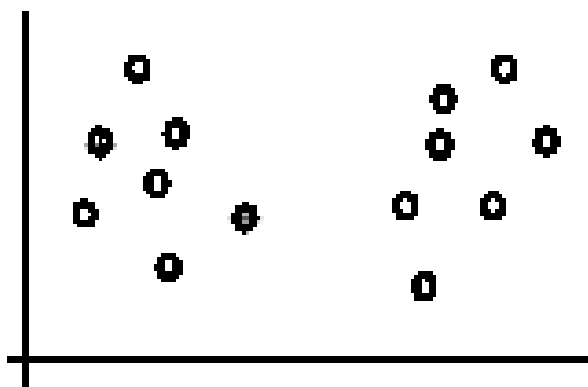
$$\sqrt{(179 - 185)^2 + (68 - 72)^2}$$

$$7.21$$

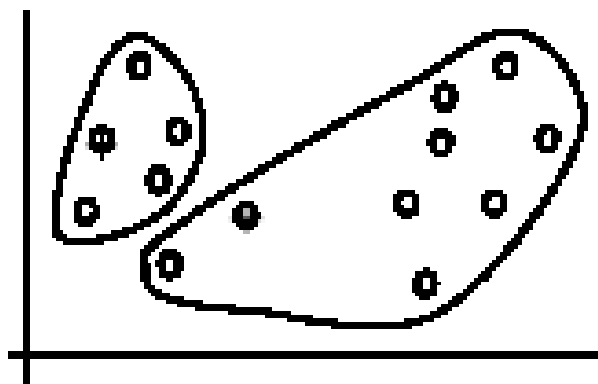
$$\sqrt{(179 - 169)^2 + (68 - 58)^2}$$

$$= 14.14$$

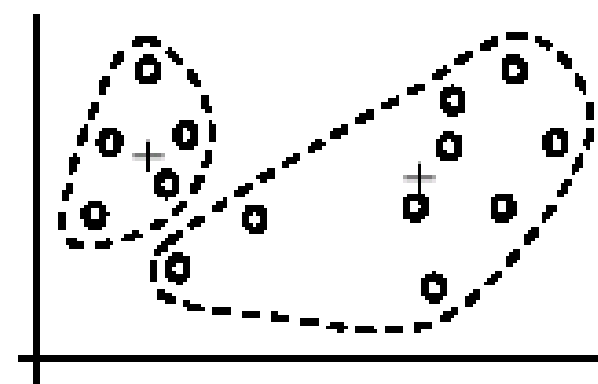
An Example



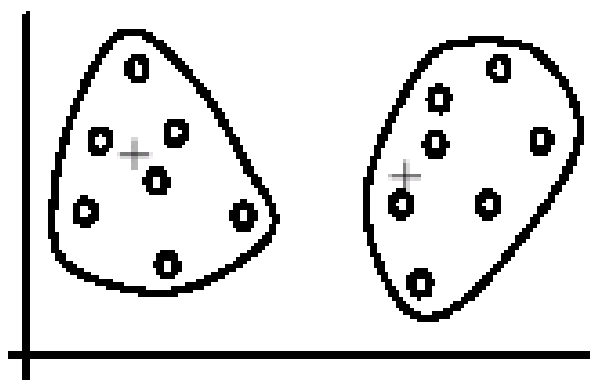
(A). Random selection of k centers



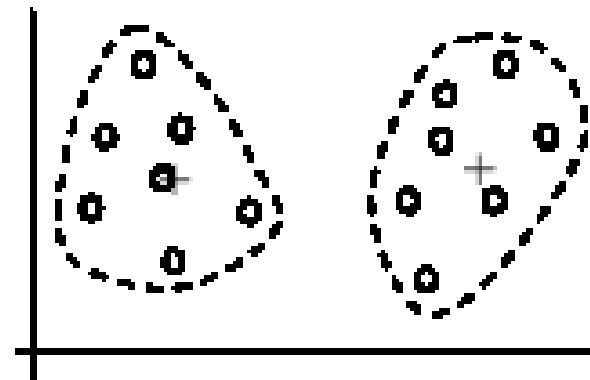
Iteration 1: (B). Cluster assignment



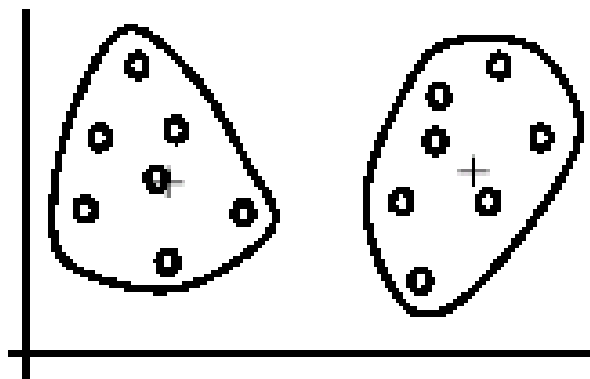
(C). Re-compute centroids



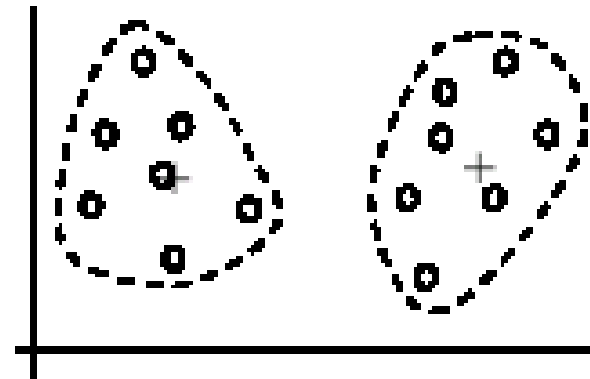
Iteration 2: (D). Cluster assignment



(E). Re-compute centroids



Iteration 3: (F). Cluster assignment



(G). Re-compute centroids

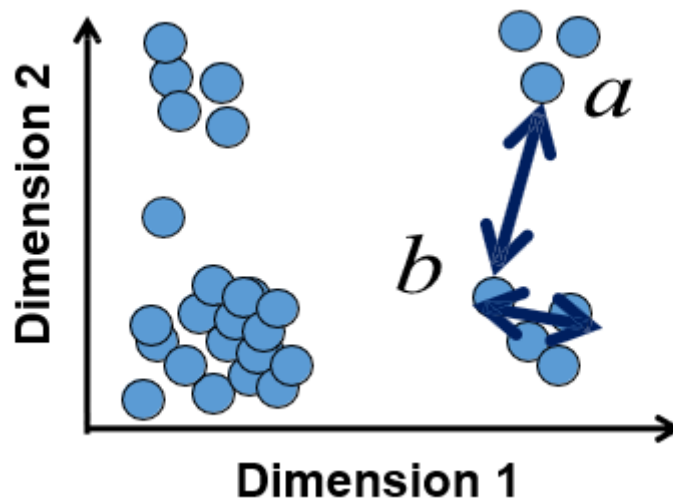
Cluster quality

- **Intra-cluster cohesion (compactness):**

- Cohesion measures how near the data points in a cluster are to the cluster centroid.
- Sum of squared error (SSE) is a commonly used measure.

- **Inter-cluster separation (isolation):**

- Separation means that different cluster centroids should be far away from one another.



Solve the Problem using K-Means Clustering

- Cluster the following eight points (with (x, y) representing locations) into three clusters:
- A1(2, 10), A2(2, 5), A3(8, 4), A4(5, 8), A5(7, 5), A6(6, 4), A7(1, 2), A8(4, 9)
- Initial cluster centers are: A1(2, 10), A4(5, 8) and A7(1, 2).

Algorithm Implementation

```
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Data
points = np.array([[2,10], [2,5], [8,4], [5,8], [7,5], [6,4], [1,2], [4,9]])
point_names = ['A1', 'A2', 'A3', 'A4', 'A5', 'A6', 'A7', 'A8']
initial_centers = np.array([[2,10], [5,8], [1,2]])
```



```
# KMeans clustering
kmeans = KMeans(n_clusters=3, init=initial_centers, n_init=1)
kmeans.fit(points)

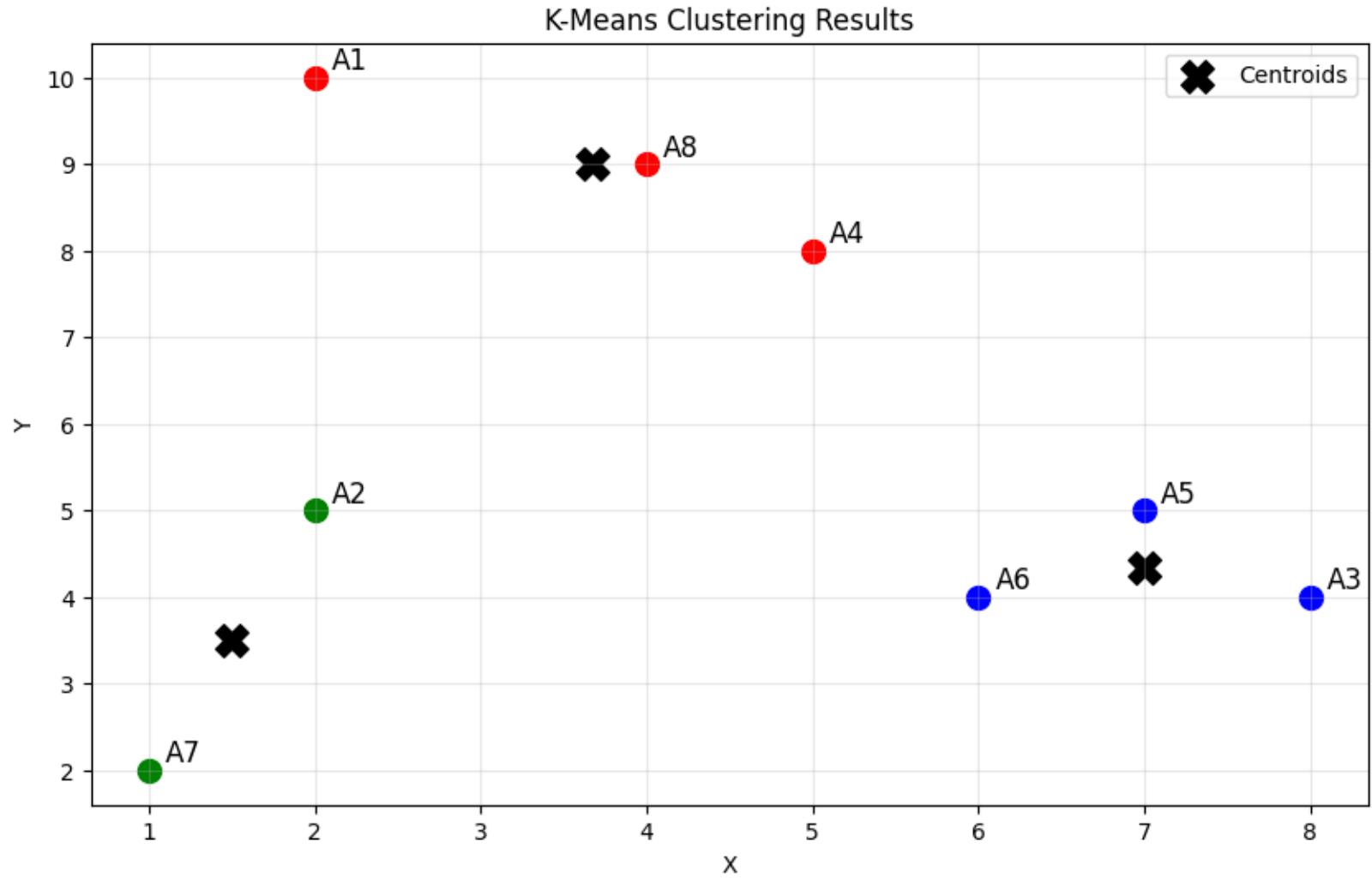
# Visualization
plt.figure(figsize=(10, 6))
colors = ['red', 'blue', 'green']

# Plot points
for i in range(len(points)):
    cluster = kmeans.labels_[i]
    plt.scatter(points[i, 0], points[i, 1], c=colors[cluster], s=100)
    plt.text(points[i, 0]+0.1, points[i, 1]+0.1, point_names[i], fontsize=12)

# Plot cluster centers
plt.scatter(kmeans.cluster_centers_[0, 0], kmeans.cluster_centers_[0, 1],
           c='black', marker='X', s=200, label='Centroids')
```

```
plt.xlabel('X')
plt.ylabel('Y')
plt.title('K-Means Clustering Results')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# Print detailed results
print("DETAILED RESULTS:")
for cluster_id in range(3):
    indices = np.where(kmeans.labels_ == cluster_id)[0]
    cluster_points = [point_names[i] for i in indices]
    print(f"\nCluster {cluster_id}: {cluster_points}")
    print(f"Center: ({kmeans.cluster_centers_[cluster_id, 0]:.2f}, {kmeans.cluster_centers_[cluster_id, 1]:.2f})")
    for idx in indices:
        print(f" {point_names[idx]}{tuple(points[idx])}")
```



- Write a program to implement Kmeans clustering on any dataset from UCI ML repository/Kaggle.

4-data point example for $k=2$ with 2 iterations of the k-means algorithm.

Step 1: Define Data Points

Let's use 4 points in 2D space:

- $P1 = (1, 1)$
- $P2 = (1, 2)$
- $P3 = (3, 4)$
- $P4 = (4, 5)$

Step 2: Initialization ($k=2$)

Choose initial centroids (often random or heuristic):

- $C1 = (1, 1)$ (assigned to $P1$)
- $C2 = (1, 2)$ (assigned to $P2$)

4-data point example for k=2 with 2 iterations of the k-means algorithm.

Iteration 1

1. Assign Points to Nearest Centroid

Compute distances (Euclidean):

Point	Dist to C1	Dist to C2	Assignment
P1	0	1	Cluster 1
P2	1	0	Cluster 2
P3	$\sqrt{(3-1)^2 + (4-1)^2} \approx 3.61$	$\sqrt{(3-1)^2 + (4-2)^2} \approx 2.83$	Cluster 2
P4	$\sqrt{(4-1)^2 + (5-1)^2} = 5$	$\sqrt{(4-1)^2 + (5-2)^2} \approx 4.24$	Cluster 2

4-data point example for $k=2$ with 2 iterations of the k-means algorithm.

Clusters after Assignment:

- Cluster 1: {P1}
- Cluster 2: {P2, P3, P4}

2. Update Centroids

- New $C1 = \text{mean}(P1) = (1, 1)$
- New $C2 = \text{mean}(P2, P3, P4) = \left(\frac{1+3+4}{3}, \frac{2+4+5}{3}\right) = (2.67, 3.67)$

Iteration 2

1. Assign Points to Nearest Centroid

Compute distances to updated centroids:

- $C1 = (1, 1), C2 = (2.67, 3.67)$

4-data point example for $k=2$ with 2 iterations of the k-means algorithm.

Point	Dist to C1	Dist to C2	Assignment
P1	0	$\sqrt{(1 - 2.67)^2 + (1 - 3.67)^2} \approx 3.14$	Cluster 1
P2	1	$\sqrt{(1 - 2.67)^2 + (2 - 3.67)^2} \approx 2.36$	Cluster 1
P3	3.61	$\sqrt{(3 - 2.67)^2 + (4 - 3.67)^2} \approx 0.47$	Cluster 2
P4	5	$\sqrt{(4 - 2.67)^2 + (5 - 3.67)^2} \approx 1.89$	Cluster 2

Clusters after Assignment:

- Cluster 1: {P1, P2}
- Cluster 2: {P3, P4}

2. Update Centroids

- New $C1 = \text{mean}(P1, P2) = (1, 1.5)$
- New $C2 = \text{mean}(P3, P4) = (3.5, 4.5)$

4-data point example for $k=2$ with 2 iterations of the k-means algorithm.

Final Result after 2 Iterations

- Cluster 1: P1, P2
- Cluster 2: P3, P4
- Centroids: $C1 = (1, 1.5)$, $C2 = (3.5, 4.5)$

The algorithm has converged to stable clusters after 2 iterations.

K-Means Algorithm Complexity

Complexity analysis of k-Means algorithm

Let us analyse the time and space complexities of k-Means algorithm.

Time complexity:

The time complexity of the k-Means algorithm can be expressed as

$$T(n) = O(n \times m \times k \times l)$$

where n = number of objects

m = number of attributes in the object definition

k = number of clusters

l = number of iterations.

Thus, time requirement is a linear order of number of objects and the algorithm runs in a modest time if $k \ll n$ and $l \ll n$ (the iteration can be moderately controlled to check the value of l).

Comments on k-Means algorithm

Space complexity: The storage complexity can be expressed as follows.

It requires $n \times m$ space to store the objects and $n \times k$ space to store the proximity measure from n objects to the centroids of k clusters.

Thus the total storage complexity is

$$S(n) = O(n \times (m + k))$$

That is, space requirement is in the linear order of n if $k \ll n$.

Advantages

- K-Means is simple and can be used for a wide variety of object types.
- It is also efficient both from storage requirement and execution time point of views. By saving distance information from one iteration to the next, the actual number of distance calculations, that must be made can be reduced (specially, as it reaches towards the termination).

How similarity metric can be utilized to run k-Means faster? What is the updation in each iteration?

Limitations

- The K-Means is not suitable for all types of data. For example, k-Means does not work on categorical data because mean cannot be defined.
- K-means finds a local optima and may actually minimize the global optimum.
- K-means has trouble clustering data that contains outliers. When the SSE is used as objective function, outliers can unduly influence the cluster that are produced. More precisely, in the presence of outliers, the cluster centroids, in fact, not truly as representative as they would be otherwise. It also influence the SSE measure as well.
- K-Means algorithm cannot handle non-globular clusters, clusters of different sizes and densities
- K-Means algorithm not really beyond the scalability issue (and not so practical for large databases).

Thanks



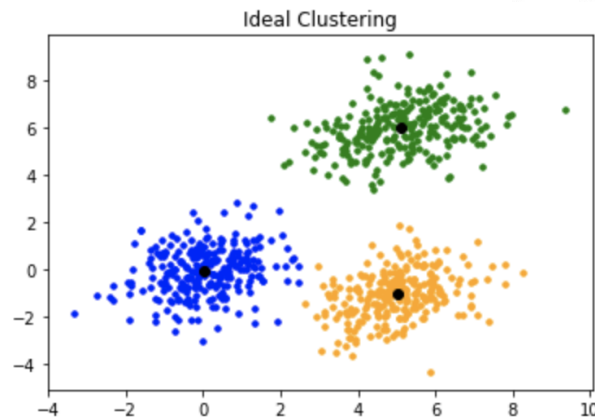
Kmeans Variants

Limitation of KMeans

- It is sensitive to the initialization of the centroids or the mean points. So, if a centroid is initialized to be a “far-off” point, it might just end up with no points associated with it, and at the same time, more than one cluster might end up linked with a single centroid.
- More than one centroid might be initialized into the same cluster resulting in poor clustering.

- A poor initialization of centroids resulted in poor clustering.

- This is how the clustering should have been:



Kmeans++

- KMeans++ is an improved initialization technique for the KMeans clustering algorithm.
- The primary goal of KMeans++ is to provide a better initial set of cluster centroids, which in turn leads to faster convergence and higher-quality clustering results compared to the standard random initialization used in traditional KMeans.

Kmeans++ Algorithm

- **Select the First Centroid:**

- Choose one data point randomly from the dataset as the first centroid.

- **Calculate Distances:**

- For each data point that has not been selected as a centroid, calculate the distance (usually using Euclidean distance) from that point to the nearest centroid that has already been chosen.

- **Select Subsequent Centroids:**

- The next centroid is chosen from the remaining data points with a probability proportional to the square of the distance from the nearest existing centroid. This ensures that points that are far from existing centroids are more likely to be selected as new centroids.

- **Repeat Step 2 and 3:**

- Repeat the distance calculation and centroid selection process until the desired number of centroids is reached.

Kmeans++ Numerical

- Let's say we have a small dataset of two-dimensional points:
- Data Points:
- $(2, 3), (3, 2), (8, 8), (10, 9), (11, 10)$
- We want to perform KMeans clustering with $K=2$ clusters.

Step 1

- **Select First Centroid:**
- Choose a random data point as the first centroid. Let's say we choose (2, 3).

Step 2

- **Calculate Distances:**
- Calculate the squared distances from each data point to the chosen centroid (2, 3):
 - Distance from (3, 2) = $(3 - 2)^2 + (2 - 3)^2 = 2$
 - Distance from (8, 8) = $(8 - 2)^2 + (8 - 3)^2 = 61$
 - Distance from (10, 9) = $(10 - 2)^2 + (9 - 3)^2 = 100$
 - Distance from (11, 10) = $(11 - 2)^2 + (10 - 3)^2 = 130$

Step 3

- **Select Subsequent Centroid:**
- The next centroid is selected based on the **calculated squared distances**. The probability of selecting a point as the next centroid **is proportional to its squared distance** from the nearest existing centroid.
- Calculate the probabilities for each point:
 - Probability for (3, 2) = $2 / (2 + 61 + 100 + 130) \approx 0.0068$
 - Probability for (8, 8) = $61 / (2 + 61 + 100 + 130) \approx 0.208$
 - Probability for (10, 9) = $100 / (2 + 61 + 100 + 130) \approx 0.341$
 - Probability for (11, 10) = $130 / (2 + 61 + 100 + 130) \approx 0.443$
- Choose the next centroid based on these probabilities. For simplicity, let's assume (11,10) is selected as the second centroid. #selected as probability is highest.

Step 4

- **Repeat Steps 2 and 3:**
- Recalculate distances from each data point to the two centroids (2, 3) and (11,10) and calculate new probabilities.
- Choose the third centroid based on the probabilities.

- Implement Kmeans++ algorithm on make_blobs dataset using and without using Kmeans library.

KMedians

- "KMedians," another clustering algorithm that is similar to KMeans but **uses medians** instead of means to calculate the cluster centroids.
- Medians is particularly useful when dealing with data that might have **outliers** or when you want to create clusters based on the median values of the data points rather than the mean values.

Kmedians Algorithm

- **Initialization:**

- Just like KMeans, KMedians also requires an initial set of **cluster centroids**. These centroids can be randomly chosen data points or selected using other methods.

- **Assignment Step:**

- Each data point is assigned to the nearest centroid based on the **Manhattan distance** (also known as L1 distance) rather than Euclidean distance, which is used in KMeans. The Manhattan distance is the **sum of the absolute differences** between the coordinates of the data point and the centroid.

- **Update Step:**

- After the assignment step, the **medians of the data points** in each cluster are calculated. The median value of a set of numbers is the middle value when they are sorted in order. It's a more robust measure than the mean because it's less affected by outliers.

- **Iteration:**

- The assignment and update steps are repeated iteratively until **convergence**. In each iteration, data points are reassigned to the nearest centroids based on Manhattan distance, and then the centroids are updated using the medians of the data points in each cluster.

Advantages of KMedians

- The main advantage of KMedians over KMeans is its **robustness to outliers**.
- Outliers can significantly influence the mean-based centroids in KMeans, whereas medians are less affected by **extreme values**.
- However, KMedians can also be computationally more expensive than KMeans since calculating medians involves sorting data points.

Kmedians Numerical

- Let's consider a small dataset of one-dimensional points:
- Data Points: 2, 3, 7, 8, 10, 12, 15, 20, 25
- Assuming we want to perform KMedians clustering with $K=2$ clusters.

Step 1

- **Initialization:**
- Choose two initial medians randomly or using some other initialization method. Let's say we start with medians at 7 and 20.

Step 2

- **Assignment Step:**
- For each data point, calculate the Manhattan distance (L1 distance) from each of the cluster medians. Assign each data point to the nearest median's cluster.

Data

Point |Median 7|Median 20| Assigned Cluster

2		5		18		1
3		4		17		1
7		0		13		1
8		1		12		1
10		3		10		1
12		5		8		1
15		8		5		2
20		13		0		2
25		18		5		2

Step 3

- **Update Step (Calculation of Medians):**

- For each cluster, calculate the median of the data points in that cluster. The median is the middle value when the data points are sorted.

Cluster 1 Median: 7

Cluster 2 Median: 20

- Repeat the assignment and update steps until convergence. In each iteration, reassign data points to the nearest medians and update the medians based on the median of the data points in each cluster.

Example

- Let's consider a small dataset of one-dimensional points:
- Data Points: Points: (2, 4), (3, 6), (5, 8), (10, 12), (12, 14), (15, 18), (20, 25), (25, 30)
- Assuming we want to perform KMedians clustering with $K=2$ clusters.
- Suppose
- Cluster 1 centroid: (3, 6) Cluster 2 centroid: (15, 18)

- Implement Kmedian algorithm using make_blobs dataset

K-Medoids

- K-Medoids is particularly useful when dealing with non-numerical data or when you want clusters that are centered around actual data points.
- "K-Medoids," is a clustering algorithm similar to KMeans but uses actual data points as cluster representatives (medoids) instead of the mean or centroid.

K-Medoids

- **Initialization:**
 - Choose K initial data points as medoids (cluster representatives). These can be randomly selected data points or chosen using other methods.
- **Assignment Step:**
 - Each data point is assigned to the nearest medoid based on a distance metric (usually something like Manhattan distance, Euclidean distance, etc.).
- **Update Step:**
 - For each cluster, compute the **total cost (sum of distances)** of assigning each data point in the cluster to the medoid.
 - Select the data point with **the lowest total cost as the new medoid** for that cluster.
- **Iteration:**
 - Repeat the assignment and update steps until the medoids stabilize, which indicates convergence.
- The main advantage of K-Medoids is its **robustness to outliers** and its ability to **handle non-numerical data types**. Since the medoid is an actual data point, it **is less affected by extreme values or noise** in the data.

K-Medoid

- K-Medoids is particularly useful when dealing with non-numerical data or when you want clusters that are centered around actual data points.
- "K-Medoids," is a clustering algorithm similar to KMeans but uses actual data points as cluster representatives (medoids) instead of the mean or centroid.
- Medoids are representative objects of a data set or a cluster within a data set whose sum of dissimilarities to all the objects in the cluster is minimal.
- Medoids are similar in concept to means or centroids, but medoids are always restricted to be members of the data set.
- Medoids are most commonly used on data when a mean or centroid cannot be defined, such as graphs.

K-Medoid

1. First, we select K random data points from the dataset and use them as medoids.
2. Now, we will calculate the distance of each data point from the medoids. You can use any of the Euclidean, Manhattan distance, or squared Euclidean distance as the distance measure.
3. Once we find the distance of each data point from the medoids, we will assign the data points to the clusters associated with each medoid. The data points are assigned to the medoids at the closest distance.
4. After determining the clusters, we will calculate the sum of the distance of all the non-medoid data points to the medoid of each cluster. Let the cost be C_i .

5. Now, we will select a random data point D_j from the dataset and swap it with a medoid M_i . Here, D_j becomes a temporary medoid. After swapping, we will calculate the distance of all the non-medoid data points to the current medoid of each cluster. Let this cost be C_j .
6. If $C_i > C_j$, the current medoids with D_j as one of the medoids are made permanent medoids. Otherwise, we undo the swap, and M_i is reinstated as the medoid.
7. Repeat 4 to 6 until no change occurs in the clusters.

K-Medoids Numerical with $m_1=(3,4)$, $m_2 = (7,3)$

Now, we will calculate the distance between each data point and the medoids using the Manhattan distance

Point	Coordinates
A1	(2, 6)
A2	(3, 8)
A3	(4, 7)
A4	(6, 2)
A5	(6, 4)
A6	(7, 3)
A7	(7,4)
A8	(8, 5)
A9	(7, 6)
A10	(3, 4)

- $M1 = (3, 4)$
- $M2 = (7, 3)$

Now, we will calculate the distance between each data point and the medoids using the Manhattan distance measure. The results have been tabulated as follows.

Point	Coordinates	Distance From M1 (3,4)	Distance from M2 (7,3)	Assigned Cluster
A1	(2, 6)	3	8	Cluster 1
A2	(3, 8)	4	9	Cluster 1
A3	(4, 7)	4	7	Cluster 1
A4	(6, 2)	5	2	Cluster 2
A5	(6, 4)	3	2	Cluster 2
A6	(7, 3)	5	0	Cluster 2
A7	(7,4)	4	1	Cluster 2
A8	(8, 5)	6	3	Cluster 2
A9	(7, 6)	6	3	Cluster 2
A10	(3, 4)	0	5	Cluster 1

The clusters made with medoids (3, 4) and (7, 3) are as follows.

- Points in cluster1= {(2, 6), (3, 8), (4, 7), (3, 4)}
- Points in cluster 2= {(7,4), (6,2), (6, 4), (7,3), (8,5), (7,6)}

After assigning clusters, we will calculate the cost for each cluster and find their sum. The cost is nothing but the sum of distances of all the data points from the medoid of the cluster they belong to.

Hence, the cost for the current cluster will be $3+4+4+2+2+0+1+3+3+0=22$.

Iteration 2

Now, we will select another non-medoid point (7, 4) and make it a temporary medoid for the second cluster.

Hence,

- $M1 = (3, 4)$
- $M2 = (7, 4)$

Now, let us calculate the distance between all the data points and the current medoids.

Point	Coordinates	Distance From M1 (3,4)	Distance from M2 (7,4)	Assigned Cluster
A1	(2, 6)	3	7	Cluster 1
A2	(3, 8)	4	8	Cluster 1
A3	(4, 7)	4	6	Cluster 1
A4	(6, 2)	5	3	Cluster 2
A5	(6, 4)	3	1	Cluster 2
A6	(7, 3)	5	1	Cluster 2
A7	(7,4)	4	0	Cluster 2
A8	(8, 5)	6	2	Cluster 2
A9	(7, 6)	6	2	Cluster 2
A10	(3, 4)	0	4	Cluster 1

Iteration 2

The data points haven't changed in the clusters after changing the medoids. Hence, clusters are:

- Points in cluster1: {(2, 6), (3, 8), (4, 7), (3, 4)}
- Points in cluster 2: {(7,4), (6,2), (6, 4), (7,3), (8,5), (7,6)}

Now, let us again calculate the cost for each cluster and find their sum. The total cost this time will be $3+4+4+3+1+1+0+2+2+0=20$.

Here, the current cost is less than the cost calculated in the previous iteration. Hence, we will make the swap permanent and make (7,4) the medoid for cluster 2. If the cost this time was greater than the previous cost i.e. 22, we would have to revert the change. New medoids after this iteration are (3, 4) and (7, 4) with no change in the clusters.

Iteration 3

Now, let us again change the medoid of cluster 2 to (6, 4). Hence, the new medoids for the clusters are $M1=(3, 4)$ and $M2=(6, 4)$.

Let us calculate the distance between the data points and the above medoids to find the new cluster. The results have been tabulated as follows.

Point	Coordinates	Distance From M1 (3,4)	Distance from M2 (6,4)	Assigned Cluster
A1	(2, 6)	3	6	Cluster 1
A2	(3, 8)	4	7	Cluster 1
A3	(4, 7)	4	5	Cluster 1
A4	(6, 2)	5	2	Cluster 2
A5	(6, 4)	3	0	Cluster 2
A6	(7, 3)	5	2	Cluster 2
A7	(7,4)	4	1	Cluster 2
A8	(8, 5)	6	3	Cluster 2
A9	(7, 6)	6	3	Cluster 2
A10	(3, 4)	0	3	Cluster 1

Iteration 3

Again, the clusters haven't changed. Hence, clusters are:

- Points in cluster1: $\{(2, 6), (3, 8), (4, 7), (3, 4)\}$
- Points in cluster 2: $\{(7,4), (6,2), (6, 4), (7,3), (8,5), (7,6)\}$

Now, let us again calculate the cost for each cluster and find their sum. The total cost this time will be $3+4+4+2+0+2+1+3+3+0=22$.

The current cost is 22 which is greater than the cost in the previous iteration i.e. 20. Hence, we will revert the change and the point (7, 4) will again be made the medoid for cluster 2.

So, the clusters after this iteration will be cluster1 = $\{(2, 6), (3, 8), (4, 7), (3, 4)\}$ and cluster 2 = $\{(7,4), (6,2), (6, 4), (7,3), (8,5), (7,6)\}$. The medoids are (3,4) and (7,4).

We keep replacing the medoids with a non-medoid data point. The set of medoids for which the cost is the least, the medoids, and the associated clusters are made permanent. So, after all the iterations, you will get the final clusters and their medoids.

The K-Medoids clustering algorithm is a computation-intensive algorithm that requires many iterations. In each iteration, we need to calculate the distance between the medoids and the data points, assign clusters, and compute the cost. Hence, K-Medoids clustering is not well suited for large data sets.

K-MEDIAN

I think the median
is a good choice.

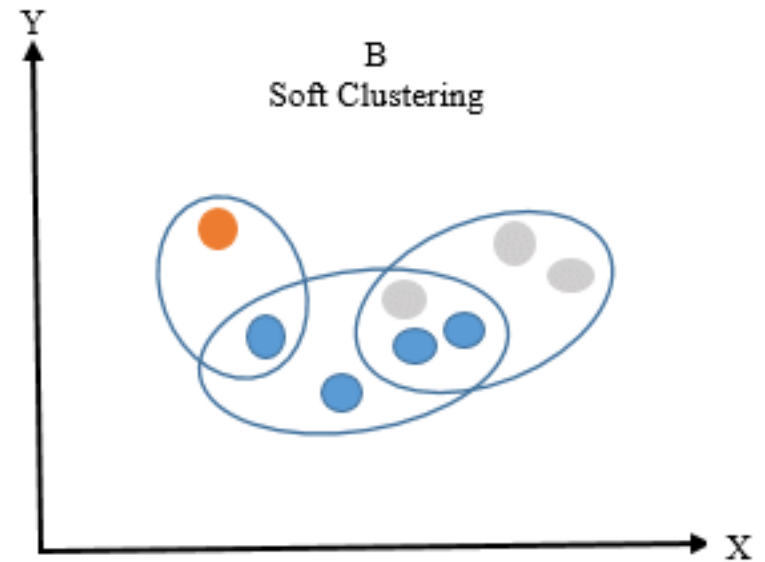
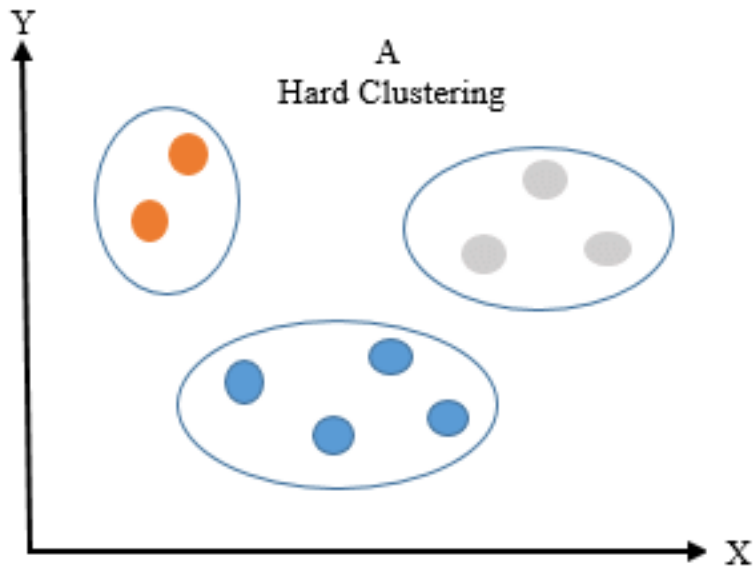


K-MEDOID

I think the medoid
is a better option.



Hard vs Soft Clustering



Hard Clustering

- Also known as Exclusive Clustering or Crisp Clustering.
- Each data point is assigned to **exactly one cluster.**
- Results in non-overlapping clusters with clear boundaries.
- E.g., K-means

Advantages

- Simplicity and Ease of Implementation: Hard clustering algorithms are straightforward to understand and implement.
- Computational Efficiency: Ideal for handling large datasets efficiently.
- Clear Cluster Membership: Each data point unambiguously belongs to a single cluster.

Disadvantages

- Sensitive to Initial Placement: Results can vary depending on the initial cluster centroids.
- Limited Handling of Overlapping Data: May struggle with complex data structures that have overlapping clusters.
- Impact of Outliers: Outliers can significantly affect cluster assignments.

Soft Clustering

- Also known as Fuzzy Clustering.
- Allows data points to **belong to multiple clusters** with varying membership degrees.
- Provides more flexibility and captures uncertainty in cluster assignments.
- E.g., Fuzzy C-means (FCM)

Advantages

- Nuanced Cluster Assignments: Data points can have partial membership in multiple clusters, providing a more nuanced representation.
- Handling Overlapping Data: Well-suited for datasets with complex or overlapping structures.
- Robustness to outliers: The ability of a clustering algorithm to minimize the influence of outliers on the assignment of data points to clusters. (probability distribution over clusters instead of a hard assignment to a single cluster)

Disadvantages

- Computational Complexity: Soft clustering methods can be more computationally expensive than their hard clustering counterparts.
- Determining the Number of Clusters: Requires the pre-specification of the number of clusters or fuzziness coefficient.
- Interpretability Challenges: Fuzzy memberships might be more challenging to interpret.

How to Find the Optimal Number of Clusters

- The elbow method is a popular technique used to determine the optimal number of clusters (k) in a clustering algorithm, such as K-means.
- It involves plotting the cost (inertia) of clustering as a function of the number of clusters and looking for the "elbow point" in the plot.
- The elbow point is the value of k at which the inertia starts to level off or decrease at a slower rate.
- This point indicates that adding more clusters does not significantly improve the clustering quality and suggests the appropriate number of clusters for the data.

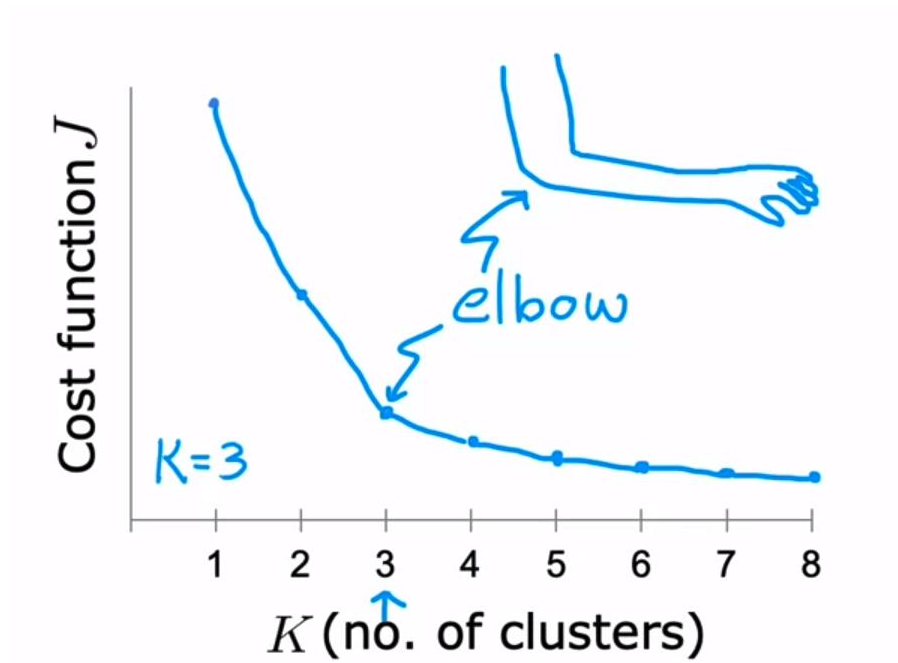
Cont.

- The **elbow method** helps us find the optimal number of clusters for our data.
- It involves analyzing the inertia (**within-cluster sum of squares**) as a function of the number of clusters.

Elbow Method: Step by Step

- Choose a range of k values to consider (e.g., 1 to 10).
- For each k value, run the K-means algorithm with k clusters on the data.
- Calculate the inertia (sum of squared distances within clusters) for each k .
- Plot the inertia values against the corresponding k values.

Cont.



Identifying the Elbow Point

- The elbow point is the optimal k value.
- It is the point where inertia starts to level off or decreases at a slower rate.
- Adding more clusters beyond this point may not significantly improve clustering quality.

Detailed Mathematical Solution: K-Means Clustering

We apply **K-Means Clustering** on a dataset with two features (Age, Amount). Initial centroids are given as C1(20, 500) and C2(40, 1000). Euclidean distance is used for clustering.

Point	Age	Amount
C1	20	500
C2	40	1000
C3	30	800
C4	18	300
C5	28	1200
C6	35	1400
C7	45	1800

Iteration 1:

Initial Centroids:

$Z1(0) = (20, 500)$, $Z2(0) = (40, 1000)$

Euclidean Distance Formula:

$$d(p, z) = \sqrt{(\text{Age} - \text{Age}_c)^2 + (\text{Amount} - \text{Amount}_c)^2}$$

Assignments after Iteration 1:

Cluster 1 $\rightarrow \{C1, C4\}$

Cluster 2 $\rightarrow \{C2, C3, C5, C6, C7\}$

New Centroids:

$Z1(1) = (19.0, 400.0)$

$Z2(1) = (35.6, 1240.0)$

Iteration 2:

$Z1(1) = (19.0, 400.0)$, $Z2(1) = (35.6, 1240.0)$

After recomputation of distances:

Cluster 1 $\rightarrow \{C1, C3, C4\}$

Cluster 2 $\rightarrow \{C2, C5, C6, C7\}$

New Centroids:

$Z1(2) = (22.67, 533.33)$

$Z2(2) = (37.0, 1350.0)$

Final Clusters:

Cluster A (C1, C3, C4): Centroid = (22.67, 533.33)

Cluster B (C2, C5, C6, C7): Centroid = (37.0, 1350.0)

Assignments remain unchanged $\rightarrow$ **Converged**.

Within-cluster SSE $\approx 476,907.33$

Note: Amount dominates due to scale difference. Feature scaling is recommended for balanced clustering.

Let's say we have a small dataset of two-dimensional points: Data Points: (2, 3), (3, 2), (8, 8), (10, 9), (11, 10) We want to perform KMeans++ clustering with K=2 clusters.

Step 1: Initial Center Selection (KMeans++)

First center: Choose randomly from the data points. Let's say we randomly select (2, 3).

Second center: Calculate distances from each point to the nearest center and select probabilistically based on squared distances.

Distances from (2, 3):

- (2, 3): 0
- (3, 2): $\sqrt{((3-2)^2 + (2-3)^2)} = \sqrt{2} \approx 1.414$
- (8, 8): $\sqrt{((8-2)^2 + (8-3)^2)} = \sqrt{61} \approx 7.810$
- (10, 9): $\sqrt{((10-2)^2 + (9-3)^2)} = \sqrt{100} = 10$
- (11, 10): $\sqrt{((11-2)^2 + (10-3)^2)} = \sqrt{130} \approx 11.402$

Squared distances: 0, 2, 61, 100, 130 Sum of squared distances: 293

Probabilities:

- (2, 3): $0/293 = 0$
- (3, 2): $2/293 \approx 0.0068$
- (8, 8): $61/293 \approx 0.2082$
- (10, 9): $100/293 \approx 0.3413$
- (11, 10): $130/293 \approx 0.4437$

Based on these probabilities, let's say we select (11, 10) as the second center.

Initial centers: (2, 3) and (11, 10)

Step 2: Assign Points to Clusters

Calculate distances to both centers:

Point	Distance to (2,3)	Distance to (11,10)	Assignment
(2,3)	0	$\sqrt{130} \approx 11.40$	Cluster 1
(3,2)	$\sqrt{2} \approx 1.41$	$\sqrt{128} \approx 11.31$	Cluster 1
(8,8)	$\sqrt{61} \approx 7.81$	$\sqrt{13} \approx 3.61$	Cluster 2
(10,9)	10	$\sqrt{2} \approx 1.41$	Cluster 2
(11,10)	$\sqrt{130} \approx 11.40$	0	Cluster 2

Cluster assignments:

- Cluster 1: (2, 3), (3, 2)

- Cluster 2: (8, 8), (10, 9), (11, 10)

Step 3: Update Centroids

Cluster 1 centroid: $((2+3)/2, (3+2)/2) = (2.5, 2.5)$

Cluster 2 centroid: $((8+10+11)/3, (8+9+10)/3) = (29/3, 27/3) \approx (9.67, 9)$

Step 4: Reassign Points

Calculate distances to new centroids:

Point	Distance to (2.5,2.5)	Distance to (9.67,9)	Assignment
(2,3)	$\sqrt{0.5} \approx 0.71$	$\sqrt{94.78} \approx 9.74$	Cluster 1
(3,2)	$\sqrt{0.5} \approx 0.71$	$\sqrt{93.45} \approx 9.67$	Cluster 1
(8,8)	$\sqrt{60.5} \approx 7.78$	$\sqrt{3.79} \approx 1.95$	Cluster 2
(10,9)	$\sqrt{98.5} \approx 9.92$	$\sqrt{0.11} \approx 0.33$	Cluster 2
(11,10)	$\sqrt{128.5} \approx 11.34$	$\sqrt{2.79} \approx 1.67$	Cluster 2

Final cluster assignments remain the same:

- **Cluster 1:** (2, 3), (3, 2)
- **Cluster 2:** (8, 8), (10, 9), (11, 10)

The algorithm has converged since no points changed clusters. The KMeans++ initialization successfully separated the data into two natural groups: the lower-left points and the upper-right points.

Dataset: 2, 3, 7, 8, 10, 12, 15, 20, 25

K: 2

Initial centroids: 7 and 20

Iteration 1

Step 1: Assign points to clusters based on Manhattan distance

Point	Distance to 7	Distance to 20	Assignment
2	5	18	Cluster 1
3	4	17	Cluster 1
7	0	13	Cluster 1
8	1	12	Cluster 1
10	3	10	Cluster 1
12	5	8	Cluster 1
15	8	5	Cluster 2
20	13	0	Cluster 2
25	18	5	Cluster 2

Cluster assignments:

- Cluster 1: 2, 3, 7, 8, 10, 12
- Cluster 2: 15, 20, 25

Step 2: Calculate new centroids (medians)

- Cluster 1 median: $(7 + 8)/2 = 7.5$
- Cluster 2 median: 20

Iteration 2

Step 1: Reassign points to clusters

Point	Distance to 7.5	Distance to 20	Assignment
2	5.5	18	Cluster 1
3	4.5	17	Cluster 1
7	0.5	13	Cluster 1
8	0.5	12	Cluster 1
10	2.5	10	Cluster 1
12	4.5	8	Cluster 1

Point	Distance to 7.5	Distance to 20	Assignment
15	7.5	5	Cluster 2
20	12.5	0	Cluster 2
25	17.5	5	Cluster 2

Cluster assignments (unchanged):

- Cluster 1: 2, 3, 7, 8, 10, 12
- Cluster 2: 15, 20, 25

Since the cluster assignments didn't change, the algorithm has converged.

Final Result

Final clusters:

- **Cluster 1:** 2, 3, 7, 8, 10, 12 with median = 7.5
- **Cluster 2:** 15, 20, 25 with median = 20

The clustering successfully separated the data into a lower-value group (2-12) and a higher-value group (15-25).

- Let's consider a small dataset of one-dimensional points:
- Data Points: Points: (2, 4), (3, 6), (5, 8), (10, 12), (12, 14), (15, 18), (20, 25), (25, 30)
- Assuming we want to perform KMedians clustering with $K=2$ clusters.
- Suppose
- Cluster 1 centroid: (3, 6) Cluster 2 centroid: (15, 18)

Step 1: Initial Setup

- **Cluster 1 centroid:** (3, 6)
- **Cluster 2 centroid:** (15, 18)
- **Distance metric:** Manhattan distance (L1 norm)

Step 2: Calculate Distances and Assign Points

Let's calculate Manhattan distances for each point:

Point (2, 4):

- To C1: $|2-3| + |4-6| = 1 + 2 = 3$
- To C2: $|2-15| + |4-18| = 13 + 14 = 27$
- **Assignment: Cluster 1**

Point (3, 6):

- To C1: $|3-3| + |6-6| = 0 + 0 = 0$
- To C2: $|3-15| + |6-18| = 12 + 12 = 24$
- **Assignment: Cluster 1**

Point (5, 8):

- To C1: $|5-3| + |8-6| = 2 + 2 = 4$
- To C2: $|5-15| + |8-18| = 10 + 10 = 20$
- **Assignment: Cluster 1**

Point (10, 12):

- To C1: $|10-3| + |12-6| = 7 + 6 = 13$
- To C2: $|10-15| + |12-18| = 5 + 6 = 11$
- **Assignment: Cluster 2**

Point (12, 14):

- To C1: $|12-3| + |14-6| = 9 + 8 = 17$
- To C2: $|12-15| + |14-18| = 3 + 4 = 7$
- **Assignment: Cluster 2**

Point (15, 18):

- To C1: $|15-3| + |18-6| = 12 + 12 = 24$
- To C2: $|15-15| + |18-18| = 0 + 0 = 0$
- **Assignment: Cluster 2**

Point (20, 25):

- To C1: $|20-3| + |25-6| = 17 + 19 = 36$
- To C2: $|20-15| + |25-18| = 5 + 7 = 12$
- **Assignment: Cluster 2**

Point (25, 30):

- To C1: $|25-3| + |30-6| = 22 + 24 = 46$
- To C2: $|25-15| + |30-18| = 10 + 12 = 22$
- **Assignment: Cluster 2**

Step 3: Update Centroids (Using Median)

Cluster 1 points: (2, 4), (3, 6), (5, 8)

- x-values: 2, 3, 5 $\rightarrow$ median = 3
- y-values: 4, 6, 8 $\rightarrow$ median = 6
- **New centroid:** (3, 6)

Cluster 2 points: (10, 12), (12, 14), (15, 18), (20, 25), (25, 30)

- x-values: 10, 12, 15, 20, 25 $\rightarrow$ median = 15
- y-values: 12, 14, 18, 25, 30 $\rightarrow$ median = 18
- **New centroid:** (15, 18)

Final Result

Since the centroids didn't change, the algorithm has converged:

- **Cluster 1:** (2, 4), (3, 6), (5, 8) with centroid (3, 6)
- **Cluster 2:** (10, 12), (12, 14), (15, 18), (20, 25), (25, 30) with centroid (15, 18)

The clustering successfully separated the data into two groups based on their positions in the 2D space.

Based on the given data points and initial medoids $m1=(3,4)$ and $m2=(7,3)$, the Manhattan distance is used to calculate the distance between each point and the medoids. The initial cluster assignment is performed by assigning each point to the closest medoid. After the initial assignment, new medoids are selected for each cluster by minimizing the total cost (sum of distances to all other points in the cluster).

Initial Distance Calculation and Cluster Assignment

Point	Coordinates	Distance to $m1=(3,4)$	Distance to $m2=(7,3)$	Closest Medoid	Cluster
A1	(2,6)	3	8	m1	1
A2	(3,8)	4	9	m1	1
A3	(4,7)	4	7	m1	1
A4	(6,2)	5	2	m2	2
A5	(6,4)	3	2	m2	2
A6	(7,3)	5	0	m2	2
A7	(7,4)	4	1	m2	2
A8	(8,5)	6	3	m2	2
A9	(7,6)	6	3	m2	2
A10	(3,4)	0	5	m1	1

Initial Clusters:

- Cluster 1: A1, A2, A3, A10
- Cluster 2: A4, A5, A6, A7, A8, A9

New Medoid Selection

Cluster 1 (Points: A1, A2, A3, A10)

- Cost for A1: distance to A2 + A3 + A10 = $3 + 3 + 3 = 9$
- Cost for A2: distance to A1 + A3 + A10 = $3 + 2 + 4 = 9$
- Cost for A3: distance to A1 + A2 + A10 = $3 + 2 + 4 = 9$
- Cost for A10: distance to A1 + A2 + A3 = $3 + 4 + 4 = 11$
- Minimum cost is 9, so new medoid can be A1, A2, or A3. We choose A2 as the new medoid for Cluster 1.

Cluster 2 (Points: A4, A5, A6, A7, A8, A9)

- Cost for A4: distance to A5 + A6 + A7 + A8 + A9 = $2 + 2 + 3 + 5 + 5 = 17$
- Cost for A5: distance to A4 + A6 + A7 + A8 + A9 = $2 + 2 + 1 + 3 + 3 = 11$
- Cost for A6: distance to A4 + A5 + A7 + A8 + A9 = $2 + 2 + 1 + 3 + 3 = 11$
- Cost for A7: distance to A4 + A5 + A6 + A8 + A9 = $3 + 1 + 1 + 2 + 2 = 9$
- Cost for A8: distance to A4 + A5 + A6 + A7 + A9 = $5 + 3 + 3 + 2 + 2 = 15$
- Cost for A9: distance to A4 + A5 + A6 + A7 + A8 = $5 + 3 + 3 + 2 + 2 = 15$

- Minimum cost is 9, so new medoid is A7 for Cluster 2.

New Medoids:

- Cluster 1: A2 (3,8)
- Cluster 2: A7 (7,4)

Reassignment with New Medoids

After reassignment with new medoids, the clusters remain the same because the points are closer to their respective medoids.

- Cluster 1: A1, A2, A3, A10
- Cluster 2: A4, A5, A6, A7, A8, A9

The algorithm converges as the medoids do not change further, and the clusters are stable.

Final Clusters and Medoids:

- Medoid for Cluster 1: A2 (3,8)
- Medoid for Cluster 2: A7 (7,4)
- Points in Cluster 1: A1, A2, A3, A10
- Points in Cluster 2: A4, A5, A6, A7, A8, A9



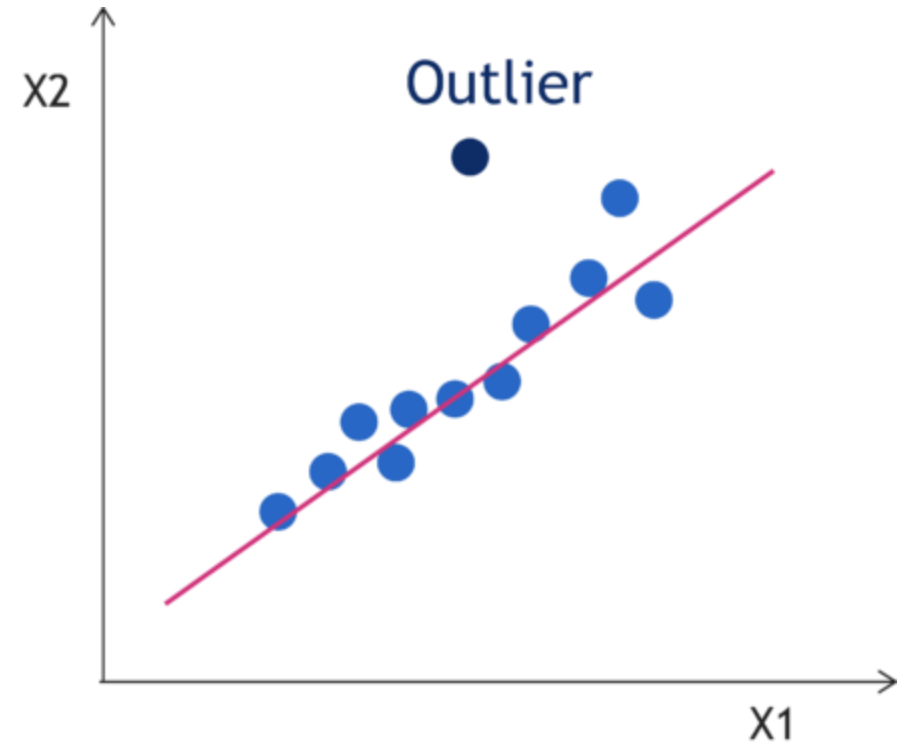
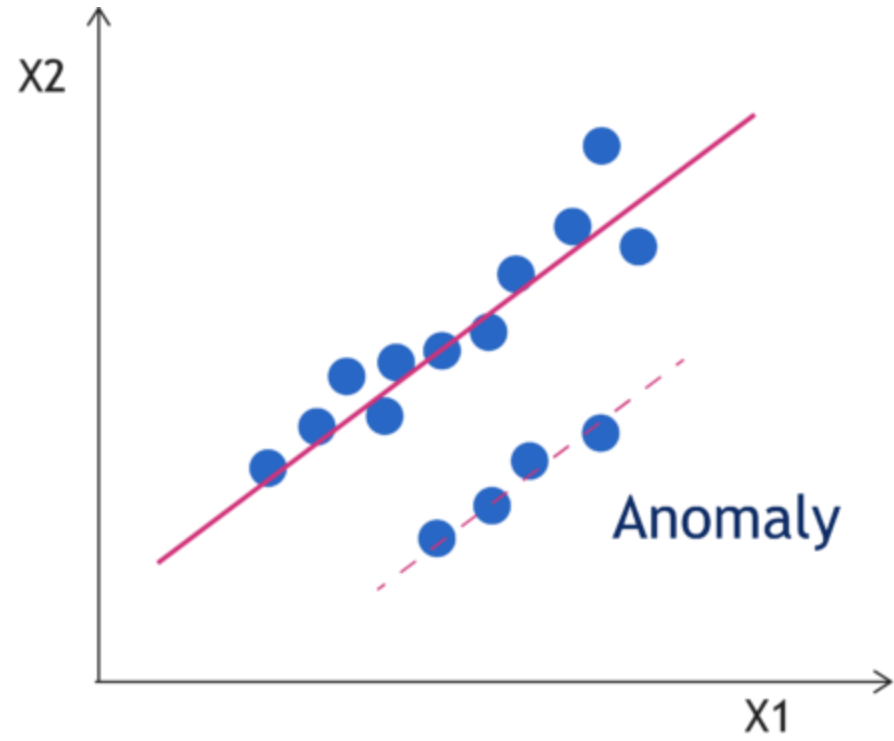
Unit: 2 (Machine learning clustering algorithms-II)

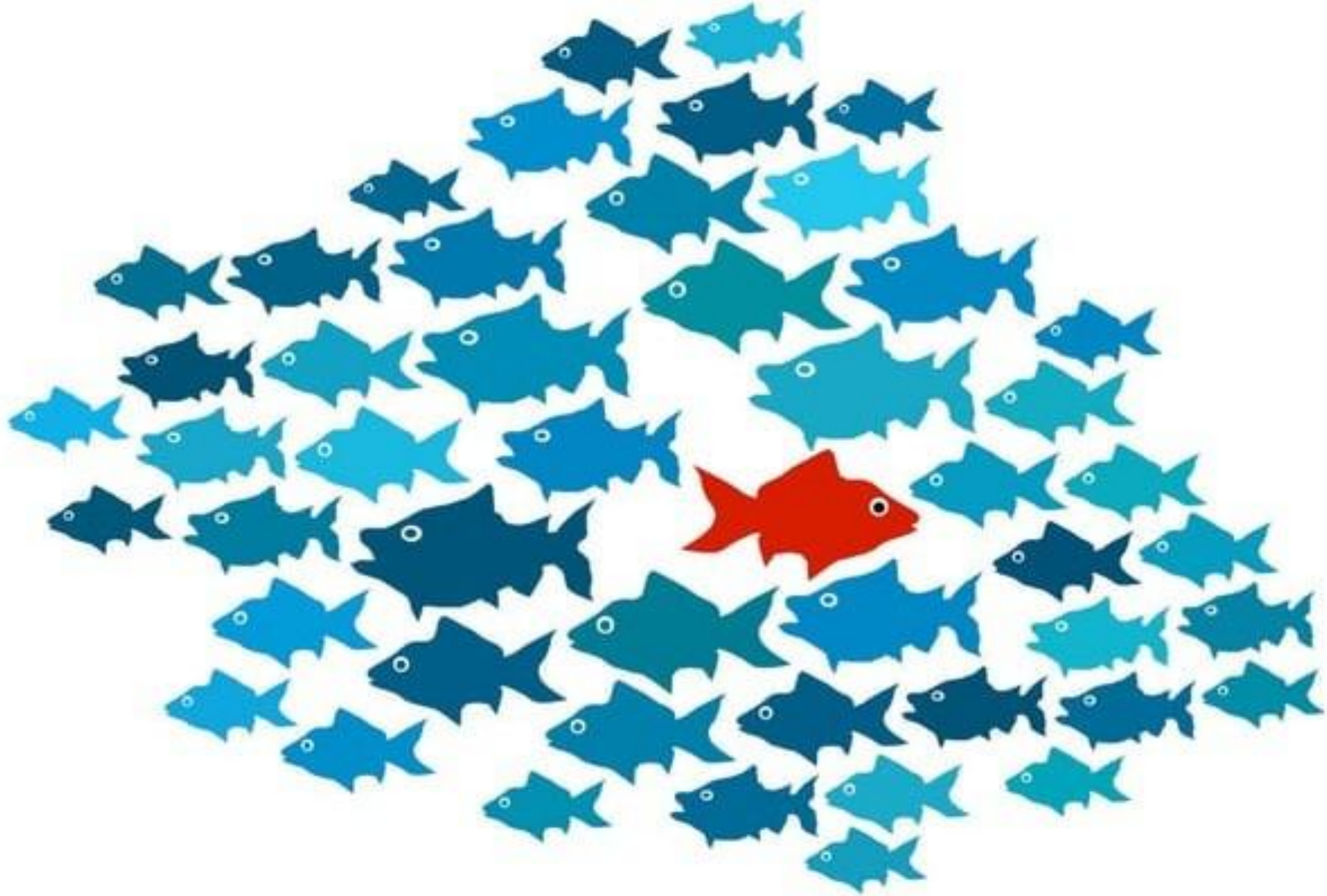
What is Anomaly Detection?

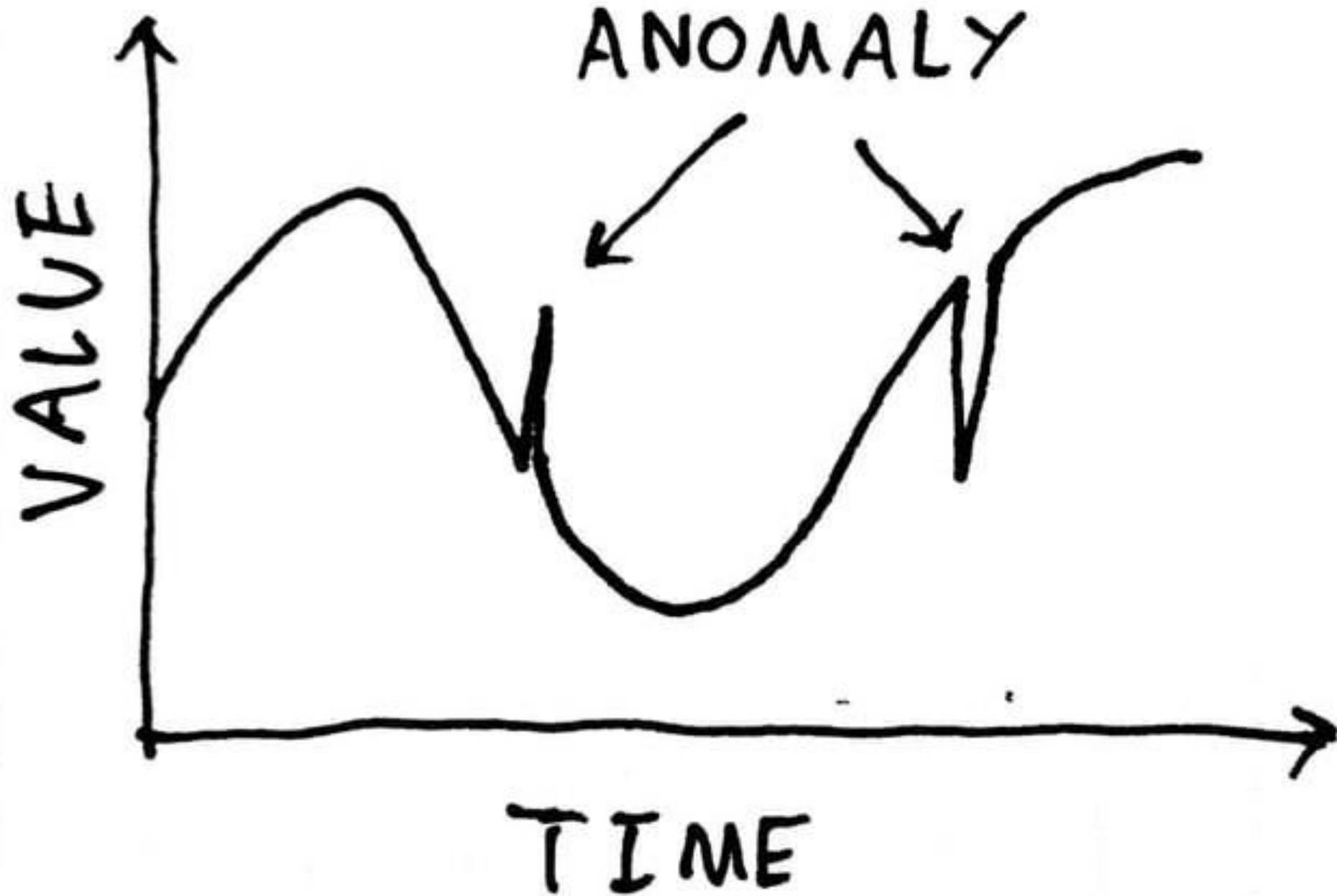
- Trying to find patterns in data that are different from the expected.
- Some applications:

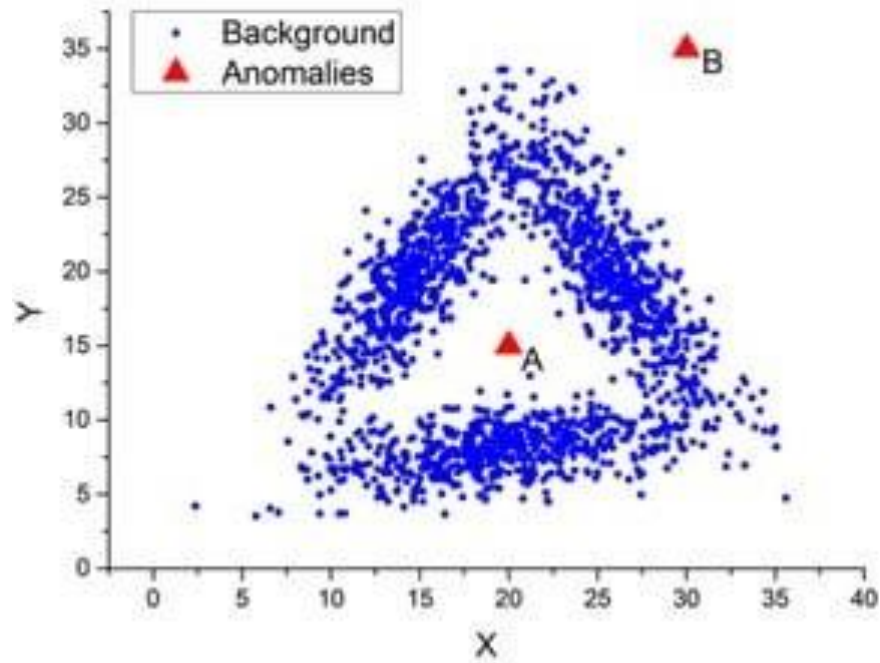
credit card fraud	insurance fraud
image processing	intrusion detection (cybersecurity)
text analysis	sensor networks
insider threats	industrial damage

Anomaly detection in machine learning refers to the process of identifying data points or patterns that deviate significantly from the norm or the expected behavior.

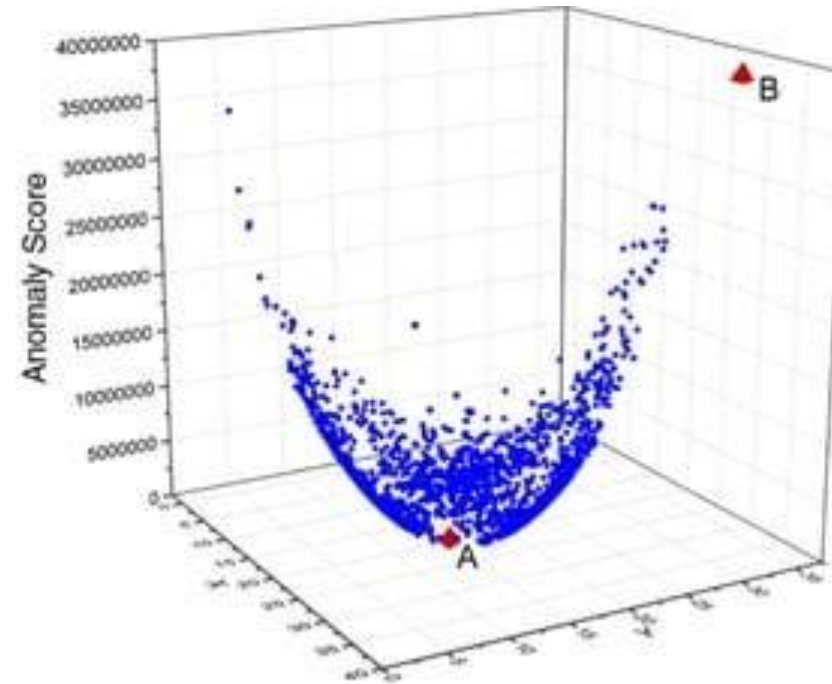




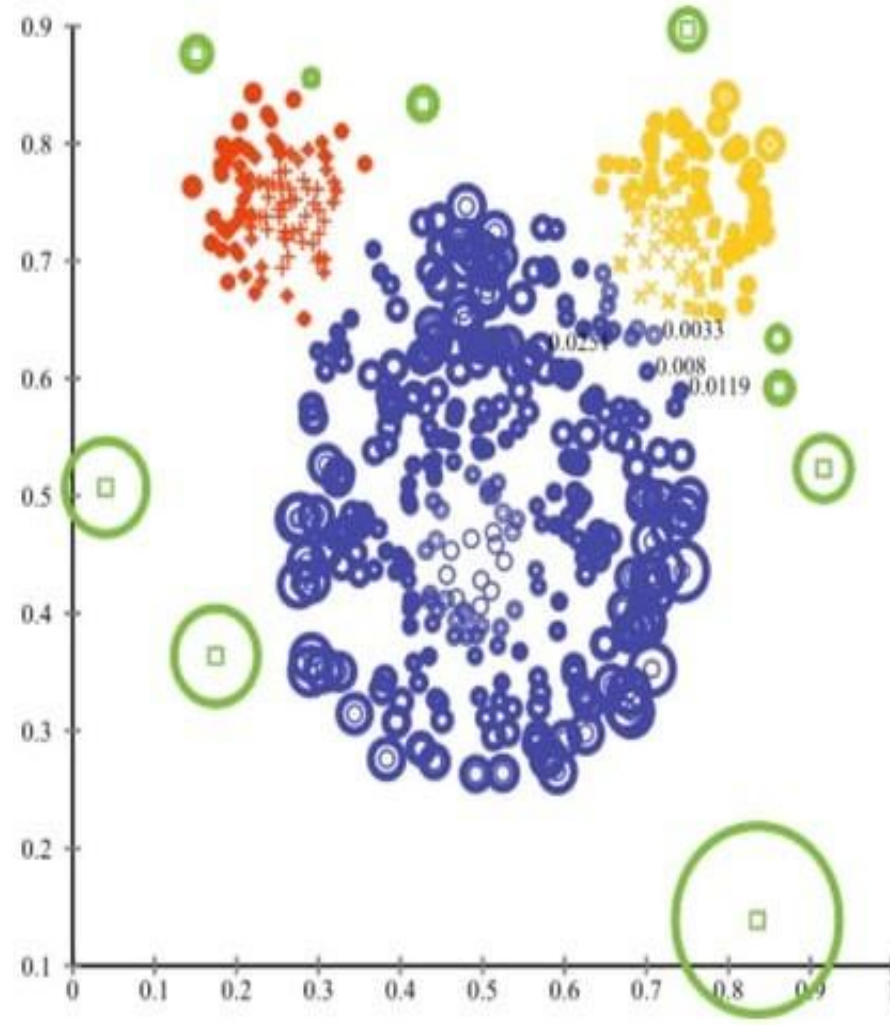


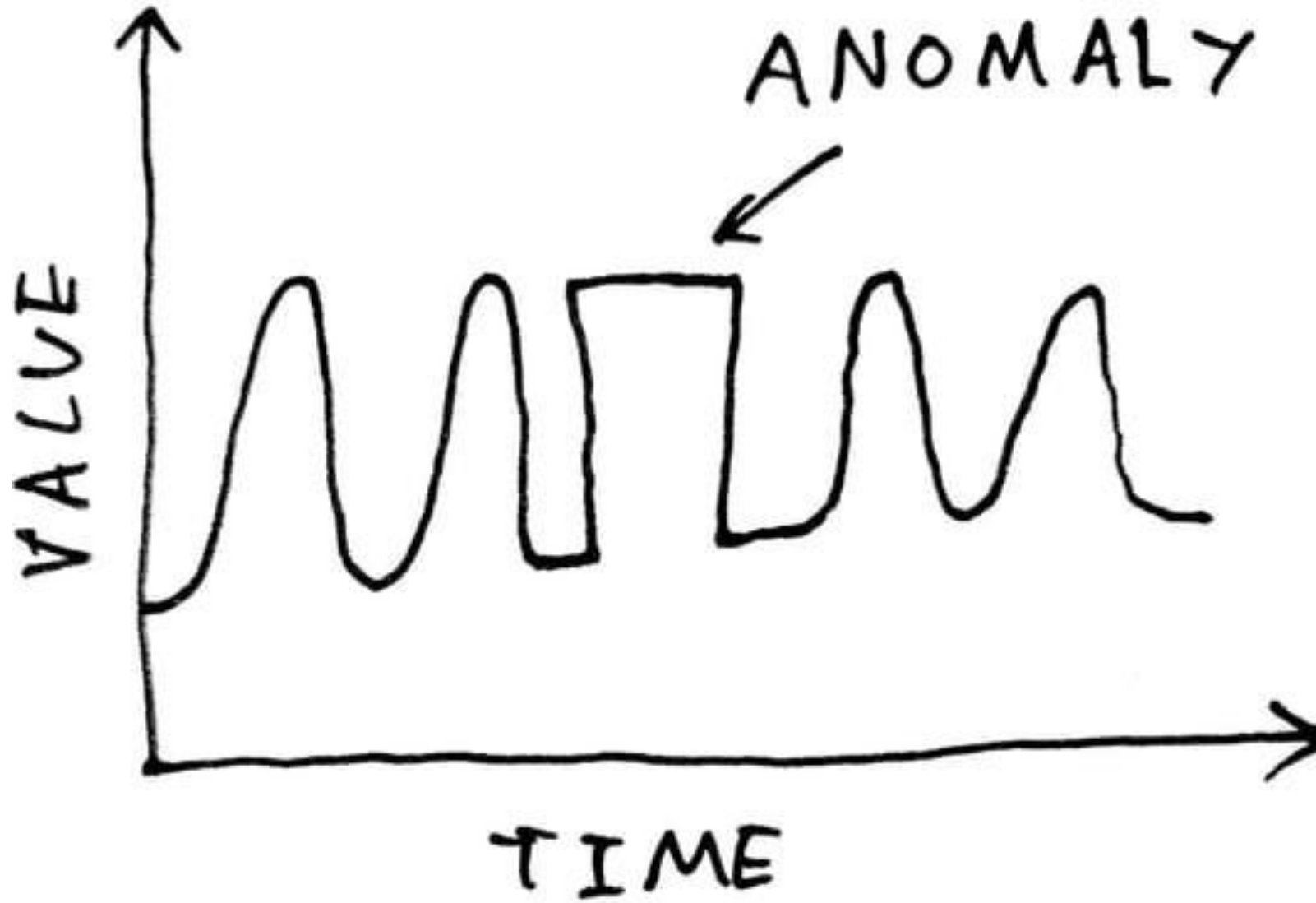


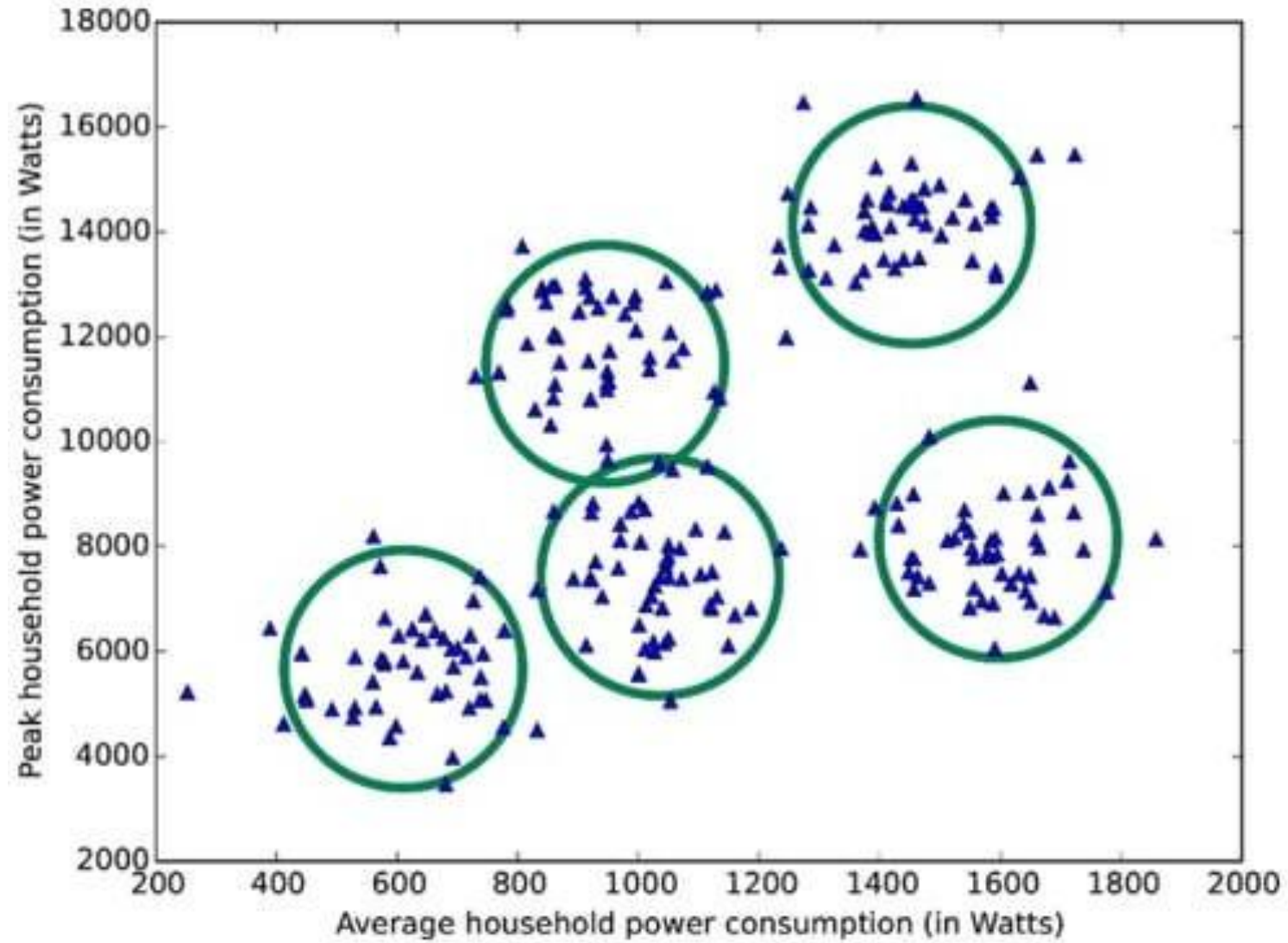
(a)



(b)







Detecting Anomalies

So, how would we detect some of these? Let's take a naive approach.

1. Define a "normal" region.
2. Observations not in the "normal" region are anomalies.

Will this work?

- Boundary hard to define
- Definitions change over time
- Definitions are domain-dependent
- Labeled training data is hard to find
- Training data, is often heavily imbalanced

Types of Data

- Collection of data instances
- a data instance has a set of attributes
- Attributes can be of different types
 - binary
 - categorical
 - continuous



- The attributes help determine the detection method.
- The relationship between data instances is important.
- Most existing anomaly detection techniques don't assume any particular relationship between the data instances. We have to identify relationships.

Types of input data

- Sequential
 - time-series, sequences of symbols
- Spatial
 - each data instance is related to its neighbors
 - images, vehicular traffic
- Graph
 - data instances are nodes in a graph or network

Three Types of Anomalies

- 😊 There are only three.
- 😞 No, that doesn't make it any easier to detect them.
- Point anomaly
- Contextual anomaly
- Collective anomaly

- Generally a single data instance.
 - Anomalous compared to the entirety of the data
 - Most research focuses on point anomalies
 - Can occur in any dataset
-
- Point anomalies, also known as global anomalies, refer to individual data points that deviate significantly from the rest of the data. These anomalies are isolated and stand out from the majority of the data points.
 - Example: Detecting a fraudulent credit card transaction where the transaction amount is far larger or smaller than the typical spending behavior of the cardholder.

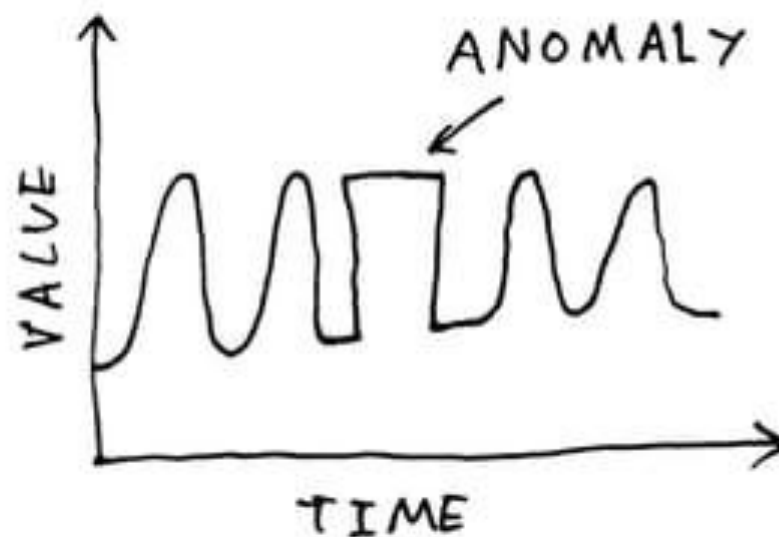
Contextual Anomalies

- Contextual anomalies, also called **conditional anomalies**, are data points that are considered anomalous only within a **specific context or condition**. These anomalies might be normal in one context but abnormal in another.
- Example: An increase in web traffic on a retail website during holiday seasons might not be anomalous, but the same traffic increase on a different website not related to retail during the same time could be considered anomalous.

Contextual Anomaly

- Anomalous in relation to a specific context
- Context comes from how data is structured
- Context has to be specified as a part of the problem formulation
- Each data instance can be defined using two sets of attributes:
 - contextual: determines the context (e.g. lat/long or time)
 - behavioral: non-contextual characteristics of an instance

- Anomalous behavior is determined by the behavioral attributes within a specific context
- A data instance might be a contextual anomaly in a given context, but a data instance with identical behavioral attributes could be considered normal in a different context.





- Contextual anomalies are generally found in time-series data. Example:
 - Avg monthly temp. of an area over last few years.
 - 35 degrees F in winter might be normal
 - 35 degrees F in summer in same place is anomalous

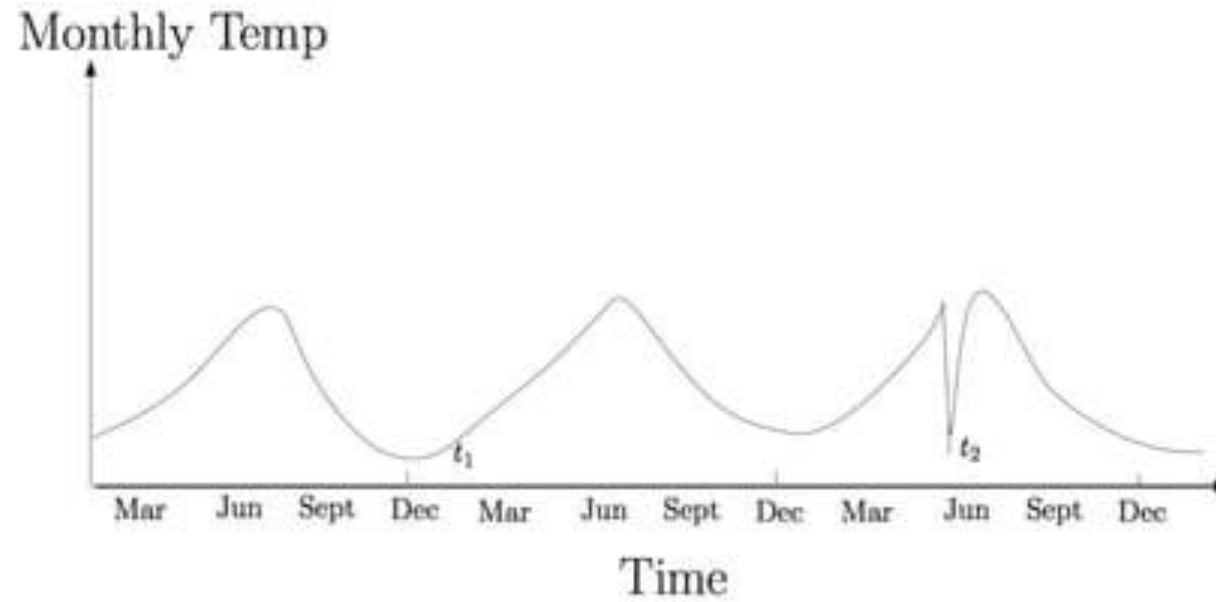


Fig. 3. Contextual anomaly t_2 in a temperature time series. Note that the temperature at time t_1 is same as that at time t_2 but occurs in a different context and hence is not considered as an anomaly.

- Another example: Credit card fraud
- Contextual attribute: time of purchase.
 - \$100 average weekly shopping bill, except during the Christmas week, when it reaches \$1000.
 - A new purchase of \$1000 in July would be considered a contextual anomaly, since it's unusual for July.
 - The same amount spent during Christmas week will be considered normal.



Collective Anomalies

- Collective anomalies, also known as **group anomalies** or cluster-level anomalies, involve a group or a subset of data instances that **collectively exhibit anomalous behavior** when considered together. Individually, the data points might appear normal, but their combination is abnormal.
- Example: Identifying a cluster of network devices that exhibit unusual **communication patterns** compared to the overall network traffic. While each device's behavior might appear normal on its own, the collective behavior of the devices as a group is anomalous.

Collective Anomaly

- A group of data instances are anomalous
- They need not be anomalies by themselves
- Again, the relationship between the data matters
- Point | Collective problem + context = Contextual problem



Applications

Credit Card Fraud

Data used

- user ID
- amount spent
- time between consecutive card usage

Credit card companies have complete, labeled data and user profiles



Kinds of anomalies

- point anomalies in transaction records
 - high payments
 - items never before purchased by the user
 - high rate of purchase
- contextual anomalies
 - User defines the context
 - Each credit card user is profiled based on card usage history.
 - Each new transaction compared to user profile, flagged if it doesn't match
 - Location defines the context
 - Detects anomalies among transactions at a specific geographic location.

Cellphone Fraud

Data used

- Call data records (CDRs)
- CDR = vector of features
 - continuous (e.g., CALL-DURATION)
 - discrete (e.g., CALLING-CITY).

Kinds of anomalies

- point anomalies from aggregated CDR data
 - aggregated by time, user, or area
 - high volume of calls
 - calls made to unlikely destinations



Insider Trading

Data used

- Option trading data
- Stock trading data
- News
- Data is time-series or otherwise temporally sequenced.

Medical

- Patient records
 - [Electronic Health Records \(EHRs\)](#)
 - demographics, medical history, medication and allergies, immunization status, laboratory test results, radiology images, vital signs, personal statistics like age and weight, and billing information
 - Electrocardiograms (ECG) and Electroencephalograms (EEG)
- Temporal and/or spatial data

Types of anomalies

- point anomalies
 - e.g., abnormal patient condition, instrumentation errors, recording errors
- contextual
 - Disease outbreaks can be contextual anomalies (e.g. geo-temporal pattern of viral infections)
- collective



Advantages of Anomaly Detection

- **Early Detection:** Anomaly detection can identify unusual patterns or observations in **data at an early stage**, allowing for early intervention and prevention of potential issues.
- **Automation:** Anomaly detection can be automated, allowing for continuous data monitoring and **reducing the need for manual intervention**.
- **Scalability:** Anomaly detection can be applied to **large datasets**, making it suitable for big data applications.
- **Adaptability:** Anomaly detection can be applied to various data types, including numerical data, time series data, and categorical data.
- **Real-time Monitoring:** Anomaly detection can be used for real-time data monitoring, allowing for **immediate action** in case of an anomaly.



Limitations of Anomaly Detection

- **Data Quality:** Anomaly detection's performance depends on the data's quality. Poor quality data can result in **false positives or false negatives**.
- **Choice of Algorithm:** The choice of algorithm can also affect anomaly detection performance. Some algorithms may be better suited for certain types of data or specific use cases.
- **The threshold for Determining Anomalies:** The threshold for determining anomalies **is subjective** and can affect anomaly detection performance.
- **The imbalance between Normal and Anomalous Data:** As anomalous data is typically rare, it can be challenging to train a model that **can accurately** identify them. This is known as the class imbalance problem.



Anomaly Detection Vs Supervised Learning

- Supervised learning excels with labeled data, offering clear classification of both normal and anomalous data points.
- Anomaly detection works well with unlabeled data, identifying deviations from the learned normal behavior.



Anomaly Detection using Decision Tree

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import importlib
importlib.reload(np.random)
```

```
# Generate synthetic data
np.random.seed(42)
normal_data = np.random.normal(0, 2, (1000, 2))
anomaly_data = np.random.uniform(-20, 20, (50, 2))
```

```
# Create labels (0 for normal, 1 for anomalies)
data = np.vstack((normal_data, anomaly_data))
labels = np.array([0] * 1000 + [1] * 50)
```

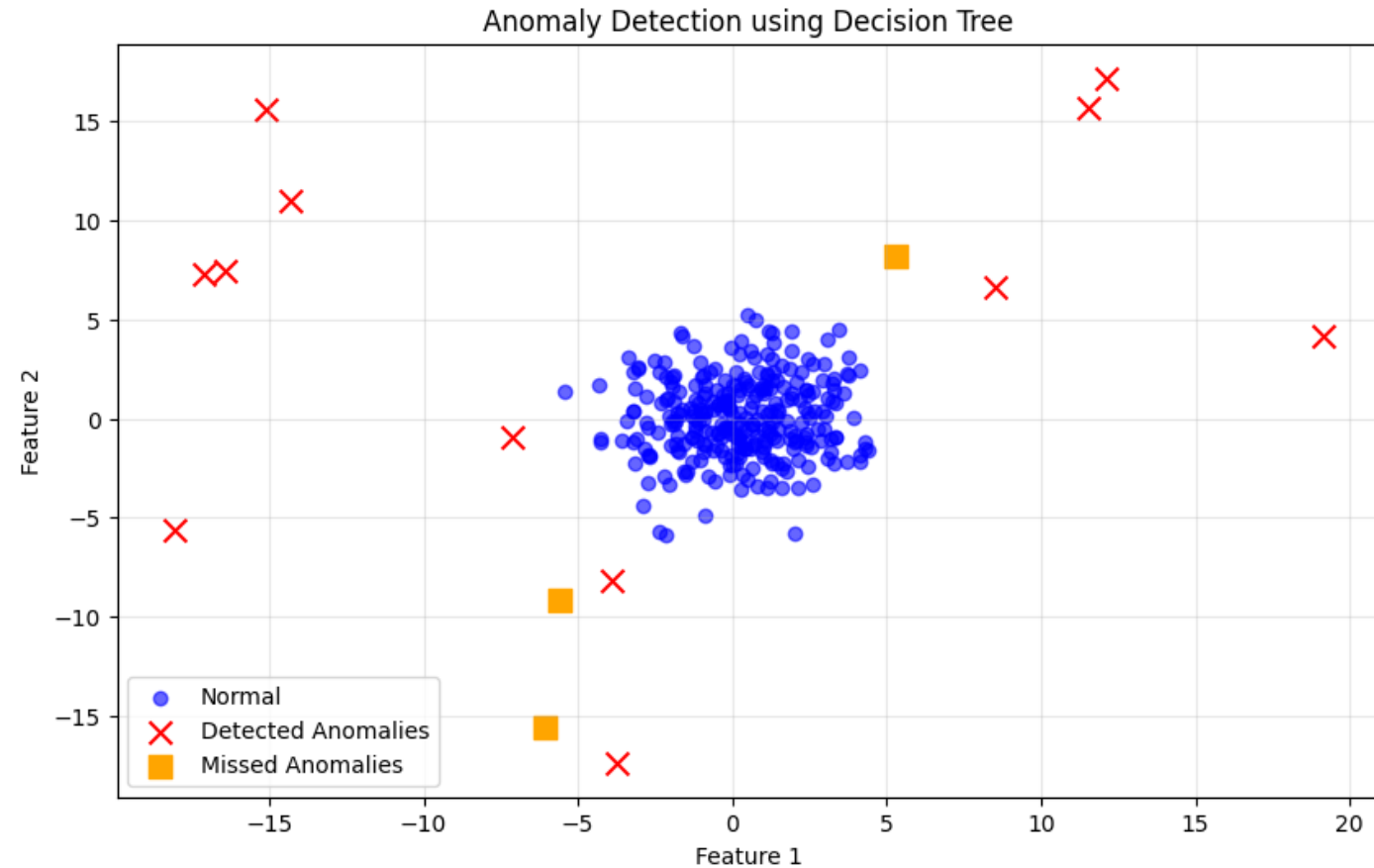


- # Split data into training and testing sets
- `X_train, X_test, y_train, y_test = train_test_split(data, labels, test_size=0.3, random_state=42, stratify=labels)`
- # Fit Decision Tree model
- `model = DecisionTreeClassifier(max_depth=10, random_state=42)`
- `model.fit(X_train, y_train)`
- # Predict anomalies
- `predictions = model.predict(X_test)`
- # Evaluate model
- `print("Classification Report:")`
- `print(classification_report(y_test, predictions))`

- # Plot normal points
- `normal_mask = (y_test == 0)`
- `plt.scatter(X_test[normal_mask, 0], X_test[normal_mask, 1], c='blue', label='Normal', alpha=0.6)`
- # Plot correctly detected anomalies
- `correct_anomalies = (predictions == 1) & (y_test == 1)`
- `plt.scatter(X_test[correct_anomalies, 0], X_test[correct_anomalies, 1], c='red', label='Detected Anomalies', marker='x', s=100)`
- # Plot missed anomalies (false negatives)
- `missed_anomalies = (predictions == 0) & (y_test == 1)`
- `plt.scatter(X_test[missed_anomalies, 0], X_test[missed_anomalies, 1], c='orange', label='Missed Anomalies', marker='s', s=100)`
- `plt.title("Anomaly Detection using Decision Tree")`
- `plt.xlabel("Feature 1")`
- `plt.ylabel("Feature 2")`
- `plt.legend()`
- `plt.grid(True, alpha=0.3)`
- `plt.show()`

**Classification Report:**

	precision	recall	f1-score	support
0	0.99	0.99	0.99	300
1	0.86	0.80	0.83	15
accuracy			0.98	315
macro avg	0.92	0.90	0.91	315
weighted avg	0.98	0.98	0.98	315



```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Generate synthetic data
np.random.seed=42
normal_data = np.random.randn(1000, 2) * 2 #
Normal data points
anomaly_data = np.random.randn(50, 2) * 10 #
Anomaly data points

# Combine normal and anomaly data
data = np.vstack((normal_data, anomaly_data))

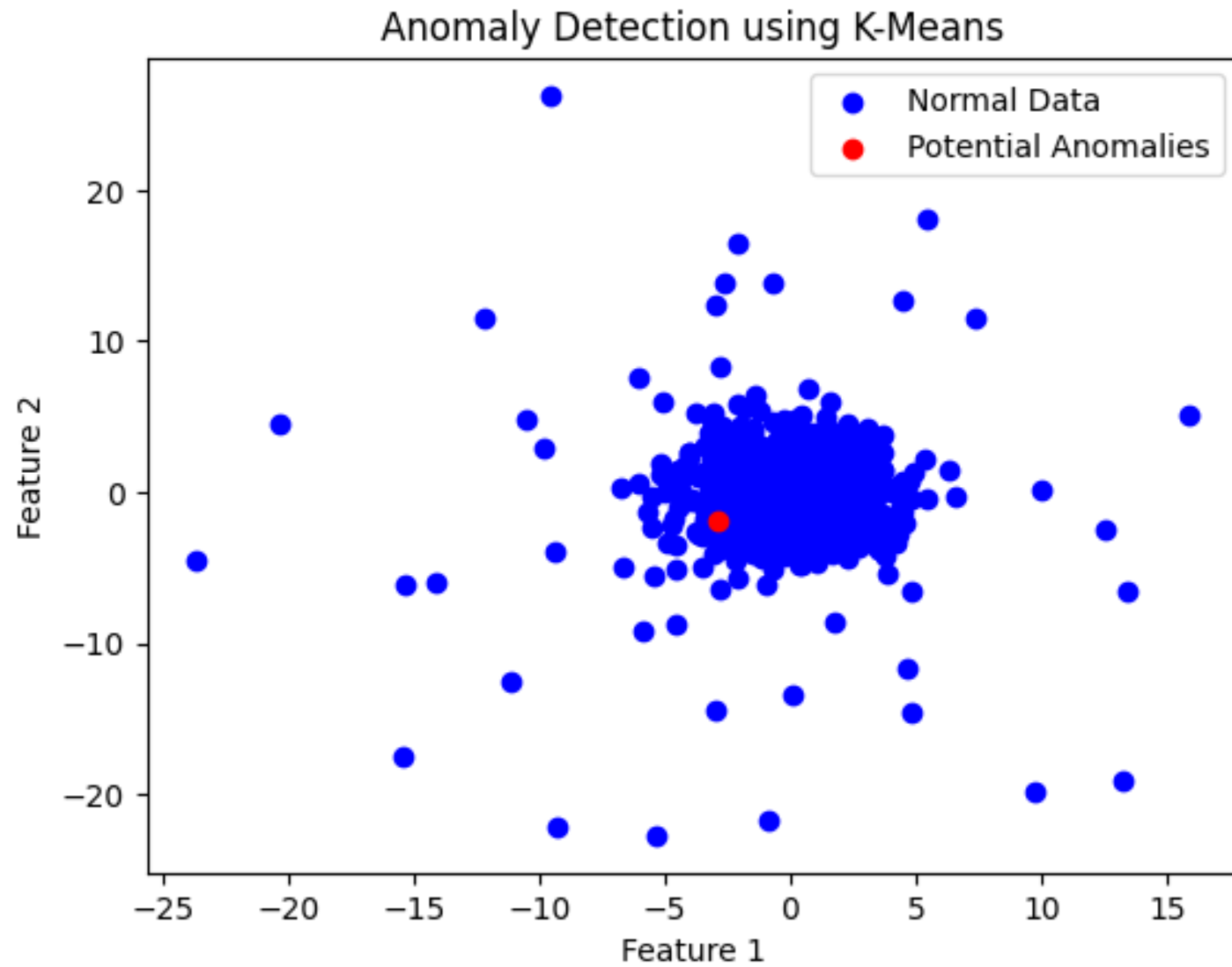
# Fit K-Means model
num_clusters = 5
model =
KMeans(n_clusters=num_clusters)
model.fit(data)

# Assign data points to clusters
cluster_labels = model.labels_

# Calculate cluster sizes
cluster_sizes =
np.bincount(cluster_labels)
```



- **# Define a threshold for cluster size to identify potential anomalies**
- `anomaly_threshold = np.percentile(cluster_sizes, 5) # 5th percentile cluster size`
- `# Identify potential anomalies`
- `potential_anomalies = data[cluster_labels[np.where(cluster_sizes <= anomaly_threshold)]]`
- `# Plot the data and potential anomalies`
- `plt.scatter(data[:, 0], data[:, 1], c='blue', label='Normal Data')`
- `plt.scatter(potential_anomalies[:, 0], potential_anomalies[:, 1], c='red', label='Potential Anomalies')`
- `plt.legend()`
- `plt.title("Anomaly Detection using K-Means")`
- `plt.xlabel("Feature 1")`
- `plt.ylabel("Feature 2")`
- `plt.show()`





- **Apply anomaly detection algorithm using KMeans clustering on make_blobs dataset. Use threshold as 4th percentile of the cluster size.**



Star Course Activity

- Make 3 teams of students present in the class.
- Allocate any one type of Anomaly to each group.
- Ask them to discuss different examples associated with the anomaly allocated to their team.
- The team members will share their views with the whole class.



Gaussian Distribution

- The Gaussian distribution, also known as the **normal distribution**. It is a key concept used to model the distribution of **real-valued random variables** and is essential for understanding **various statistical methods** and algorithms.
- It is a **continuous probability distribution function** that is symmetrical at the mean, and the majority of data falls within one standard deviation of the mean. It is characterized by its **bell-shaped curve**.

Gaussian Distribution Formula

The PDF (probability density function) of the Gaussian distribution is given by the formula:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

where:

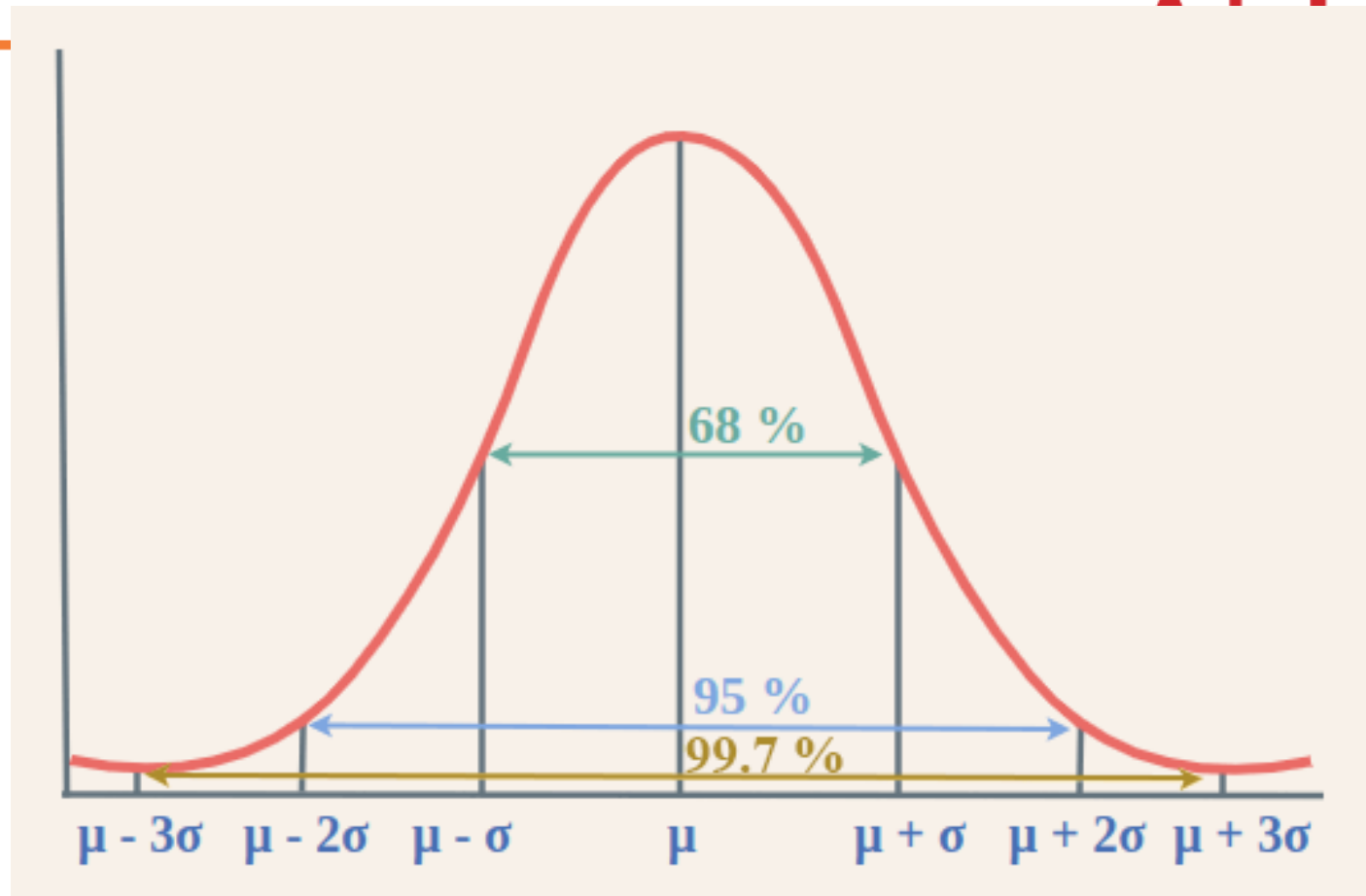
- x represents the **Variable**
- μ represents the **Mean**
- σ represents the **Standard Deviation**
- e represents the **base of the Natural Logarithm**.



Gaussian Distribution Curve

- The curve is symmetric and bell-shaped, and it mathematically represents the probability distribution of a continuous random variable. The Gaussian distribution is characterized by two parameters: the mean (μ) and the standard deviation (σ), which determine the location and the spread of the curve.

- Within one standard deviation of the mean ($\text{Mean} \pm 1 \text{ SD}$), approximately **68%** of the data is expected to fall.
- Within two standard deviations of the mean ($\text{Mean} \pm 2 \text{ SD}$), approximately **95%** of the data is expected to fall.
- Within three standard deviations of the mean ($\text{Mean} \pm 3 \text{ SD}$), approximately **99.7%** of the data is expected to fall.





- The standard deviations are used to **subdivide the area under the normal curve**. Each subdivided section defines the percentage of data, which falls into the specific region of a graph.
- Analysis : **A smaller standard deviation results in a narrower and taller bell curve**, indicating that data points are clustered closely around the mean. Conversely, **a larger standard deviation leads to a wider and shorter bell curve**, suggesting that data points are more spread out from the mean.
- The Empirical Rule, also known as the 68-95-99.7 rule, quantifies the proportion of data falling within certain intervals around the mean in a normal distribution.



Gaussian Distribution Table

- A Gaussian distribution table, also known as a standard normal distribution table or z-table, is a tabulated form that provides values of the cumulative distribution function (CDF) for the standard normal distribution.
- The standard normal distribution has a mean(central value) of 0 and a standard deviation of 1.
- Normally , the table consists of two columns namely Z-value and their Cumulative probability . Z-value is the number of standard deviations away from the mean. It ranges from negative infinity to positive infinity.
- Cumulative probability represents the probability that a standard normal random variable is less than or equal to the corresponding z-value.

***Gaussian Distribution Anomaly Detection Algorithm:***

Let us consider that there are m data-points (instances) each with n selected features.

- The Mean parameter for each feature ($j = 1$ to n) is fit.
- The Variance parameter for each feature ($j = 1$ to n) is fit.
- Given a new data-point, $x = \{x_1, x_2, \dots, x_j\}$, $p(x)$ is given by,

or,

- Now, a threshold parameter, ϵ is selected such that,

if $p(x) < \epsilon$:

x is an ANOMALY or OUTLIER !!!!

else:

x is NOT an ANOMALY or OUTLIER !!!!



Thank you!

Hierarchical clustering is another unsupervised machine learning algorithm, which is used to group the unlabeled datasets into a cluster and also known as **hierarchical cluster analysis** or HCA.

In this algorithm, we develop the hierarchy of clusters in the form of a tree, and this tree-shaped structure is known as the **dendrogram**.

Sometimes the results of K-means clustering and hierarchical clustering may look similar, but they both differ depending on how they work. As there is no requirement to predetermine the number of clusters as we did in the K-Means algorithm.

The hierarchical clustering technique has two approaches:

1. **Agglomerative:** Agglomerative is a **bottom-up** approach, in which the algorithm starts with taking all data points as single clusters and merging them until one cluster is left.
2. **Divisive:** Divisive algorithm is the reverse of the agglomerative algorithm as it is a **top-down approach**.

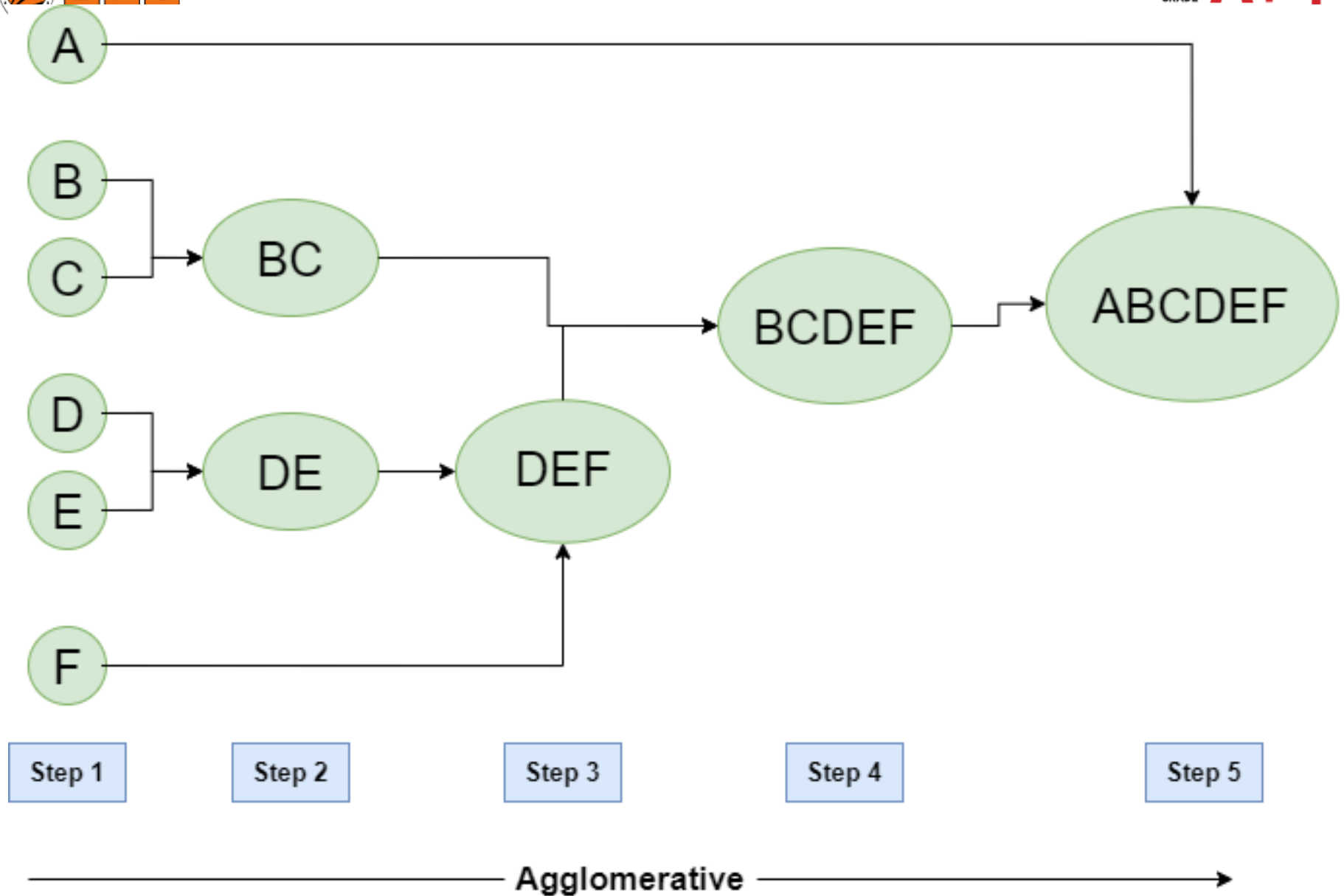
Why hierarchical clustering?

As we already have other clustering algorithms such as **K-Means Clustering**, then why we need hierarchical clustering? So, as we have seen in the K-means clustering that there are some challenges with this algorithm, which are a predetermined number of clusters, and it always tries to create the clusters of the same size. To solve these two challenges, we can opt for the hierarchical clustering algorithm because, in this algorithm, we don't need to have knowledge about the predefined number of clusters.

Agglomerative Hierarchical clustering

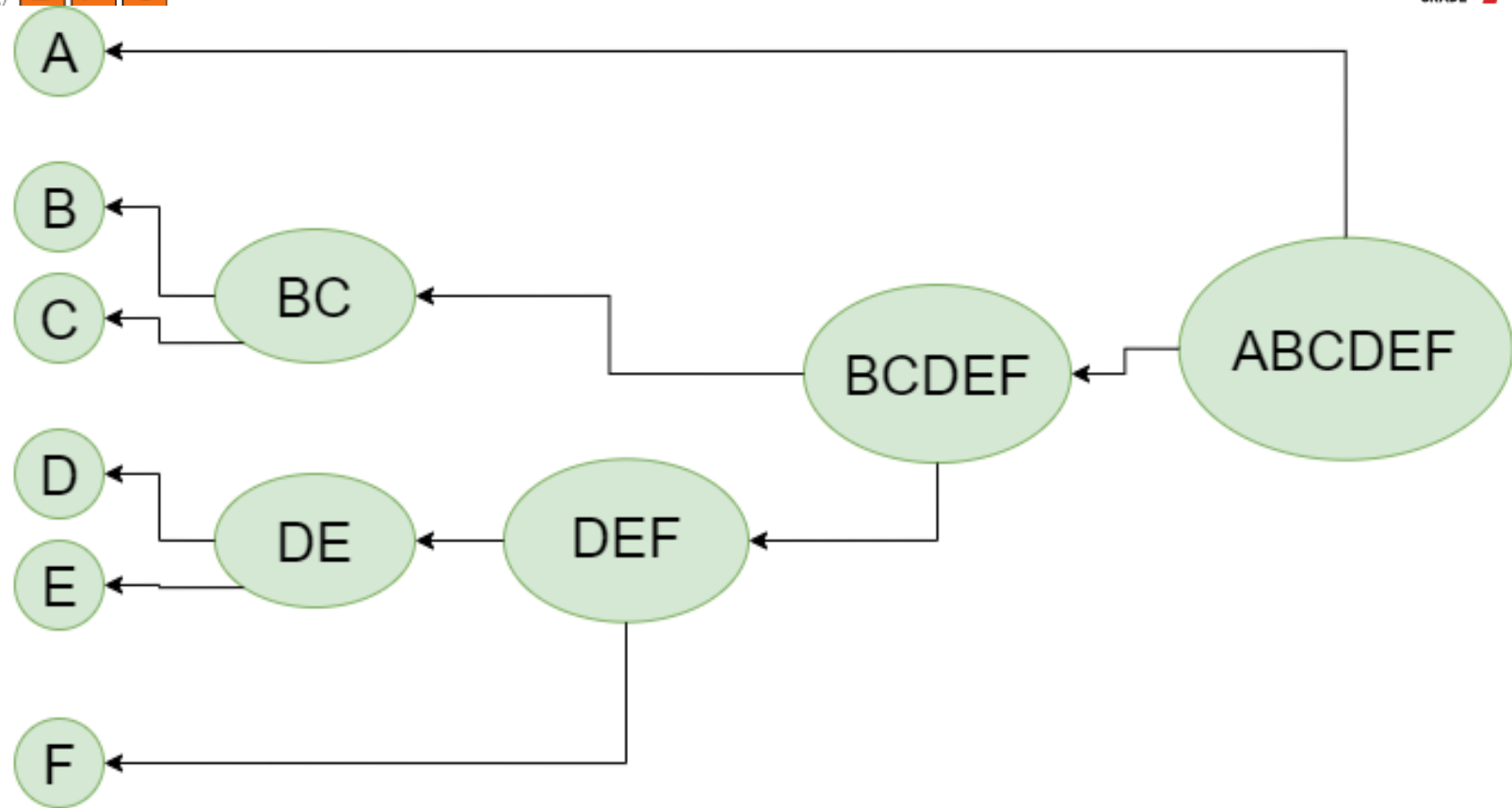
The agglomerative hierarchical clustering algorithm is a popular example of HCA. To group the datasets into clusters, it follows the **bottom-up approach**. It means, this algorithm considers each dataset as a single cluster at the beginning, and then start combining the closest pair of clusters together. It does this until all the clusters are merged into a single cluster that contains all the datasets.

This hierarchy of clusters is represented in the form of the dendrogram.



Hierarchical Divisive clustering

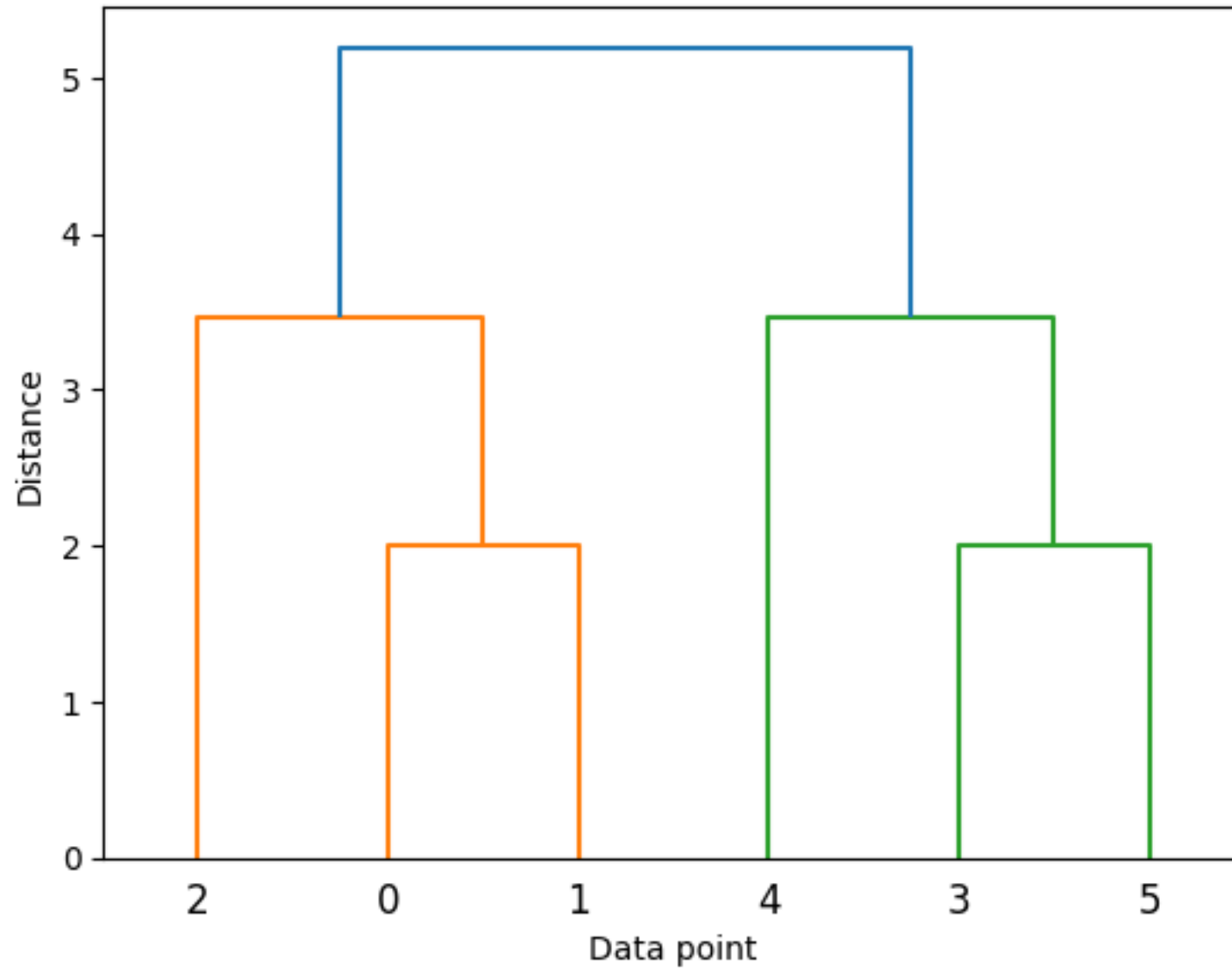
It is also known as a top-down approach. This algorithm also does not require to pre-specify the number of clusters. Top-down clustering **requires a method for splitting a cluster** that contains the whole data and proceeds by splitting clusters **recursively** until individual data have been split into singleton clusters.



Step 5
Step 4
Step 3
Step 2
Step 1

← Divisive →

Hierarchical Clustering Dendrogram



Hierarchical clustering algorithm Numerical Example

To illustrate hierarchical clustering algorithm, let us use the following simple example. Suppose we have 6 objects (with name A, B, C, D, E and F) and each object have two measured feature X1 and X2. We can plot the features in a scattered plot to get the visualization of proximity between objects.

A(1, 1)

B (1.5, 1.5)

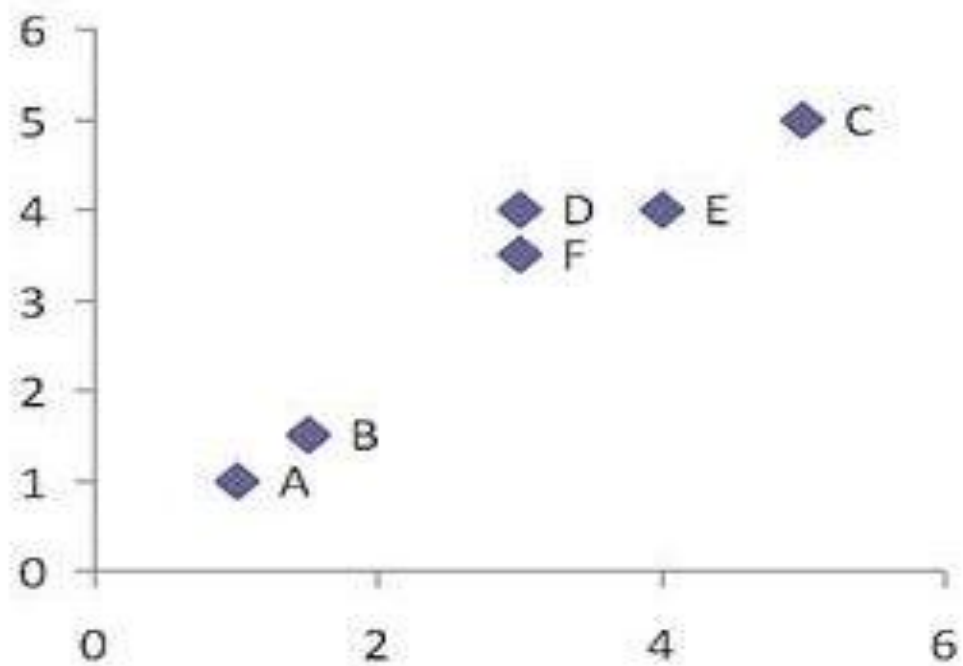
C (5, 5)

D (3, 4)

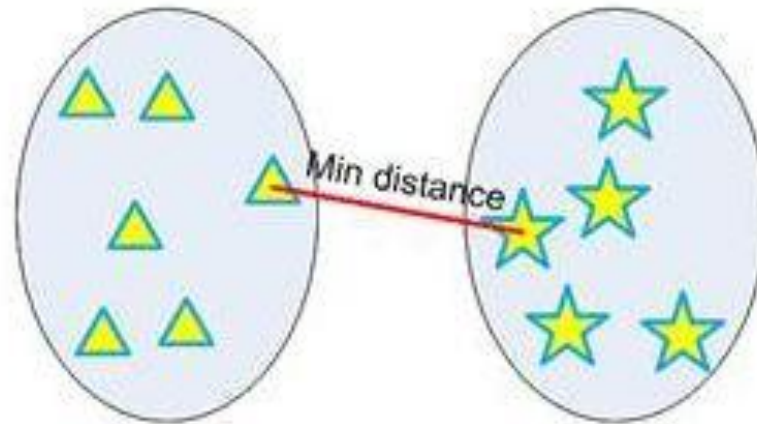
E(4, 4)

F (3, 3.5)

	X1	X2
A	1	1
B	1.5	1.5
C	5	5
D	3	4
E	4	4
F	3	3.5



- Minimum distance clustering is also called as single linkage hierarchical clustering or nearest neighbor clustering. Distance between two clusters is defined by the minimum distance between objects of the two clusters, as shown below:



- For example, we have given an input distance matrix of size 6 by 6. This distance matrix was calculated based on the object features as explained in the previous section.

Dist	A	B	C	D	E	F
A	0.00	0.71	5.66	3.61	4.24	3.20
B	0.71	0.00	4.95	2.92	3.54	2.50
C	5.66	4.95	0.00	2.24	1.41	2.50
D	3.61	2.92	2.24	0.00	1.00	0.50
E	4.24	3.54	1.41	1.00	0.00	1.12
F	3.20	2.50	2.50	0.50	1.12	0.00

- We have 6 objects and we put each object into one cluster. Thus, in the beginning we have 6 clusters. Our goal is to group those 6 clusters such that at the end of the iterations, we will have only single cluster consists of the whole six original objects.
- In each step of the iteration, we find the closest pair clusters. In this case, the closest cluster is between cluster F and D with shortest distance of 0.5. Thus, we group cluster D and F into cluster (D, F). Then we update the distance matrix (see distance matrix below). Distance between ungrouped clusters will not change from the original distance matrix. Now the problem is how to calculate distance between newly grouped clusters (D, F) and other clusters?

Min Distance (Single Linkage)

Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	?	4.24
B	0.71	0.00	4.95	?	3.54
C	5.66	4.95	0.00	?	1.41
D, F	?	?	?	0.00	?
E	4.24	3.54	1.41	?	0.00

- That is exactly where the linkage rule comes into effect. Using single linkage, we specify minimum distance between original objects of the two clusters.
- Using the input distance matrix, distance between cluster (D, F) and cluster A is computed as

Using the input distance matrix, distance between cluster (D, F) and cluster A is computed as

$$d_{(D,F) \rightarrow A} = \min(d_{DA}, d_{FA}) = \min(3.61, 3.20) = 3.20$$

Distance between cluster (D, F) and cluster B is

$$d_{(D,F) \rightarrow B} = \min(d_{DB}, d_{FB}) = \min(2.92, 2.50) = 2.50$$

Similarly, distance between cluster (D, F) and cluster C is

$$d_{(D,F) \rightarrow C} = \min(d_{DC}, d_{FC}) = \min(2.24, 2.50) = 2.24$$

Finally, distance between cluster E and cluster (D, F) is calculated as

$$d_{E \rightarrow (D,F)} = \min(d_{ED}, d_{EF}) = \min(1.00, 1.12) = 1.00$$

- Then, the updated distance matrix becomes

Min Distance (Single Linkage)

Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	3.20	4.24
B	0.71	0.00	4.95	2.50	3.54
C	5.66	4.95	0.00	2.24	1.41
D, F	3.20	2.50	2.24	0.00	1.00
E	4.24	3.54	1.41	1.00	0.00

Looking at the lower triangular updated distance matrix, we found out that the closest distance between cluster B and cluster A is now 0.71. Thus, we group cluster A and cluster B into a single cluster name (A, B).

- Now we update the distance matrix. Aside from the first row and first column, all the other elements of the new distance matrix are not changed.

Dist	A,B	C	(D, F)	E
A,B	0	?	?	?
C	?	0	2.24	1.41
(D, F)	?	2.24	0	1.00
E	?	1.41	1.00	0

Using the input distance matrix (size 6 by 6), distance between cluster C and cluster (D, F) is computed as

$$d_{C \rightarrow (A,B)} = \min(d_{CA}, d_{CB}) = \min(5.66, 4.95) = 4.95$$

Distance between cluster (D, F) and cluster (A, B) is the minimum distance between all objects involves in the two clusters

$$d_{(D,F) \rightarrow (A,B)} = \min(d_{DA}, d_{DB}, d_{FA}, d_{FB}) = \min(3.61, 2.92, 3.20, 2.50) = 2.50$$

Similarly, distance between cluster E and (A, B) is

$$d_{E \rightarrow (A,B)} = \min(d_{EA}, d_{EB}) = \min(4.24, 3.54) = 3.54$$

Then the updated distance matrix is

Min Distance (Single Linkage)

Dist	A,B	C	(D, F)	E
A,B	0	4.95	2.50	3.54
C	4.95	0	2.24	1.41
(D, F)	2.50	2.24	0	1.00
E	3.54	1.41	1.00	0

- Observing the lower triangular of the updated distance matrix, we can see that the closest distance between clusters happens between cluster E and (D, F) at distance 1.00. Thus, we cluster them together into cluster ((D, F), E).
- The updated distance matrix is given below.

Min Distance (Single Linkage)

Dist	(A,B)	C	(D, F), E
(A,B)	0.00	4.95	2.50
C	4.95	0.00	1.41
(D, F), E	2.50	1.41	0.00

Distance between cluster ((D, F), E) and cluster (A, B) is calculated as

$$d_{((D,F),E) \rightarrow (A,B)} = \min(d_{DA}, d_{DB}, d_{FA}, d_{FB}, d_{EA}, d_{EB}) = \min(3.61, 2.92, 3.20, 2.50, 4.24, 3.54) = 2.50$$

Distance between cluster ((D, F), E) and cluster C yields the minimum distance of 1.41. This distance is computed as

$$d_{((D,F),E) \rightarrow C} = \min(d_{DC}, d_{FC}, d_{EC}) = \min(2.24, 2.50, 1.41) = 1.41$$

After that, we merge cluster ((D, F), E) and cluster C into a new cluster name (((D, F), E), C).

The updated distance matrix is shown in the figure below

Min Distance (Single Linkage)

Dist	(A,B)	((D, F), E),C
(A,B)	0.00	2.50
((D, F), E),C	2.50	0.00

The minimum distance of 2.5 is the result of the following computation

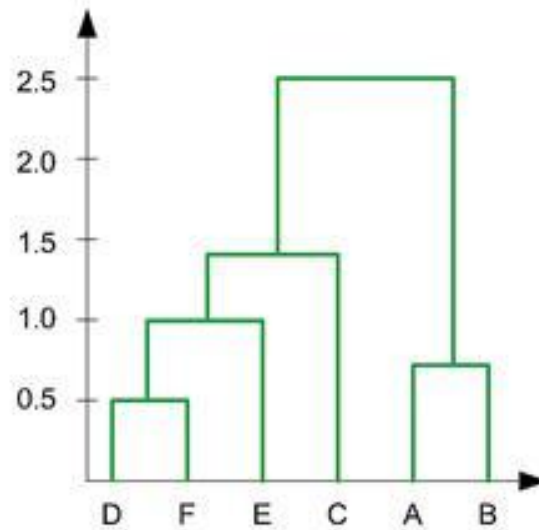
$$d_{(((D,F),E),C) \rightarrow (A,B)} = \min \{d_{DA}, d_{DB}, d_{FA}, d_{FB}, d_{EA}, d_{EB}, d_{CA}, d_{CB}\}$$

$$d_{(((D,F),E),C) \rightarrow (A,B)} = \min \{3.61, 2.92, 3.20, 2.50, 4.24, 3.54, 5.66, 4.95\} = 2.50$$

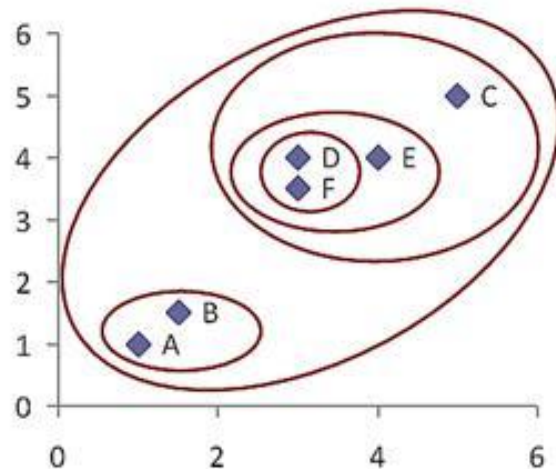
Now if we merge the remaining two clusters, we will get only single cluster contain the whole 6 objects. Thus, our computation is finished. We summarized the results of computation as follow:

1. In the beginning we have 6 clusters: A, B, C, D, E and F
2. We merge cluster D and F into cluster (D, F) at distance **0.50**
3. We merge cluster A and cluster B into (A, B) at distance **0.71**
4. We merge cluster E and (D, F) into ((D, F), E) at distance **1.00**
5. We merge cluster ((D, F), E) and C into (((D, F), E), C) at distance **1.41**
6. We merge cluster (((D, F), E), C) and (A, B) into ((((D, F), E), C), (A, B)) at distance **2.50**
7. The last cluster contain all the objects, thus conclude the computation

- Using this information, we can now draw the final results of a dendrogram. The dendrogram is drawn based on the distances to merge the clusters above.



The hierarchy is given as $((((D, F), E), C), (A, B))$. We can also plot the clustering hierarchy into XY space.





Isolation Forest



Isolation Forest

- Isolation Forest is a technique for identifying outliers in data that was first introduced by Fei Tony Liu and Zhi-Hua Zhou in 2008. The approach employs binary trees to detect anomalies, resulting in a linear time complexity and low memory usage that is well-suited for processing large datasets.
- Isolation Forest is a machine learning algorithm specifically designed for anomaly detection.
- It works by isolating anomalies in a dataset using random forests.
- This algorithm is efficient and effective, particularly for high-dimensional data or data with complex patterns.



Isolation Forests Anomaly Detection

- Isolation Forests(IF), similar to Random Forests, are build based on decision trees. And since there are no pre-defined labels here, it is an unsupervised model.
- Isolation Forests were built based on the fact that anomalies are the data points that are “few and different”.
- In an Isolation Forest, randomly sub-sampled data is processed in a tree structure based on randomly selected features. The samples that travel deeper into the tree are less likely to be anomalies as they required more cuts to isolate them. Similarly, the samples which end up in shorter branches indicate anomalies as it was easier for the tree to separate them from other observations.



How do Isolation Forests work?

- Isolation Forests outlier detection are nothing but an ensemble of **binary decision trees**. And each tree in an Isolation Forest is called an Isolation Tree (iTree). The algorithm starts with the **training** of the data, by generating Isolation Trees.
- It uses the **average** of the predictions by several decision trees when assigning the final anomaly score to a given data point. Unlike other anomaly detection algorithms, which first define what's "**normal**" and then report anything else as anomalous, Isolation Forest attempts to isolate anomalous data points from the get go.



Step by Step Guide on How Isolation Forest Work

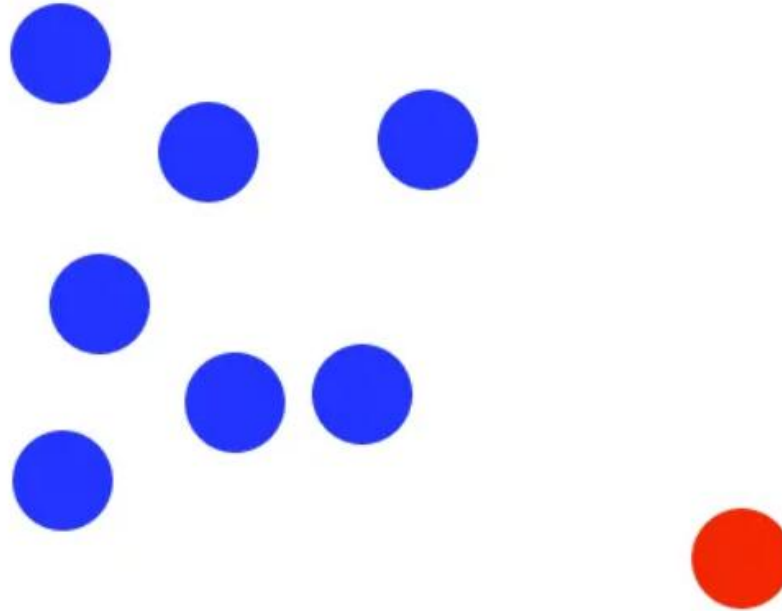
- When given a dataset, a random sub-sample of the data is selected and assigned to a binary tree.
- Branching of the tree starts by selecting a **random feature** (from the set of all N features) first. And then branching is done on a random threshold (any value in the range of minimum and maximum values of the selected feature).
- If the value of a data point is **less than** the selected threshold, it goes to the left branch else to the right. And thus a node is split into left and right branches.
- This process from step 2 is continued **recursively** till each data point is completely isolated or till **max depth**(if defined) is reached.
- The above steps are repeated to construct random binary trees.



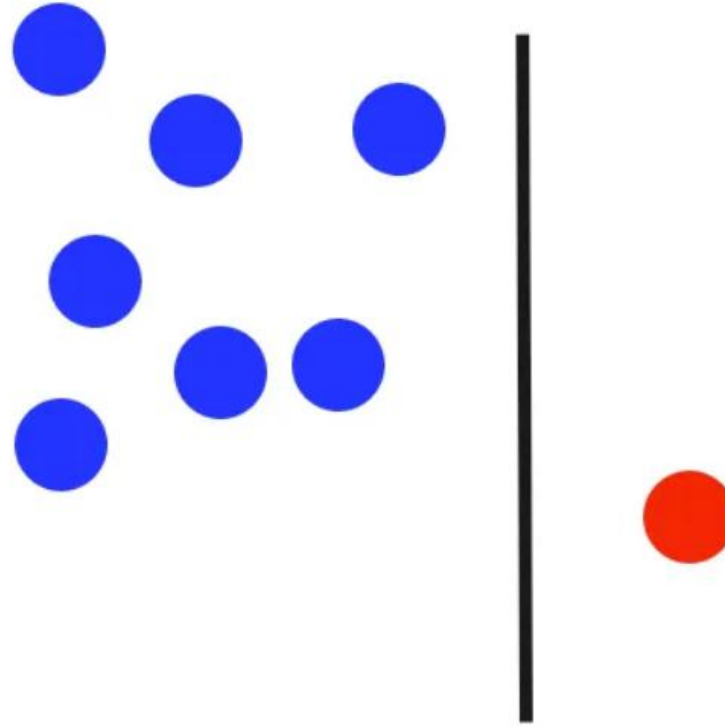
- After creating an ensemble of iTrees (Isolation Forest), the model training is complete. During scoring, the system traverses a data point through all the trees that were trained earlier. Now, an ‘anomaly score’ is assigned to each of the data points based on the depth of the tree required to arrive at that point. This score is an aggregation of the depth obtained from each of the iTrees. An anomaly score of -1 assigns anomalies and 1 to normal points based on the contamination parameter (percentage of anomalies present in the data).

Algorithm

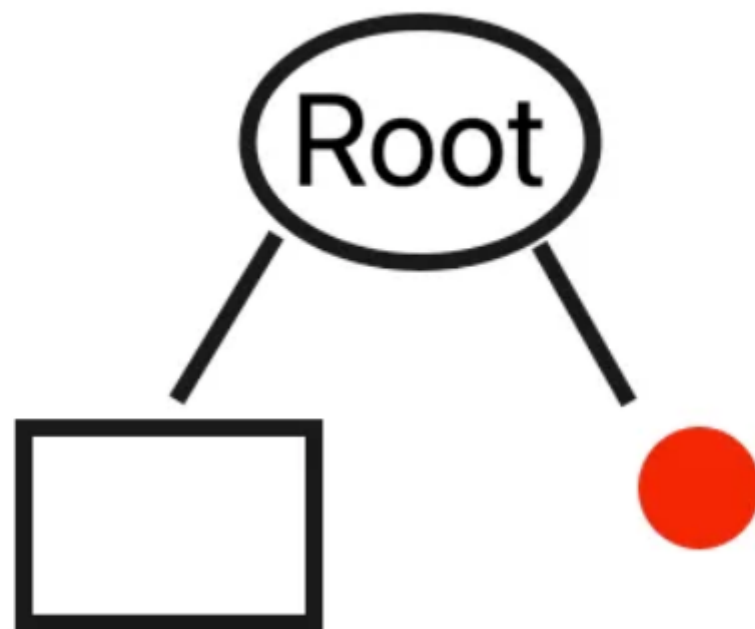
Suppose we had the following data points:



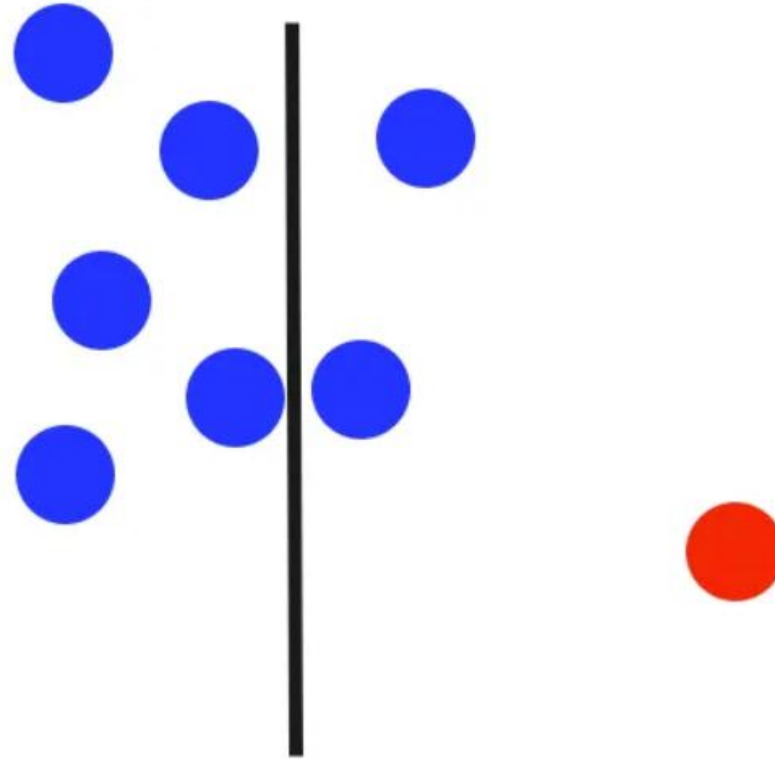
The isolation forest algorithm selects a random dimension (in this case, the dimension associated with the x axis) and randomly splits the data along that dimension.



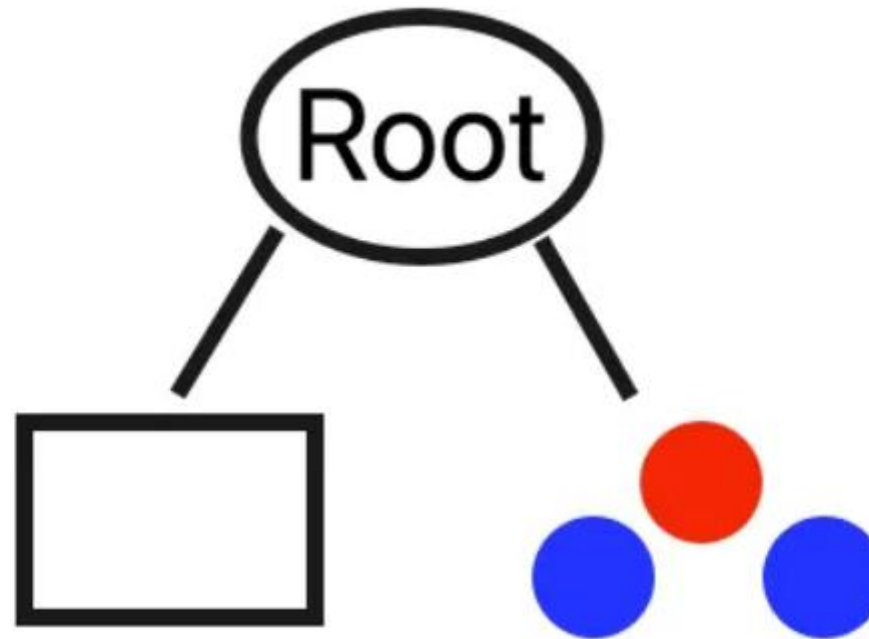
The two resulting subspaces define their own sub tree. In this example, the cut happens to separate a lone point from the remainder of the dataset. The first level of the resulting binary tree consists of two nodes, one which will consist of the subtree of points to the left of the initial cut and the other representing the single point on the right.



It's important to note, the other trees in the ensemble will select different starting splits. In the following example, the first split doesn't isolate the outlier.

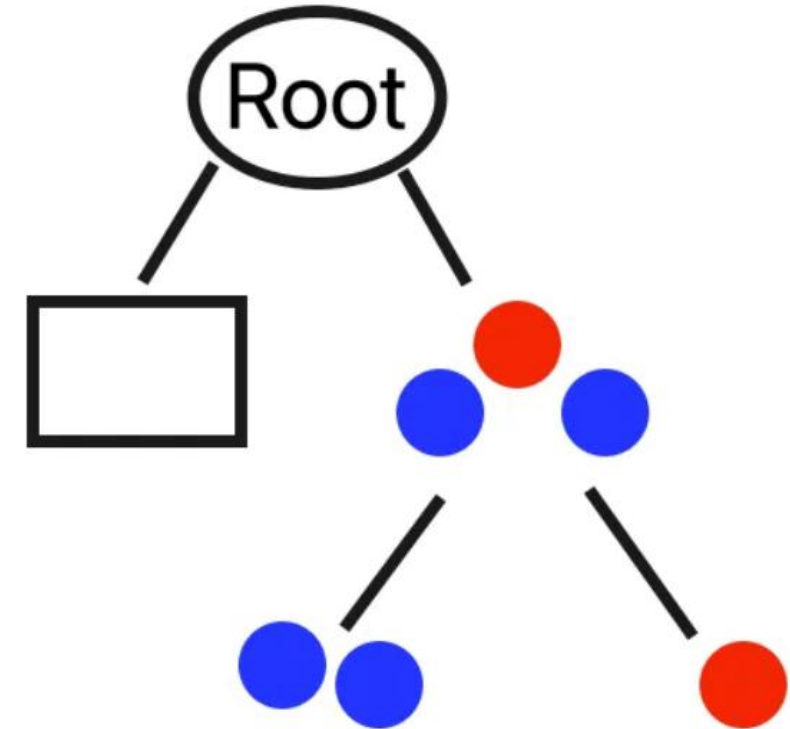
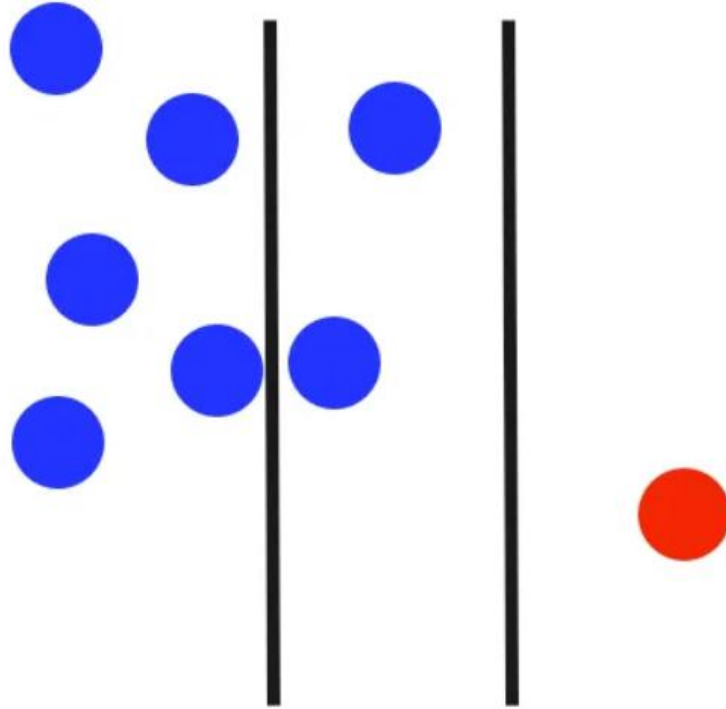


We end up with a tree consisting of two nodes, one that contains the points to the left of the line and the other representing the points on the right side of the line.

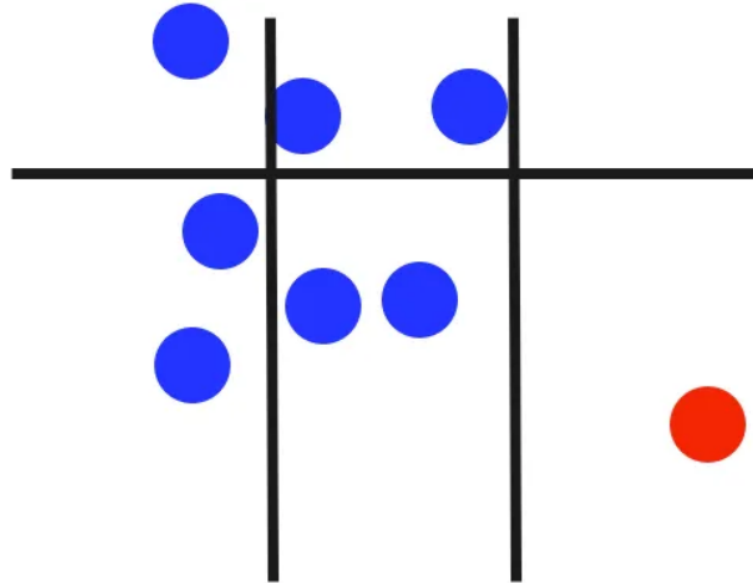


The process is repeated until every leaf of the tree represents a single data point from the dataset. In our example, the second iteration manages to isolate the outlier.

After this step, the tree would look as follows:



Remember that a split can occur along the other dimension as is the case for this 3rd decision tree.



On average, an anomalous data point is going to be isolated in a bounding box at a smaller tree depth than other points. When performing inference using a trained Isolation Forest model the final anomaly score is reported as the average across scores reported by each individual decision tree.

Anomaly Detection using Isolation Forest

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest

# Generate synthetic data
np.random.seed(42)
normal_data = np.random.randn(1000, 2) * 2 #
Normal data points
anomaly_data = np.random.randn(50, 2) * 10 #
Anomaly data points

# Combine normal and anomaly data
data = np.vstack((normal_data,
anomaly_data))

# Fit Isolation Forest model
model =
IsolationForest(contamination=0.05)
# Contamination parameter sets
expected proportion of anomalies
model.fit(data)
```



- # Predict anomalies
- `predictions = model.predict(data)`
- # Plot the data and detected anomalies
- `plt.scatter(data[:, 0], data[:, 1], c=predictions, cmap=plt.cm.Paired)`
- `plt.title("Anomaly Detection using Isolation Forest")`
- `plt.xlabel("Feature 1")`
- `plt.ylabel("Feature 2")`
- `plt.show()`

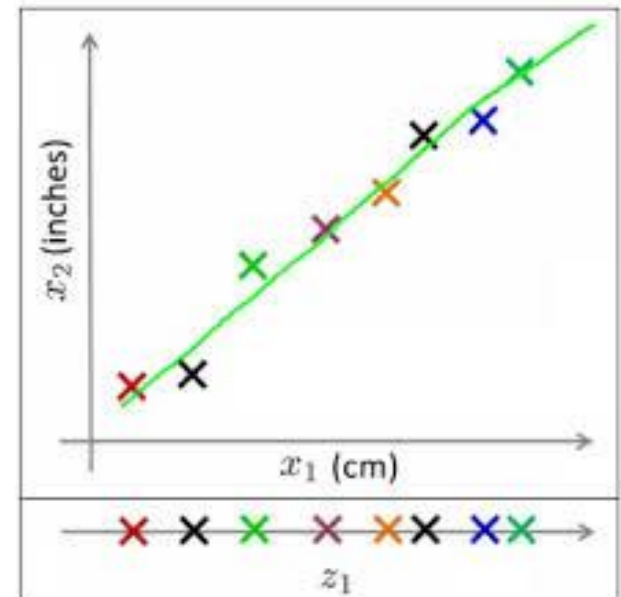


- **Write a program to detect anomalies in iris dataset using Isolation Forest.**
- 1. Load the Iris dataset
- 2. Preprocess data if needed
- 3. initialize isolation forest model
- 4. Set appropriate contamination parameter
- 5. Train model and make predictions
- 6. Evaluate and visualize results

Principal Component Analysis

Dimensionality Reduction

- Finds a low-dimensional representation of the data that retains as much information as possible.
- Eg: two dimensions x_1 and x_2 converted into one dimension i.e z_1 .



Getting rid of Unnecessary

House No.	Value	Area	Floors	Household
1	148	72	4	20
2	156	76	4	22
3	160	86	4	22
4	165	79	4	24
5	169	88	5	30
6	184	90	5	35

Sample Variance (s^2)

$$s^2 = \frac{\sum (x_i - \bar{x})^2}{n-1}$$

s^2 = variance

x_i = term in data set

$\bar{x}$ = Sample mean

$\sum$ = Sum

n = Sample size

Getting rid of Unnecessary

- Can you characterize each house by its floor count?
- No, as they have low variance, $\sigma^2=0.2$
- Can you characterize each house by its households?
- May be, as its variance is $\sigma^2=28$
- Area has the variance 43.
- Value has the variance 127

So, Area and Value are more representative than other two features.

Getting rid of Unnecessary

- What about the relation between features?
- Value is roughly double the Area.
- i.e. covariance is high in between Area and Value.
- Higher the covariance, the more correlated the two features are, which implies redundancy in the data.

Dimensionality Reduction via Feature Extraction

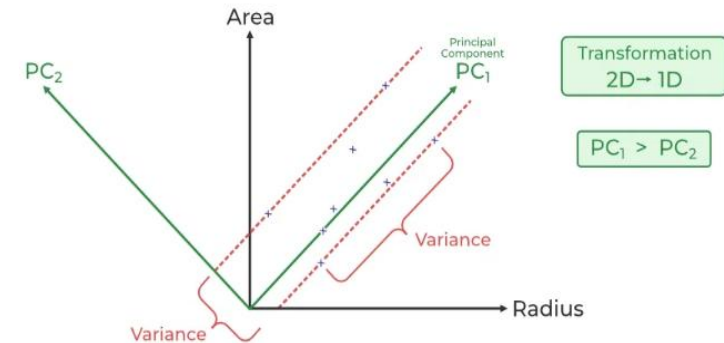
- **Principal Component Analysis (PCA):** unsupervised data compression
- **Linear Discriminant Analysis (LDA):** supervised technique for maximizing class separability.
- **Kernal Principal Component Analysis: Nonlinear Dimensionality Reduction**

Principal Component Analysis

- **Principal Component Analysis (PCA) is a statistical procedure that uses an orthogonal transformation that converts a set of correlated variables to a set of uncorrelated variables.**
- **Unsupervised linear transformation technique**
- **Identify patterns in data based on the correlation between features.**
- **The main goal of Principal Component Analysis (PCA) is to reduce the dimensionality of a dataset while preserving the most important patterns or relationships between the variables without any prior knowledge of the target variables.**

- **Principal Component Analysis (PCA) is used to reduce the dimensionality of a data set by finding a new set of variables, smaller than the original set of variables, retaining most of the sample's information.**

- **Principal Component Analysis (PCA)** is a technique for dimensionality reduction that identifies a set of orthogonal axes, called principal components, that capture the maximum variance in the data.
- The principal components are linear combinations of the original variables in the dataset and are ordered in decreasing order of importance.
- The first principal component captures the most variation in the data, but the second principal component captures the maximum variance that is orthogonal to the first principal component, and so on.



Steps of PCA

1. Standardize the d -dimensional dataset.
2. Construct the covariance matrix.
3. Decompose the covariance matrix into its eigenvectors and eigenvalues.
4. Select k eigenvectors that correspond to the k largest eigenvalues, where k is the dimensionality of the new feature space ($k \leq d$).
5. Construct a projection matrix W from the “top” k eigenvectors.
6. Transform the d -dimensional input dataset X using the projection matrix W to obtain the new k -dimensional feature subspace.

After transformation:

- The first **principal component will have the largest variance.**
- All principal components will have largest possible variance given that they are uncorrelated to the other principal components.
- PCA directions are highly sensitive to data scaling, so *features* must be *standardized* prior to PCA.

Clustering metrics

PART-A

Clustering metrics

- Clustering metrics are quantitative measures used to evaluate the quality of clustering algorithms and the resulting clusters.
- **Internal Evaluation Metrics:** These metrics evaluate the quality of clusters without any external reference. They measure the compactness of data points within the same cluster and the separation between different clusters.
- **Such as:** Silhouette Score, Davies-Bouldin Index, Calinski-Harabasz Index (Variance Ratio Criterion), Dunn Index
- **External Evaluation Metrics:** These metrics require a ground truth or a reference set of labels to compare the clustering results against. They measure the agreement between the generated clusters and the true classes in the reference set.
- **Such as:** Adjusted Rand Index (ARI), Normalized Mutual Information (NMI), Fowlkes-Mallows Index

Silhouette Score

- Silhouette Coefficient or silhouette score is a metric used to calculate the **goodness** of a clustering technique. This score is calculated by measuring each data point's similarity to the cluster it belongs to and how different it is from other clusters. The Silhouette score is commonly used to assess the performance of clustering algorithms like K-Means.
- Its value ranges from **-1 to 1**.
- 1: Means clusters are well apart from each other and clearly distinguished (**Best Case**)
- 0: Means clusters are indifferent, or we can say that the distance between clusters is not significant (**Overlapping Clusters**)
- -1: Means clusters are assigned in the wrong way (**Worst Case**)

It is computed as:

- The silhouette value is a measure of how similar an object is to its own cluster (**cohesion**) compared to other clusters (**separation**).
- Silhouette Score = $(b-a)/\max(a,b)$

where

- a = average intra-cluster distance i.e the average distance between each point within a cluster
- b = average inter-cluster distance i.e the average distance between all clusters

Example:

Datapoints	Cluster Label
A1	C1
A2	C1
A3	C2
A4	C2

Datapoint	A1	A2	A3	A4
A1	0	0.10	0.65	0.55
A2	0.10	0	0.70	0.60
A3	0.65	0.70	0	0.30
A4	0.55	0.60	0.30	0

For point A1:

$$a = 0.1/1 = 0.1$$

$$b = (0.65 + 0.55)/2 = 0.6$$

$$\text{Silhouette Score} = (b - a) / \max(a, b)$$

$$(0.6 - 0.1) / 0.6 = 0.833$$

For point A2:

$$a = 0.1/1 = 0.1$$

$$b = (0.70 + 0.60)/2 = 0.65$$

$$\text{Silhouette Score} = (b - a) / \max(a, b)$$

$$(0.65 - 0.1) / 0.65 = 0.846$$

For point A3:

$$a = 0.30/1 = 0.30$$

$$b = (0.65 + 0.70)/2 = 0.675$$

$$\text{Silhouette Score} = (b - a) / \max(a, b)$$

$$(0.675 - 0.30) / 0.675 = 0.555$$

For point A4:

$$a = 0.30/1 = 0.30$$

$$b = (0.55 + 0.60)/2 = 0.575$$

$$\text{Silhouette Score} = (b - a) / \max(a, b)$$

$$(0.575 - 0.30) / 0.575 = 0.478$$

Point A1 and A2 are lying cluster C1 , so for computing Silhouette Score for cluster C1,

$$(0.833+0.846)/2= 1.679/2= 0.839$$

Point A3 and A4 are lying cluster C2 , so for computing Silhouette Score for cluster C2,

$$(0.555+0.478)/2= 1.033/2= 0.5165$$

Silhouette Score/Coefficient for overall clustering problem is:

$$(0.839+0.5165)/2= 0.6775 \text{ or } 0.678$$

Example:

To illustrate the calculation of the silhouette coefficient, let's consider a scenario with 3 clusters, where each cluster has 3 (2-dimensional points).

Cluster 1:

Point A1: (2, 5)

Point A2: (3, 4)

Point A3: (4, 6)

Cluster 2:

Point B1: (8, 3)

Point B2: (9, 2)

Point B3: (10, 5)

Cluster 3:

Point C1: (6, 10)

Point C2: (7, 8)

Point C3: (8, 9)

Davies-Bouldin Index

- The Davies-Bouldin index (DBI) is a metric for assessing the separation and compactness of clusters.
- It is based on the idea that good clusters are those that have low within-cluster variation and high between-cluster separation.
- The minimum score is zero, with lower values indicating better clustering.
- Lower the DB index value, better is the clustering.

$$\text{DB index}(U) = 1/k \sum_{i=1}^k \max_{i \neq j} \left\{ \frac{\Delta(X_i) + \Delta(X_j)}{\delta(X_i, X_j)} \right\}$$

Where,

$\delta(X_i, X_j)$ is the intercluster distance i.e.
the distance between cluster X_i and X_j

$\Delta(X_k)$ is the intracluster distance of
cluster X_k i.e. distance within the cluster X_k

Example

- Data Point 1: (2, 3)
- Data Point 2: (3, 2)
- Data Point 3: (8, 8)
- Data Point 4: (9, 7)
- Data Point 5: (15, 14)
- Data Point 6: (16, 13)

- Let's say we want to perform K-means clustering on this dataset with $K = 2$. After clustering, the data points are divided into two clusters:
- Cluster 1: {Data Point 1, Data Point 2} Cluster 2: {Data Point 3, Data Point 4, Data Point 5, Data Point 6}
- The centroids of these clusters are approximately:
- Centroid of Cluster 1: (2.5, 2.5) Centroid of Cluster 2: (12, 10.5)
- Now, let's calculate the pairwise distances between the centroids of each cluster:

- Now, let's calculate the pairwise distances between the centroids of each cluster:

The centroids of your clusters are:

- Centroid of Cluster 1: (2.5, 2.5)
- Centroid of Cluster 2: (12, 10.5)

To find the Euclidean distance between these centroids:

$$\text{Distance} = \sqrt{(12 - 2.5)^2 + (10.5 - 2.5)^2}$$

Substituting the values:

$$\text{Distance} = \sqrt{(9.5)^2 + (8)^2} = \sqrt{90.25 + 64} = \sqrt{154.25} \approx 12.42$$

For Cluster 1:

- Points: (2, 3) and (3, 2)
- Centroid: (2.5, 2.5)

Calculate the distances:

1. Distance from (2, 3) to centroid:

$$\sqrt{(2 - 2.5)^2 + (3 - 2.5)^2} = \sqrt{0.25 + 0.25} = \sqrt{0.5} \approx 0.71$$

2. Distance from (3, 2) to centroid:

$$\sqrt{(3 - 2.5)^2 + (2 - 2.5)^2} = \sqrt{0.25 + 0.25} = \sqrt{0.5} \approx 0.71$$

Average Scatter for Cluster 1:

$$\text{Scatter}_1 = \frac{0.71 + 0.71}{2} = 0.71$$

For Cluster 2:

- Points: (8, 8), (9, 7), (15, 14), (16, 13)
- Centroid: (12, 10.5)

Calculate each distance:

1. Distance from (8, 8) to centroid:

$$\sqrt{(8 - 12)^2 + (8 - 10.5)^2} = \sqrt{16 + 6.25} = \sqrt{22.25} \approx 4.72$$

2. Distance from (9, 7) to centroid:

$$\sqrt{(9 - 12)^2 + (7 - 10.5)^2} = \sqrt{9 + 12.25} = \sqrt{21.25} \approx 4.61$$

3. Distance from (15, 14) to centroid:

$$\sqrt{(15 - 12)^2 + (14 - 10.5)^2} = \sqrt{9 + 12.25} = \sqrt{21.25} \approx 4.61$$

4. Distance from (16, 13) to centroid:

$$\sqrt{(16 - 12)^2 + (13 - 10.5)^2} = \sqrt{16 + 6.25} = \sqrt{22.25} \approx 4.72$$

Average Scatter for Cluster 2:

$$\text{Scatter}_2 = \frac{4.72 + 4.61 + 4.61 + 4.72}{4} = 4.67$$

Step 3: Calculate the Davies-Bouldin Index (DBI)

The DBI for two clusters is calculated as:

$$\text{DBI} = \frac{\text{Scatter}_1 + \text{Scatter}_2}{\text{Distance between Centroids}}$$

Substituting the values:

$$\text{DBI} = \frac{0.71 + 4.67}{12.42} \approx \frac{5.38}{12.42} \approx 0.43$$

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import
davies_bouldin_score
data1 = np.array([[6, 8], [9, 5], [5, 4], [2,
6], [5, 6], [3, 4]])
labels1 = np.array([0, 0, 1, 1, 1, 0])
davies_bouldin_index =
davies_bouldin_score(data1, labels1)
print("Davies-Bouldin Index:",
davies_bouldin_index)
```

Dunn index:

- A metric for evaluating clustering algorithms, is an internal evaluation scheme, where the result is based on the clustered data itself.
- Like all other such indices, the aim of this Dunn index to identify sets of clusters that are compact, with a small variance between members of the cluster, and well separated, where the means of different clusters are sufficiently far apart, as compared to the within cluster variance.
- Higher the Dunn index value, better is the clustering.
- The number of clusters that maximizes Dunn index is taken as the optimal number of clusters k .

$$\text{Dunn index}(U) = \min_{1 \leq i \leq c} \left\{ \min_{1 \leq j \leq c, j \neq i} \left\{ \frac{\delta(X_i, X_j)}{\max_{1 \leq k \leq c} \{\Delta(X_k)\}} \right\} \right\}$$

Where,

$\delta(X_i, X_j)$ is the intercluster distance i.e. the distance between cluster X_i and X_j

$\Delta(X_k)$ is the intracluster distance of cluster X_k i.e. distance within the cluster X_k

- A higher Dunn Index value indicates better clustering, as it implies that the clusters are compact and well-separated. Let's go through a numerical example to illustrate the concept.
- Consider a dataset with 8 data points in a 2D space:
- Data Point 1: (2, 3) Data Point 2: (3, 2) Data Point 3: (4, 3) Data Point 4: (8, 8) Data Point 5: (9, 7) Data Point 6: (10, 8) Data Point 7: (15, 14) Data Point 8: (16, 13)

- Let's say we want to perform K-means clustering on this dataset with $K = 2$. After clustering, the data points are divided into two clusters:
- Cluster 1: {Data Point 1, Data Point 2, Data Point 3} Cluster 2: {Data Point 4, Data Point 5, Data Point 6, Data Point 7, Data Point 8}
- Now, let's calculate the inter-cluster distances (minimum distance between any two points from different clusters) and intra-cluster distances (maximum distance between any two points within the same cluster):

1. Calculate the Cluster Centroids

Cluster 1: $\{(2, 3), (3, 2), (4, 3)\}$

Centroid of Cluster 1: $C_1 = \left(\frac{2+3+4}{3}, \frac{3+2+3}{3}\right) = \left(\frac{9}{3}, \frac{8}{3}\right) = \left(3, \frac{8}{3}\right) \approx (3, 2.67)$

Cluster 2: $\{(8, 8), (9, 7), (10, 8), (15, 14), (16, 13)\}$

Centroid of Cluster 2: $C_2 = \left(\frac{8+9+10+15+16}{5}, \frac{8+7+8+14+13}{5}\right) = \left(\frac{58}{5}, \frac{50}{5}\right) = (11.6, 10)$

2. Calculate the Distance Between Centroids

Using the Euclidean distance formula: $d(C_1, C_2) = \sqrt{(11.6 - 3)^2 + (10 - 2.67)^2}$

$$d(C_1, C_2) = \sqrt{(8.6)^2 + (7.33)^2} \quad d(C_1, C_2) = \sqrt{73.96 + 53.53} \quad d(C_1, C_2) = \sqrt{127.49} \approx 11.29$$

3. Calculate the Maximum Intra-Cluster Distance

Cluster 1: Compute the pairwise distances:

- Distance between (2, 3) and (3, 2): $\sqrt{(2-3)^2 + (3-2)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41$
- Distance between (2, 3) and (4, 3): $\sqrt{(2-4)^2 + (3-3)^2} = \sqrt{4} = 2$
- Distance between (3, 2) and (4, 3): $\sqrt{(3-4)^2 + (2-3)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41$

Maximum intra-cluster distance for Cluster 1 is 2.

Cluster 2: Compute the pairwise distances:

- Distance between (8, 8) and (9, 7): $\sqrt{(8-9)^2 + (8-7)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41$
- Distance between (8, 8) and (10, 8): $\sqrt{(8-10)^2 + (8-8)^2} = \sqrt{4} = 2$
- Distance between (8, 8) and (15, 14): $\sqrt{(8-15)^2 + (8-14)^2} = \sqrt{49+36} = \sqrt{85} \approx 9.22$
- Distance between (8, 8) and (16, 13): $\sqrt{(8-16)^2 + (8-13)^2} = \sqrt{64+25} = \sqrt{89} \approx 9.43$
- Distance between (9, 7) and (10, 8): $\sqrt{(9-10)^2 + (7-8)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41$
- Distance between (9, 7) and (15, 14): $\sqrt{(9-15)^2 + (7-14)^2} = \sqrt{36+49} = \sqrt{85} \approx 9.22$
- Distance between (9, 7) and (16, 13): $\sqrt{(9-16)^2 + (7-13)^2} = \sqrt{49+36} = \sqrt{85} \approx 9.22$
- Distance between (10, 8) and (15, 14): $\sqrt{(10-15)^2 + (8-14)^2} = \sqrt{25+36} = \sqrt{61} \approx 7.81$
- Distance between (10, 8) and (16, 13): $\sqrt{(10-16)^2 + (8-13)^2} = \sqrt{36+25} = \sqrt{61} \approx 7.81$
- Distance between (15, 14) and (16, 13): $\sqrt{(15-16)^2 + (14-13)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41$

Maximum intra-cluster distance for Cluster 2 is approximately 9.43.

4. Compute the Dunn Index

The Dunn Index is calculated as follows: $\text{Dunn Index} = \frac{\text{Minimum Inter-Cluster Distance}}{\text{Maximum Intra-Cluster Distance}}$

$$\text{Dunn Index} = \frac{11.29}{9.43} \approx 1.20$$

So, the Dunn Index for the given clustering is approximately 1.20.

Example

- Let's work through a numerical example to calculate the Dunn Index for a simple set of clusters. Suppose you have the following data points and their corresponding clusters:

Data Points:

- A(1, 2)
- B(2, 2)
- C(5, 8)
- D(6, 8)
- E(10, 12)
- F(11, 12)

Clusters:

- Cluster 1: {A, B}
- Cluster 2: {C, D}
- Cluster 3: {E, F}

- **Step 1: Understand the Dunn Index Formula**
- The Dunn Index measures cluster quality as:
Dunn Index = min(inter-cluster distance) / max(intra-cluster distance)
- Where:
- **Inter-cluster distance** = minimum distance between any two points in different clusters
- **Intra-cluster distance** = maximum distance between any two points within the same cluster

- **Step 2: Calculate Intra-Cluster Distances (Diameter)**

- **Cluster 1: {A(1,2), B(2,2)}**

- Distance AB = $\sqrt{[(2-1)^2 + (2-2)^2]} = \sqrt{1} = 1$

- **Diameter₁ = 1**

- **Cluster 2: {C(5,8), D(6,8)}**

- Distance CD = $\sqrt{[(6-5)^2 + (8-8)^2]} = \sqrt{1} = 1$

- **Diameter₂ = 1**

- **Cluster 3: {E(10,12), F(11,12)}**

- Distance EF = $\sqrt{[(11-10)^2 + (12-12)^2]} = \sqrt{1} = 1$

- **Diameter₃ = 1**

- **Max intra-cluster distance = max(1, 1, 1) = 1**

- **Step 3: Calculate Inter-Cluster Distances**
- **Between Cluster 1 & Cluster 2:**
 - $AC = \sqrt{(5-1)^2 + (8-2)^2} = \sqrt{16 + 36} = \sqrt{52} \approx 7.21$
 - $AD = \sqrt{(6-1)^2 + (8-2)^2} = \sqrt{25 + 36} = \sqrt{61} \approx 7.81$
 - $BC = \sqrt{(5-2)^2 + (8-2)^2} = \sqrt{9 + 36} = \sqrt{45} \approx 6.71$
 - $BD = \sqrt{(6-2)^2 + (8-2)^2} = \sqrt{16 + 36} = \sqrt{52} \approx 7.21$
 - **Min distance = 6.71**
- **Between Cluster 1 & Cluster 3:**
 - $AE = \sqrt{(10-1)^2 + (12-2)^2} = \sqrt{81 + 100} = \sqrt{181} \approx 13.45$
 - $AF = \sqrt{(11-1)^2 + (12-2)^2} = \sqrt{100 + 100} = \sqrt{200} \approx 14.14$
 - $BE = \sqrt{(10-2)^2 + (12-2)^2} = \sqrt{64 + 100} = \sqrt{164} \approx 12.81$
 - $BF = \sqrt{(11-2)^2 + (12-2)^2} = \sqrt{81 + 100} = \sqrt{181} \approx 13.45$
 - **Min distance = 12.81**

- **Between Cluster 2 & Cluster 3:**
- $CE = \sqrt{(10-5)^2 + (12-8)^2} = \sqrt{25 + 16} = \sqrt{41} \approx 6.40$
- $CF = \sqrt{(11-5)^2 + (12-8)^2} = \sqrt{36 + 16} = \sqrt{52} \approx 7.21$
- $DE = \sqrt{(10-6)^2 + (12-8)^2} = \sqrt{16 + 16} = \sqrt{32} \approx 5.66$
- $DF = \sqrt{(11-6)^2 + (12-8)^2} = \sqrt{25 + 16} = \sqrt{41} \approx 6.40$
- **Min distance = 5.66**
- **Min inter-cluster distance = $\min(6.71, 12.81, 5.66) = 5.66$**

- **Step 4: Calculate Dunn Index**
- **Dunn Index = min(inter-cluster distance) / max(intra-cluster distance)**
Dunn Index = 5.66 / 1 = 5.66
- **Interpretation**
- The Dunn Index of **5.66** indicates good cluster separation because:
- A higher Dunn Index suggests better clustering
- The clusters are well-separated (minimum distance between clusters is much larger than maximum distance within clusters)
- In this case, clusters are compact (diameter = 1) and well-separated (minimum gap = 5.66)

- `from sklearn.metrics import pairwise_distances`
- `import numpy as np`
- `points = np.array([[1,2], [2,2], [5,8], [6,8], [10,12], [11,12]])`
- `labels = np.array([1, 1, 2, 2, 3, 3])`
- *# One-liner Dunn Index calculation*
- `dunn = (min([np.min(pairwise_distances(points[labels==i], points[labels==j]))`
- `for i in [1,2,3] for j in [1,2,3] if i < j]) /`
- `max([np.max(pairwise_distances(points[labels==l])) for l in [1,2,3])))`
- `print(f"Dunn Index: {dunn:.2f}")`
- #OUTPUT- Dunn Index: 5.66

Drawbacks of Dunn index:

- As the number of clusters and dimensionality of the data increase, the computational cost also increases.

Clustering metrics

PART-B

Adjusted Rand Index (ARI)

- It measures the similarity between the true class labels and the clusters generated by a clustering algorithm while accounting for chance agreement.
- The ARI produces a score between -1 and 1, where higher values indicate better agreement between the predicted clusters and the true labels.
- Contingency table- $m \times n$
- m = number of clusters by c1 algo.
- n = number of clusters by c2 algo (Ground Truth)

To use the Adjusted Rand Index for evaluating clustering, follow these steps:

Perform Clustering: Apply a clustering algorithm to your dataset to create clusters of data points. This could be an algorithm like k-means, hierarchical clustering, DBSCAN, or any other clustering technique.

Obtain True Class Labels: If you have access to ground truth labels or class assignments for your data, this is ideal. These true labels represent the actual groups or categories that your data points belong to.

$$ARI = \frac{RI - \text{Expected RI}}{\text{Max(RI)} - \text{Expected RI}}$$

Let's say we have 6 data points and two different clustering results:

True Labels (Ground Truth):

[A, A, A, B, B, B]

Cluster 1 (Good clustering):

[1, 1, 1, 2, 2, 2]

Cluster 2 (Poor clustering):

[1, 1, 2, 2, 3, 3]

1. Create Contingency Table for Cluster 1

	Cluster 1-1	Cluster 1-2
True A	3	0
True B	0	3

2. Calculate Pair Counting

We count pairs of points that are:

- **a**: Same cluster in both (true positives)
- **b**: Different clusters in both (true negatives)
- **c**: Same in true, different in clustering (false negatives)
- **d**: Different in true, same in clustering (false positives)

For Cluster 1:

- $a = C(3,2) + C(3,2) = 3 + 3 = 6$
- $b = (3 \times 3) = 9$
- $c = 0$
- $d = 0$

3. Calculate Rand Index (RI)

$$RI = (a + b) / (a + b + c + d) = (6 + 9) / (6 + 9 + 0 + 0) = 15/15 = 1.0$$

4. Calculate ARI

$$ARI = (RI - \text{Expected_RI}) / (\text{Max_RI} - \text{Expected_RI})$$

For this perfect clustering, $ARI = 1.0$

Compute Adjusted rand Index

1. Adjusted Rand Index (ARI)

```
from sklearn import metrics
```

```
import numpy as np
```

```
# Sample data
```

```
true_labels = [0, 0, 0, 1, 1, 1, 2, 2, 2]
```

```
pred_labels = [0, 0, 1, 1, 1, 2, 2, 2, 2]
```

```
# Calculate ARI
```

```
ari_score = metrics.adjusted_rand_score(true_labels, pred_labels)
```

```
print(f"Adjusted Rand Index: {ari_score:.3f}")
```

```
# Output: Adjusted Rand Index: 0.444
```


Normalized Mutual Information (NMI)

- Normalized Mutual Information (NMI) is a normalization of the Mutual Information (MI) score to scale the results between 0 (no mutual information) and 1 (perfect correlation).

NMI

- NMI is a good measure for determining the quality of clustering.
- It is an external measure because we need the class labels of the instances to determine the NMI.
- Since it's normalized we can measure and compare the NMI between different clusterings having different number of clusters.

Normalized Mutual Information

Scenario

We have 4 animals and two ways to group them:

True Groups (by type):

•Cats: 🐱, 🐱

•Dogs: 🐶, 🐶

Clustering Result (by color):

•Black: 🐱, 🐶

•White: 🐱, 🐶

Normalized Mutual Information

Step 1: Create Contingency Table

	Black Cluster	White Cluster	Total
Cats	1	1	2
Dogs	1	1	2
Total	2	2	4

Step 2: Simple Calculation

Perfect clustering would be:

- All cats together, all dogs together
- **NMI = 1.0**

Our clustering (by color):

- Mixed cats and dogs in both clusters

NMI = 0.0 (random grouping)

Normalized Mutual Information

Step 3: Python Code

```
from sklearn.metrics import normalized_mutual_info_score
import numpy as np

# Just 2 animals
true_labels = np.array([0, 1]) # Cat, Dog

# Random clustering (wrong)
clustering_wrong = np.array([0, 0]) # Puts both in same cluster
nmi_wrong = normalized_mutual_info_score(true_labels, clustering_wrong)
print(f"Wrong clustering NMI: {nmi_wrong:.1f}") # Output: 0.0

# Perfect clustering
clustering_perfect = np.array([0, 1]) # Separate clusters
nmi_perfect = normalized_mutual_info_score(true_labels, clustering_perfect)
print(f"Perfect clustering NMI: {nmi_perfect:.1f}") # Output: 1.0
```

Homogeneity

- A clustering result satisfies homogeneity if all of its clusters contain only data points which are members of a single class.
- This metric is independent of the absolute values of the labels: a permutation of the class or cluster label values won't change the score value in any way.
- **Syntax** : `sklearn.metrics.homogeneity_score(labels_true, labels_pred)`
- *The Metric is not symmetric, switching label_true with label_pred will return the completeness_score.*

Parameters :

- labels_true:**<int array, shape = [n_samples]> : It accept the ground truth class labels to be used as a reference.
- labels_pred:** <array-like of shape (n_samples,)>: It accepts the cluster labels to evaluate.

Returns:

homogeneity:<float>: Its return the score between 0.0 and 1.0 stands for perfectly homogeneous labeling.

- To calculate homogeneity numerically, you can use the following formula:

$$H = 1 - \frac{H(C)}{H(T)}$$

Where:

- $H(C)$ is the entropy of the clustering results (how mixed the clusters are).
- $H(T)$ is the entropy of the true class labels (how mixed the original classes are).

1. Compute the entropy of the clustering results, $H(C)$:

$$H(C) = - \sum_{i=1}^K p_i \cdot \log(p_i)$$

Where:

- K is the number of clusters.
- p_i is the proportion of data points in cluster i .

2. Compute the entropy of the true class labels, $H(T)$:

$$H(T) = - \sum_{j=1}^C q_j \cdot \log(q_j)$$

Where:

- C is the number of true classes or categories.
- q_j is the proportion of data points belonging to true class j .

Completeness score

- This score is complementary to the previous one. Its purpose is to provide a piece of information about the assignment of samples belonging to the same class.
- More precisely, a good clustering algorithm should assign all samples with the same true label to the same cluster.

$$c = 1 - \frac{H(Y_{pred} | Y_{true})}{H(Y_{pred})}$$

Completeness portrays the closeness of the clustering algorithm to this (completeness_score) perfection.

This metric is autonomous of the outright values of the labels. A permutation of the cluster label values won't change the score value in any way.

`sklearn.metrics.completeness_score()`

Syntax: `sklearn.metrics.completeness_score(labels_true, labels_pred)`

- **labels_true:** *<int array, shape = [n_samples]>: It accepts the ground truth class labels to be used as a reference.*

- **labels_pred:** *<array-like of shape (n_samples,)>: It accepts the cluster labels to evaluate.*

Returns: *completeness score between 0.0 and 1.0. 1.0 stands for perfectly completeness labeling.*

V-Measure

One of the primary disadvantages of any clustering technique is that it is difficult to evaluate its performance. To tackle this problem, the metric of **V-Measure** was developed. The calculation of the V-Measure first requires the calculation of two terms:-

1.Homogeneity: A perfectly homogeneous clustering is one where each cluster has data-points belonging to the same class label. Homogeneity describes the closeness of the clustering algorithm to this perfection.

2.Completeness: A perfectly complete clustering is one where all data-points belonging to the same class are clustered into the same cluster. Completeness describes the closeness of the clustering algorithm to this perfection.

```
sklearn.metrics.v_measure_score(labels_true, labels_pred, *, beta=1.0)
```

The V-measure is the harmonic mean between homogeneity and completeness:

$$v = \frac{(1 + \text{beta}) * \text{homogeneity} * \text{completeness}}{(\text{beta} * \text{homogeneity} + \text{completeness})}$$

Fowlkes-Mallows Score

- The **Fowlkes-Mallows Score** is an evaluation metric to **evaluate the similarity** among clustering's obtained after applying different clustering algorithms.
- Although technically it is used to quantify the similarity between two clustering's, it is typically used to evaluate the clustering performance of a clustering algorithm by assuming the second clustering to be the ground-truth i.e. the observed data and assuming it to be the perfect clustering.

The Fowlkes-Mallows index (FMI) is defined as the geometric mean between of the precision and recall:

$$FMI = TP / \sqrt{(TP + FP) * (TP + FN)}$$

- Where TP is the number of True Positive (i.e. the number of pair of points that belongs in the same clusters in both labels_true and labels_pred),
- FP is the number of False Positive (i.e. the number of pair of points that belongs in the same clusters in labels_true and not in labels_pred)
- and FN is the number of False Negative (i.e the number of pair of points that belongs in the same clusters in labels_pred and not in labels_True).

The score ranges from 0 to 1. A high value indicates a good similarity between two clusters.

$$FM = \sqrt{\frac{TP}{TP + FP} \cdot \frac{TP}{TP + FN}}$$

Adjusted Mutual Information (AMI)

- Adjusted Mutual Information (AMI) is an adjustment of the Mutual Information (MI) score to account for chance.
- It accounts for the fact that the MI is generally higher for two clusterings with a larger number of clusters, regardless of whether there is actually more information shared.
- For two clusterings U and V, the AMI is given as:
- $$AMI(U, V) = [MI(U, V) - E(MI(U, V))] / [\text{avg}(H(U), H(V)) - E(MI(U, V))]$$

- Write a program to evaluate the performance of clustering algorithms using different evaluation metrics.

Thank you!

UNIT- 4-PARTA

Reinforcement Learning

Introduction

Machine learning: Definition

- Machine learning is a scientific discipline that is concerned with the design and development of algorithms that allow computers to learn based on data, such as from sensor data or databases.

A major focus of machine learning research is to automatically learn to recognize complex patterns and make intelligent decisions based on data .

Machine learning Type:

With respect to the feedback type to learner:

- ❑ **Supervised learning** : Task Driven (Classification)
- ❑ **Unsupervised learning** : Data Driven (Clustering)
- ❑ **Reinforcement learning** —
 - Close to human learning.
 - Algorithm learns a policy of how to act in a given environment.
 - Every action has some impact in the environment, and the environment provides rewards that guides the learning algorithm.

Supervised Learning Vs Reinforcement Learning

Supervised Learning

- Step: 1

Teacher: Does picture 1 show a car or a flower?

Learner: A flower.

Teacher: No, it's a car.

Step: 2

Teacher: Does picture 2 show a car or a flower?

Learner: A car.

Teacher: Yes, it's a car.

Step: 3

Supervised Learning Vs Reinforcement Learning (Cont...)

Reinforcement Learning

Step: 1

World: You are in state 9. Choose action A or C.

Learner: Action A.

World: Your reward is 100.

Step: 2

World: You are in state 32. Choose action B or E.

Learner: Action B.

World: Your reward is 50.

Step: 3

Introduction (Cont..)

- **Meaning of Reinforcement:** Occurrence of an event, in the proper relation to a response, that tends to increase the probability that the response will occur again in the same situation.

Reinforcement learning is the problem faced by an agent that learns behavior through trial-and-error interactions with a dynamic environment.

- Reinforcement Learning is learning how to act in order to maximize a numerical reward.

Introduction (Cont..)

- **Reinforcement learning** is not a type of neural network, nor is it an alternative to neural networks. Rather, it is an orthogonal approach for Learning Machine.
- Reinforcement learning emphasizes learning feedback that evaluates the learner's performance **without providing standards of correctness** in the form of behavioral targets.

Example: Bicycle learning

Reinforcement Learning

- Reinforcement learning (RL) is a type of machine learning paradigm where an agent learns to make decisions by interacting with an environment.
- It is about learning the optimal behavior in an environment to obtain maximum reward.
- It is a machine learning paradigm that focuses on training intelligent agents to make sequences of decisions in order to maximize a cumulative reward signal.
- It is inspired by behavioral psychology and is often used in scenarios where an agent interacts with an environment to learn a policy that helps it achieve a specific goal over time.
- In RL, the data is accumulated from machine learning systems that use a trial-and-error method. Data is not part of the input that we would find in supervised or unsupervised machine learning.

Reinforcement learning uses algorithms that learn from outcomes and decide which action to take next.

After each action, the algorithm receives feedback that helps it determine whether the choice it made was correct, neutral or incorrect.

It is a good technique to use for automated systems that have to make a lot of small decisions without human guidance.

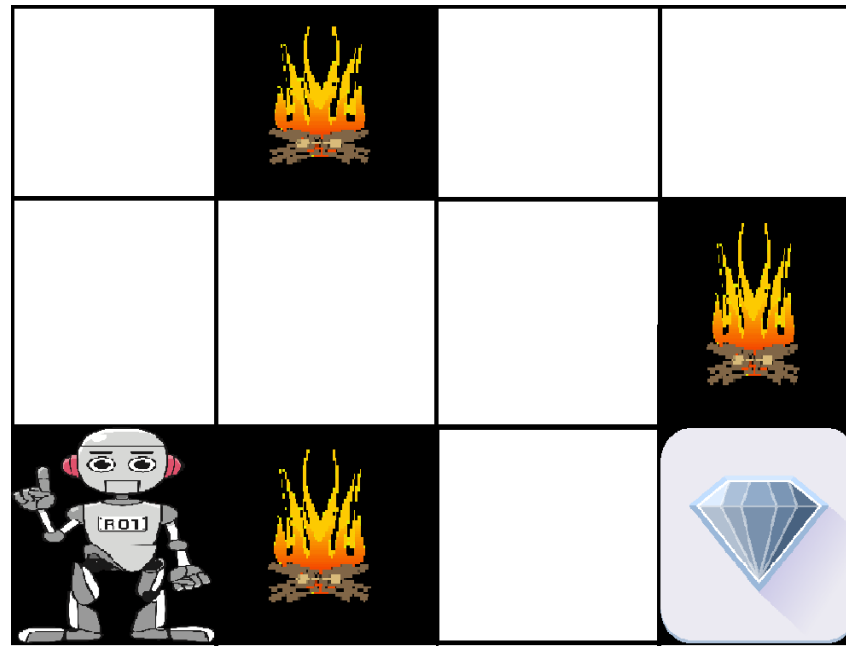
“Reinforcement learning is an autonomous, self-teaching system that essentially learns by trial and error. It performs actions with the aim of maximizing rewards, or in other words, it is learning by doing in order to achieve the best outcomes.”

Terms used in Reinforcement Learning

- **Agent():** An entity that can perceive/explore the environment and act upon it.
- **Environment():** A situation in which an agent is present or surrounded by. In RL, we assume the stochastic environment, which means it is random in nature.
- **Action():** Actions are the moves taken by an agent within the environment.
- **State():** State is a situation returned by the environment after each action taken by the agent.
- **Reward():** A feedback returned to the agent from the environment to evaluate the action of the agent.
- **Policy():** Policy is a strategy applied by the agent for the next action based on the current state.
- **Value():** It is expected long-term return with the discount factor and opposite to the short-term reward.
- **Q-value():** It is mostly similar to the value, but it takes one additional parameter as a current action (a).

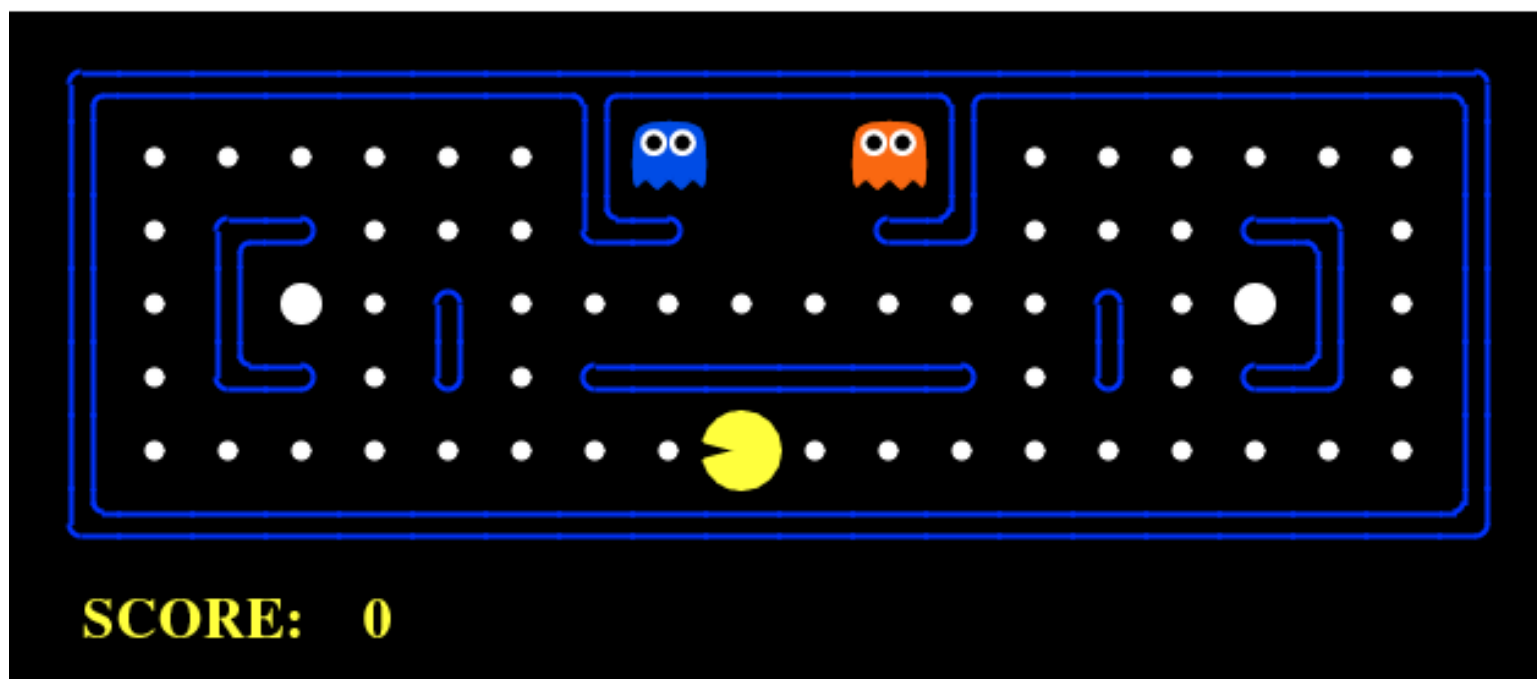
Example:

- The problem is as follows: We have an agent and a reward, with many hurdles in between. The agent is supposed to find the best possible path to reach the reward. The following problem explains the problem more easily.

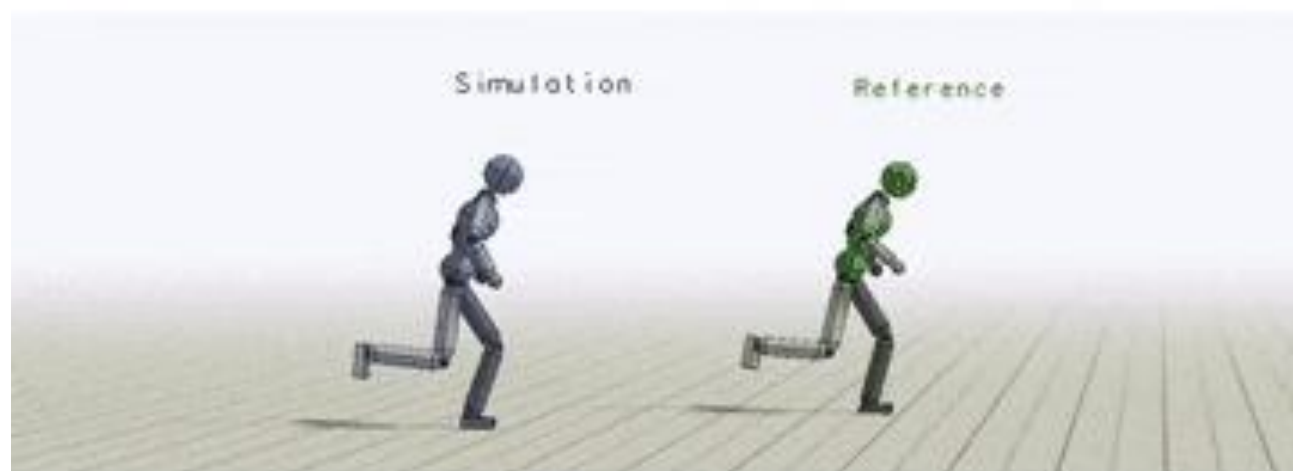


- The image shows the robot, diamond, and fire.
- The goal of the robot is to get the reward that is the diamond and avoid the hurdles that are fired.
- The robot learns by trying all the possible paths and then choosing the path which gives him the reward with the least hurdles.
- Each right step will give the robot a reward and each wrong step will subtract the reward of the robot.
- The total reward will be calculated when it reaches the final reward that is the diamond.

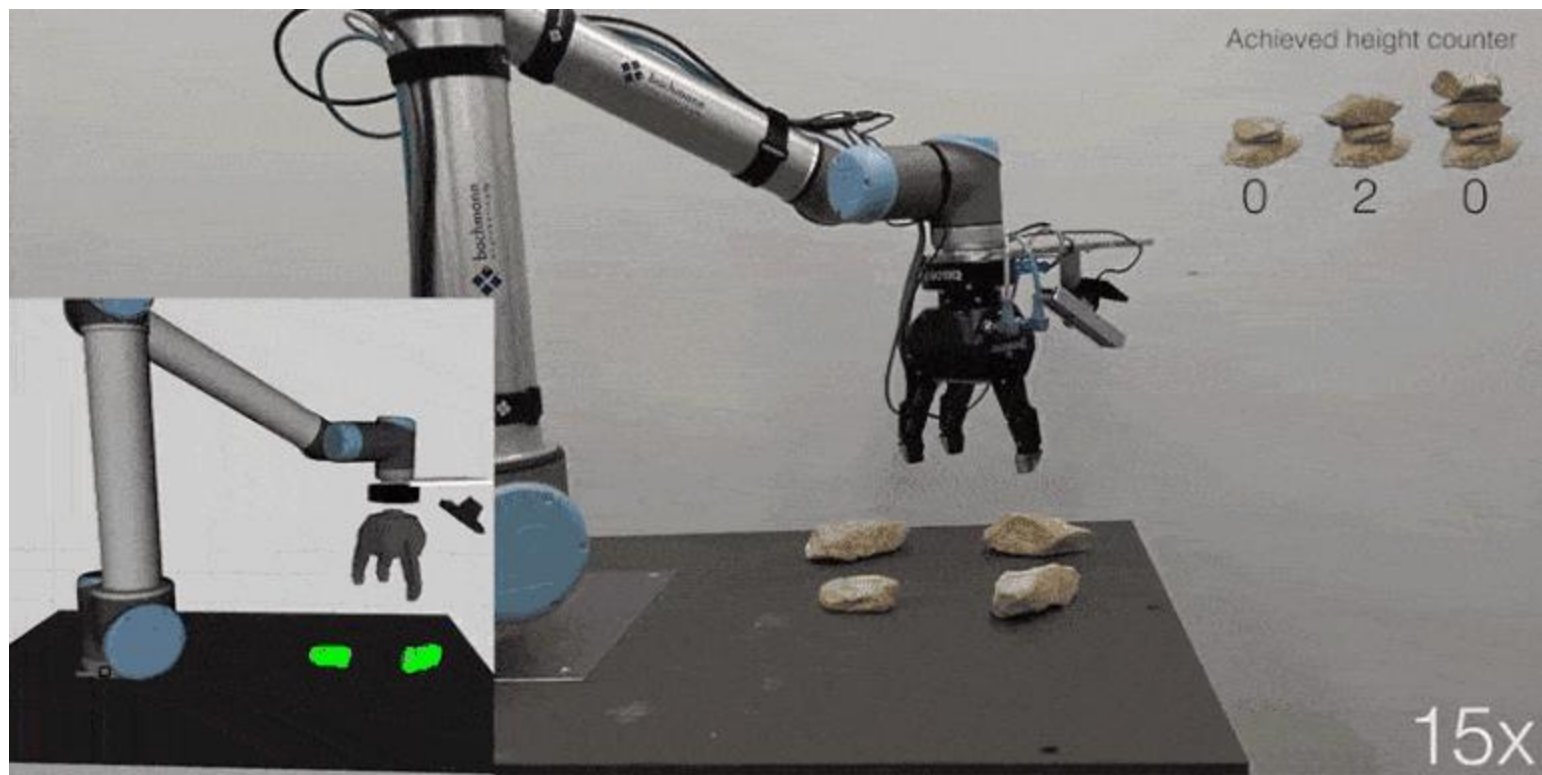
- An RL problem can be best explained through games. Let's take the game of PacMan where the goal of the agent(PacMan) is to eat the food in the grid while avoiding the ghosts on its way. In this case, the grid world is the interactive environment for the agent where it acts. Agent receives a reward for eating food and punishment if it gets killed by the ghost (loses the game). The states are the location of the agent in the grid world and the total cumulative reward is the agent winning the game.



Humanoid: Run

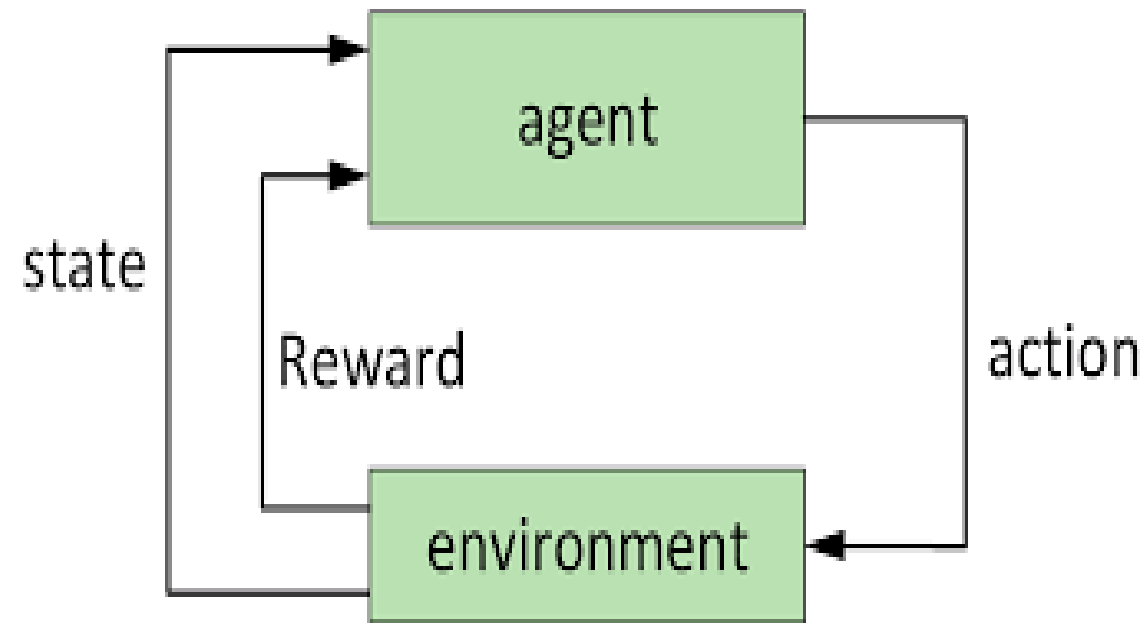


Policy trained to imitate a running clip.



Main points in Reinforcement learning:

- Input: The input should be an initial state from which the model will start.
- Output: There are many possible outputs as there are a variety of solutions to a particular problem.
- Training: The training is based upon the input, The model will return a state and the user will decide to reward or punish the model based on its output.
- The model keeps continues to learn.
- The best solution is decided based on the maximum reward.



Key Features of Reinforcement Learning

- In RL, the agent is not instructed about the environment and what actions need to be taken.
- It is based on the hit and trial process.
- The agent takes the next action and changes states according to the feedback of the previous action.
- The agent may get a delayed reward.
- The environment is stochastic, and the agent needs to explore it to reach to get the maximum positive rewards.

Approaches to implement Reinforcement Learning:

- There are mainly three ways to implement reinforcement-learning in ML, which are:

Value-based

Policy-based

Model-based

value-based approach: The value-based approach is about to find the optimal value function, which is the maximum value at a state under any policy. Therefore, the agent expects the long-term return at any state(s) under policy π .

Value-based reinforcement learning (RL) methods involve approximating the value function, which estimates the expected cumulative reward an agent can obtain from a given state or state-action pair under a certain policy. The value function helps the agent to make informed decisions by evaluating the desirability of different states or actions.

- **Policy-based:** Policy-based approach is to find the optimal policy for the maximum future rewards **without using the value function**. In this approach, the agent tries to apply such a policy that the action performed in each step helps to maximize the future reward.
- The policy-based approach has mainly two types of policy:
 - **Deterministic:** The same action is produced by the policy (π) at any state.
 - **Stochastic:** In this policy, **probability** determines the produced action.
- **Actor-Critic methods:** Combine aspects of both value-based and policy-based methods, utilizing both a policy (actor) and a value function (critic) to make more efficient and stable updates.
- **model-based approach:** In the model-based approach, a **virtual model** is created for the environment, and the agent explores that environment to learn it. There is no particular solution or algorithm for this approach because the model representation is different for each environment.

Elements of Reinforcement Learning

Reinforcement learning elements are as follows:

1. Policy
2. Reward function/Signal
3. Value function
4. Model of the environment

Policy:

- A policy can be defined as a way **how an agent behaves at a given time**. It maps the perceived states of the environment to the actions taken on those states
- Policy defines the learning agent behavior for given time period. It is a mapping from perceived states of the environment to actions to be taken when in those states.
- A policy is the core element of the RL as it alone can **define the behavior of the agent**.
- In some cases, it may be a simple function or a lookup table, whereas, for other cases, it may involve general computation as a search process. It could be deterministic or a stochastic policy:
- **For deterministic policy: $a = \pi(s)$**
For stochastic policy: $\pi(a | s) = P[A_t = a | S_t = s]$

Reward signal:

- The goal of reinforcement learning is defined by the reward signal.
- At each state, the environment sends an immediate signal to the learning agent, and this signal is known as a **reward signal**.
- These rewards are given according to the good and bad actions taken by the agent.
- The agent's main objective is to maximize the total number of rewards for good actions.
- The reward signal can change the policy, such as if an action selected by the agent leads to low reward, then the policy may change to select other actions in the future.

value function:

- The value function gives information about how **good the situation** and action are and **how much reward** an agent can expect.
- A reward indicates the **immediate signal for each good and bad action**, whereas a value function specifies **the good state and action for the future**.
- The value function depends on the reward as, without reward, there could be no value. The goal of estimating values is to achieve more rewards.

Model:

- The last element of reinforcement learning is the model, which mimics the behavior of the environment.
- With the help of the model, one can make inferences about how the environment will behave. Such as, if a state and an action are given, then a model can predict the next state and reward.
- The model is used for planning, which means it provides a way to take a course of action by considering all future situations before actually experiencing those situations.
- The approaches for solving the RL problems **with the help of the model** are termed as the **model-based approach**. Comparatively, an approach **without using a model** is called a **model-free approach**.

Example of reinforcement learning

- Imagine a small robot navigating a grid to reach a goal while avoiding obstacles.
- **Scenario**
- **Grid Layout:**
 - The robot starts at the bottom left corner (0,0).
 - The goal is at the top right corner (3,3).
 - There are obstacles in some cells (e.g., (1,1) and (2,1)).
- **Actions:** The robot can move up, down, left, or right.
- **Rewards:**
 - +1 for reaching the goal (3,3).
 - -1 for hitting an obstacle.
 - 0 for all other moves.

Scenario

1. Grid Layout:

- The robot starts at position (0,0).
- The goal is at (3,3).
- Obstacles are at (1,1) and (2,1).

SCSS

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	X	(1,2)	(1,3)
(2,0)	X	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	G

- G: Goal
- X: Obstacle

- **Actions and Rewards**
- **Actions:** Move up, down, left, right.
- **Rewards:**
 - +1 for reaching the goal (3,3).
 - -1 for hitting an obstacle.
 - 0 for all other moves.

Solution is $(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (1,2) \rightarrow (2,2) \rightarrow (3,2) \rightarrow (3,3)$

The Return in reinforcement learning

- In reinforcement learning (RL), "The Return" refers to the cumulative reward that an agent receives when it interacts with an environment over a sequence of actions and states. It is a fundamental concept in RL and is often denoted as "G" or "R."
- The Return is a way to **measure the effectiveness of a policy**, which is the strategy or set of actions that an RL agent follows to maximize its expected cumulative reward. The objective of an RL agent is typically **to find the policy** that maximizes its expected return over time.

Calculation of Return

The return can be calculated in several ways, but the most common formula is:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$$

Where:

- R_t : Reward received at time t .
- γ : Discount factor ($0 \leq \gamma < 1$) that determines the importance of future rewards. A value closer to 1 makes future rewards more significant, while a value closer to 0 emphasizes immediate rewards.

Discount Factor In Reinforcement Learning

- The discount factor, commonly denoted as γ (gamma), is a parameter that influences how much weight is given to future rewards in the decision-making process of an RL agent.
- It's a critical component used to calculate the discounted future rewards or returns in RL algorithms.
- The discount factor γ determines the trade-off between immediate rewards and future rewards.
- A higher γ places more emphasis on obtaining rewards in the future, making the agent more forward-looking and potentially more patient in its decision-making.
- Conversely, a lower γ makes the agent focus more on immediate rewards, making it more myopic in its decisions.
- A discount factor close to 1 implies that the agent values future rewards highly, potentially sacrificing short-term gains for larger long-term rewards.
- A discount factor close to 0 implies that the agent values immediate rewards more and tends to make decisions that maximize short-term gains.

Given:

- $R_0 = 1$
- $R_1 = 0.5$
- $R_2 = 0.2$
- $\gamma = 0.9$

Return at $t = 0$:

$$G_0 = R_0 + \gamma R_1 + \gamma^2 R_2$$

Compute each term:

1. $R_0 = 1$
2. $\gamma R_1 = 0.9 \times 0.5 = 0.45$
3. $\gamma^2 R_2 = (0.9)^2 \times 0.2 = 0.81 \times 0.2 = 0.162$

Now add:

$$G_0 = 1 + 0.45 + 0.162 = 1.612$$

Example 1: Simple Environment

Setup:

- An agent interacts with a simple environment consisting of 3 states: S_0 , S_1 , and S_2 .
- The goal state is S_2 .
- Rewards are as follows:
 - $R(S_0 \rightarrow S_1) = 1$
 - $R(S_1 \rightarrow S_2) = 2$
 - All other transitions yield a reward of 0.

Discount Factor: $\gamma = 0.9$

Episode Simulation

1. **Starting at S_0 :**
 - The agent moves to S_1 and receives a reward of 1.
2. **From S_1 :**
 - The agent moves to S_2 and receives a reward of 2.

Calculate Return G_0

- The rewards received:
 - $R_0 = 1$ (from S_0 to S_1)
 - $R_1 = 2$ (from S_1 to S_2)

Now, let's calculate the return G_0 :

$$G_0 = R_0 + \gamma R_1$$

Substituting the values:

$$G_0 = 1 + 0.9 \times 2 = 1 + 1.8 = 2.8$$

Example 2: Longer Episode with Multiple Rewards

Setup:

- States: S_0 , S_1 , S_2 , and S_3 .
- Rewards:
 - $R(S_0 \rightarrow S_1) = 1$
 - $R(S_1 \rightarrow S_2) = 1$
 - $R(S_2 \rightarrow S_3) = 5$
- Discount Factor: $\gamma = 0.8$

Episode Simulation

1. **Starting at S_0 :**
 - The agent moves to S_1 and receives a reward of 1.
2. **From S_1 :**
 - The agent moves to S_2 and receives a reward of 1.
3. **From S_2 :**
 - The agent moves to S_3 and receives a reward of 5.

Calculate Return G_0

- The rewards received:
 - $R_0 = 1$ (from S_0 to S_1)
 - $R_1 = 1$ (from S_1 to S_2)
 - $R_2 = 5$ (from S_2 to S_3)

Now, let's calculate the return G_0 :

$$G_0 = R_0 + \gamma R_1 + \gamma^2 R_2$$

Substituting the values:

$$G_0 = 1 + 0.8 \times 1 + 0.8^2 \times 5$$

$$G_0 = 1 + 0.8 + 0.64 \times 5$$

$$G_0 = 1 + 0.8 + 3.2 = 5.0$$

Example 3: Complex Environment with Negative Rewards

Setup:

- States: S_0 , S_1 , S_2 , and S_3 .
- Rewards:
 - $R(S_0 \rightarrow S_1) = 1$
 - $R(S_1 \rightarrow S_2) = -1$ (penalty)
 - $R(S_2 \rightarrow S_3) = 3$
- Discount Factor: $\gamma = 0.9$

Episode Simulation

1. Starting at S_0 :

- The agent moves to S_1 and receives a reward of 1.

2. From S_1 :

- The agent moves to S_2 and receives a reward of -1 .

3. From S_2 :

- The agent moves to S_3 and receives a reward of 3.

Calculate Return G_0

- The rewards received:
 - $R_0 = 1$ (from S_0 to S_1)
 - $R_1 = -1$ (from S_1 to S_2)
 - $R_2 = 3$ (from S_2 to S_3)

Now, let's calculate the return G_0 :

$$G_0 = R_0 + \gamma R_1 + \gamma^2 R_2$$

Substituting the values:

$$G_0 = 1 + 0.9 \times (-1) + 0.9^2 \times 3$$

$$G_0 = 1 - 0.9 + 0.81 \times 3$$

$$G_0 = 1 - 0.9 + 2.43 = 2.53$$

UNIT- 4-PARTB

Reinforcement Learning

State-action value function

- The **state-action value function**, often denoted as $Q(s,a)$, is a fundamental concept in reinforcement learning. It measures the expected return (total future rewards) an agent can achieve by taking a specific action a in a given state s and then following a certain policy thereafter.

The **state-action value function**, often denoted as $Q(s, a)$, is a fundamental concept in reinforcement learning. It measures the expected return (total future rewards) an agent can achieve by taking a specific action a in a given state s and then following a certain policy thereafter.

Definition

The state-action value function $Q(s, a)$ is defined mathematically as:

$$Q(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

Where:

- $\mathbb{E}$: Represents the expected value.
- G_t : The return starting from time t .
- S_t : The state at time t .
- A_t : The action taken at time t .

Components

1. **State s :** The current situation or configuration the agent is in.
2. **Action a :** The decision the agent takes when in state s .
3. **Expected Return G_t :** The total reward the agent can expect to accumulate in the future after taking action a in state s .

Deterministic Policies:

- In a deterministic policy, the same control is always output for the same input state.
- Usually the π symbol denotes a policy function, which takes as input a state from the state space and always output the same control from the control space.
- In general, the goal of RL is to learn the optimal controls for all possible states so that sum of all agent rewards is maximized.
- Sometimes, a deterministic policy doesn't cut it for designers or engineers either due to competition among agents, partially observed system state, or other sources of noise that warrant stochastic policies.
- In such challenging circumstances, a stochastic policy is more appropriate.

Mathematically, a deterministic policy can be represented as:

$$\pi(s) = a$$

Where:

- π is the policy.
- s is the current state of the environment.
- a is the action to be taken in state s .

Key characteristics and considerations related to deterministic policies in RL:

- 1.Deterministic Mapping:** Unlike stochastic policies that assign probabilities to actions in each state, deterministic policies directly specify the action to take. This means that for a given state, the agent always chooses the same action when following a deterministic policy.
- 2.Exploitation:** Deterministic policies are well-suited for exploitation, where the agent selects the action it believes will yield the highest expected reward based on its current knowledge.
- 3.Deterministic Environments:** Deterministic policies are often employed in environments where actions have predictable outcomes. In such environments, the best action can be consistently determined for each state.

4. Continuous Action Spaces: Deterministic policies are commonly used in RL problems with continuous action spaces, where specifying a single action is more practical than defining a probability distribution over a continuous range of actions.

5. Deterministic Policy Gradient: Some RL algorithms, like Deterministic Policy Gradient (DPG) or Deep Deterministic Policy Gradient (DDPG), specifically target deterministic policies. These algorithms aim to learn deterministic policies for continuous action spaces, often using neural networks.

6. Exploration Challenges: A drawback of deterministic policies is that they may struggle with exploration in uncertain environments. Since the same action is chosen for the same state, they do not naturally account for variability or risk. This can limit the agent's ability to discover better actions.

7. Local Optima: Deterministic policies can get stuck in local optima, especially in high-dimensional or complex environments. In contrast, stochastic policies provide more flexibility and randomness to escape local optima.

8. Policy Optimization: Methods for optimizing deterministic policies involve gradient-based techniques that seek to maximize the expected cumulative reward by adjusting the policy parameters. This typically requires differentiating through the policy to compute gradients.

9. Continuous Control: Deterministic policies are particularly useful in RL tasks involving continuous control, such as robotic control or autonomous driving, where precise and deterministic actions are often required.

Stochastic Policies:

- On the other hand, a stochastic policy is only a little bit more complicated.
- A stochastic policy says that for any given state, each control is selected with some possibly non-zero probability.
- One can think that for stochastic policies the agent conditions on the encountered state and selects controls according to a probability distribution. Because a stochastic policy is a distribution it must obey some rules.
- Since we are dealing with the realm of finite spaces in this article the distribution is a discrete probability mass function.
- The first rule is a sum rule, which says that for each possible state, the sum over all possible controls, the stochastic policy must sum to 1. These are the main differences between deterministic and stochastic policies.

Stochastic policies are a type of policy in reinforcement learning (RL) where, for a given state, the policy prescribes a probability distribution over the available actions. In other words, instead of selecting a single, deterministic action for each state, stochastic policies assign probabilities to different actions, allowing the agent to make decisions that incorporate randomness. Mathematically, a stochastic policy can be represented as:

$$\pi(a|s)$$

Where:

- π is the policy.
- a is an action.
- s is the current state of the environment.
- $\pi(a|s)$ represents the probability of taking action a in state s .

key characteristics and considerations related to stochastic policies in RL:

- 1.Exploration:** Stochastic policies naturally facilitate exploration because they allow the agent to choose from a range of actions with associated probabilities. This randomness helps the agent discover new actions and learn about the environment.
- 2.Uncertainty Handling:** Stochastic policies are well-suited for environments with uncertainty or noise, where the outcomes of actions may vary. They allow the agent to account for this uncertainty by assigning different probabilities to different actions.
- 3.Balancing Exploration and Exploitation:** Stochastic policies offer a balance between exploration and exploitation. The agent can explore by occasionally selecting actions with lower probabilities, while also exploiting actions with higher probabilities based on its current knowledge.

4. Continuous and Discrete Action Spaces: Stochastic policies can be used in RL problems with both continuous and discrete action spaces. In continuous action spaces, the policy typically defines a probability distribution over the entire action space.

5. Policy Gradients: Many RL algorithms that work with stochastic policies use policy gradient methods. These methods update the policy parameters in the direction that maximizes the expected cumulative reward. The gradients are typically computed using the probability distribution defined by the policy.

6. Sample-Based Execution: To execute actions according to a stochastic policy, the agent samples from the action distribution specified by the policy. This sampling process introduces randomness into the agent's actions.

7. Policy Improvement: Stochastic policies can be iteratively improved by adjusting the parameters of the policy to maximize expected rewards. Methods like REINFORCE and Proximal Policy Optimization (PPO) are commonly used for this purpose.

8. Risk-Awareness: In certain scenarios, stochastic policies can be used to express risk preferences. By assigning higher probabilities to actions with lower risk, the agent can make decisions that balance reward maximization with risk avoidance.

9. Policy Entropy: Stochastic policies have an associated entropy, which quantifies the amount of uncertainty or randomness in the policy's actions. Some algorithms use entropy regularization to encourage more exploration or to balance exploration and exploitation.

Markov Decision Process (MDP) in Reinforcement Learning

A **Markov Decision Process (MDP)** provides a **mathematical framework** for modeling decision-making in environments where outcomes are partly random and partly under the control of an agent. MDP is the foundation of **Reinforcement Learning (RL)**.

An MDP is defined by the tuple:
 (S, A, P, R, γ)

Markov Decision Process (MDP) in Reinforcement Learning

1. State (S)

Represents the **current situation** the agent is in.

Example: In a grid world, each cell is a state.

2. Action (A)

The set of all possible actions the agent can take.

3. Transition Probability (P)

Defines how likely the agent moves from one state to another after taking an action.

4. Reward (R)

Numerical feedback for taking an action in a state.

5. Discount Factor (γ)

Controls how much future rewards matter.

Markov Decision Process (MDP) in Reinforcement Learning

Markov Property

A process is **Markovian** if:

Future depends only on the current state, NOT on past states.

Example:

If the agent is in **state S3**, the next state depends only on **S3**, not on the path taken to reach S3.

MONTE CARLO LEARNING in Reinforcement Learning

Monte Carlo (MC) methods learn value functions **from complete episodes**.

MC waits **until the episode ends**.

Then computes the **actual return**:

$$G_t = R_{t+1} + R_{t+2} + \dots + R_T$$

Then updates value estimates using:

$$V(s) \leftarrow V(s) + \alpha(G_t - V(s))$$

Features

Requires **episodic tasks** (must reach a terminal state).

No model of environment needed.

Estimates are based on **average of actual returns**.

MONTE CARLO LEARNING in Reinforcement Learning

★ Example (Simple Monte Carlo)

Episode:

State visited: $S \rightarrow A \rightarrow B \rightarrow \text{Terminal}$

Rewards: 0, 1, 5

Return from state A:

$$G_A = 1 + 5 = 6$$

If current $V(A) = 4$ and $\alpha = 0.5$:

$$V(A)_{new} = 4 + 0.5(6 - 4) = 5$$

TEMPORAL DIFFERENCE (TD) LEARNING in Reinforcement Learning

TD learning updates values **after every step**, not at the end.

★ Key Idea

TD uses **bootstrapping**:

$$V(s_t) \leftarrow V(s_t) + \alpha[R_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

The term:

$$\delta_t = R_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

is called **TD error**.

TEMPORAL DIFFERENCE (TD) LEARNING in Reinforcement Learning

★ Example (Simple TD(0) Update)**

Given:

Current state value: $V(s_t) = 5$

Next state value: $V(s_{t+1}) = 8$

Reward: $R = 2$

$\gamma = 0.9, \alpha = 0.5$

Compute TD target:

$$R + \gamma V(s_{t+1}) = 2 + 0.9 \times 8 = 9.2$$

Update:

$$V_{new} = 5 + 0.5(9.2 - 5) = 7.1$$

★ Monte Carlo vs Temporal Difference

Feature	Monte Carlo	TD Learning
Update time	End of episode	After every step
Needs episode end?	Yes	No
Uses bootstrapping?	No	Yes
Variance	High	Lower
Bias	Zero	Some bias
Works for non-episodic tasks	✗ No	✓ Yes
Speed	Slower	Faster

State-action value function definition

- A state-action value function is also called the Q function. It specifies how good it is for an agent to perform a particular action in a state with a policy π . The Q function is denoted by $Q(s)$. It denotes the value of taking an action in a state following a policy π .
- We can define Q function as follows:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} [R_t | s_t = s, a_t = a]$$

This specifies the expected return starting from state s with the action a according to policy π . We can substitute the value of R_t in the Q function from (2) as follows:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a \right]$$

State-action value function example:

- Suppose you have a robot that needs to navigate a grid-world environment. In this grid world, the robot can be in different states, and it can take various actions. The goal of the robot is to maximize its cumulative reward over time.
- Here's an example of a state-action value function $Q(s, a)$ for a specific state and action:
- Let's say the robot is currently in a state 'S', which corresponds to a particular cell in the grid-world, and it has the option to take one of three possible actions: move up, move down, or move right.
- State 'S': Robot's current location in the grid-world.
- Actions:
 - Action 'A1': Move up
 - Action 'A2': Move down
 - Action 'A3': Move right

The state-action value function $Q(S, A)$ might look like this:

- $Q(S, A1) = 5$: This means that if the robot is in state 'S' and it chooses to move up ('A1'), it expects to receive a cumulative reward of 5 in the long run.
- $Q(S, A2) = 2$: If the robot decides to move down ('A2') in state 'S', it expects to receive a cumulative reward of 2 over time.
- $Q(S, A3) = 4$: Choosing to move right ('A3') in state 'S' is expected to result in a cumulative reward of 4.
- The values in the state-action value function represent the expected cumulative rewards associated with taking each action in the given state. The robot would use this information to make decisions and select the action that maximizes the expected cumulative reward in its current state.

Bellman Equation

- It describes the relationship between the value of a state or state-action pair and the expected cumulative rewards that can be obtained from that state or state-action pair.
- There are two main forms of the Bellman equation: the Bellman Expectation Equation for state values and the Bellman Expectation Equation for action values.

Bellman Equation

According to the **Bellman Equation**, long-term- reward in a **given action is equal to the reward from the current action combined with the expected reward from the future actions** taken at the following time. Let's try to understand first.

How does Reinforcement Learning Work?

To understand the working process of the RL, we need to consider two main things:

- **Environment:** It can be anything such as a room, maze, football ground, etc.
- **Agent:** An intelligent agent such as AI robot.

Let's take an example of a maze environment that the agent needs to explore.

Consider the below image:

s1	s2	s3	 R=+1
s5	s6	s7	 R=-1
s9 	s10	s11	s12

Lorem ipsum

- In the above image, the agent is at the very first block of the maze. The maze is consisting of an S_6 block, which is a **wall**, S_8 a **fire pit**, and S_4 a **diamond block**.
- The agent cannot cross the S_6 block, as it is a solid wall. If the agent reaches the S_4 block, then get the **+1 reward**; if it reaches the fire pit, then gets **-1 reward point**. It can take four actions: **move up, move down, move left, and move right**.
- The agent can take any path to reach to the final point, but he needs to make it in possible fewer steps. Suppose the agent considers the path **S9-S5-S1-S2-S3**, so he will get the +1-reward point.

The agent will try to remember the preceding steps that it has taken to reach the final step. To memorize the steps, it assigns 1 value to each previous step. Consider the below step:

V=1 s1	V=1 s2	V=1 s3	 s4
V=1 s5	s6	s7	 s8
 V=1 s9	s10	s11	s12

Now, the agent has successfully stored the previous steps assigning the 1 value to each previous block. But what will the agent do if he starts moving from the block, which has 1 value block on both sides? Consider the below diagram:



It will be a difficult condition for the agent whether he should go up or down as each block has the same value. So, the above approach is not suitable for the agent to reach the destination. Hence to solve the problem, we will use the **Bellman equation**, which is the main concept behind reinforcement learning.

The Bellman equation was introduced by the Mathematician **Richard Ernest Bellman in the year 1953**, and hence it is called as a Bellman equation. It is associated with dynamic programming and used to calculate the values of a decision **problem at a certain point by including the values of previous states.**

- It is a way of calculating the value functions in dynamic programming or environment that leads to modern reinforcement learning.
- The key-elements used in Bellman equations are:
- Action performed by the agent is referred to as "a"
- State occurred by performing the action is "s."
- The reward/feedback obtained for each good and bad action is "R."
- A discount factor is Gamma " γ ."

The Bellman equation can be written as:

$$V(s) = \max [R(s,a) + \gamma V(s')]$$

Where,

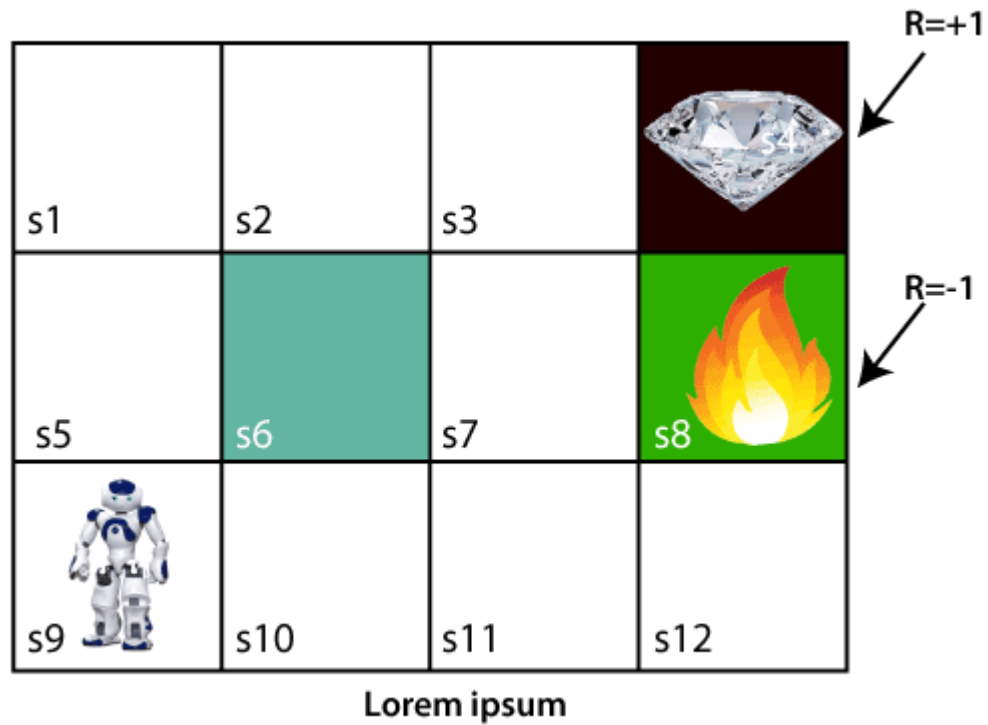
$V(s)$ = value calculated at a particular point.

$R(s,a)$ = Reward at a particular state s by performing an action.

γ = Discount factor

$V(s')$ = The value at the previous state.

In the above equation, we are taking the max of the complete values because the agent tries to find the optimal solution always.



For s3 block:

$V(s3) = \max [R(s,a) + \gamma V(s')]$, here $V(s')= 0$ because there is no further state to move.

$V(s3)= \max[R(s,a)] \Rightarrow V(s3)= \max[1] \Rightarrow V(s3)= 1.$

$R(s,a)=1$ (Reward of +1, which moves towards right), $\gamma=0.9$

For s2 block:

$V(s_2) = \max [R(s,a) + \gamma V(s')]$, here $\gamma = 0.9$ (lets), $V(s') = 1$ (Previous State Value), and $R(s, a) = 0$, because there is no reward at this state.

$$V(s_2) = \max[0.9(1)] \Rightarrow V(s_2) = \max[0.9] \Rightarrow \mathbf{V(s_2) = 0.9}$$

For s1 block:

$V(s_1) = \max [R(s,a) + \gamma V(s')]$, here $\gamma = 0.9$ (lets), $V(s') = 0.9$, and $R(s, a) = 0$, because there is no reward at this state also.

$$V(s_1) = \max[0.9(0.9)] \Rightarrow V(s_1) = \max[0.81] \Rightarrow \mathbf{V(s_1) = 0.81}$$

For s5 block:

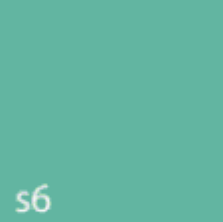
$V(s_5) = \max [R(s,a) + \gamma V(s')]$, here $\gamma = 0.9$ (lets), $V(s') = 0.81$, and $R(s, a) = 0$, because there is no reward at this state also.

$$V(s_5) = \max[0.9(0.81)] \Rightarrow V(s_5) = \max[0.73] \Rightarrow \mathbf{V(s_5) = 0.73}$$

For s9 block:

$V(s_9) = \max [R(s,a) + \gamma V(s')]$, here $\gamma = 0.9$ (lets), $V(s') = 0.73$, and $R(s, a) = 0$, because there is no reward at this state also.

$$V(s_9) = \max[0.9(0.73)] \Rightarrow V(s_9) = \max[0.657] \Rightarrow \mathbf{V(s_4) = 0.66}$$

$V=0.81$ s1	$V=0.9$ s2	$V=1$ s3	 s4
$V=0.73$ s5	 s6	s7	 s8
 $V=0.66$ s9	s10	s11	s12

Now, we will move further to the 6th block, and here agent may change the route because it always tries to find the optimal path. So now, let's consider from the block next to the fire pit.

V=0.81 s1	V=0.9 s2	V=1 s3	 s4
V=0.73 s5	 s6	 s7	 s8
V=0.66 s9	s10	s11	s12

Now, the agent has three options to move; if he moves to the blue box, then he will feel a bump if he moves to the fire pit, then he will get the -1 reward. But here we are taking only positive rewards, so for this, he will move to upwards only. The complete block values will be calculated using this formula. Consider the below image:

$V=0.81$ s1	$V=0.9$ s2	$V=1$ s3	 s4
$V=0.73$ s5	s6	$V=0.9$ s7	 s8
$V=0.66$ s9	$V=0.73$ s10	$V=0.81$ s11	$V=0.73$ s12

Thank you!

Q Learning

Q- Learning

- The key idea behind Q-learning is for an agent to learn a policy, which is a strategy that guides its actions in different states of the environment.
- Q-learning is typically applied to a specific problem where an agent interacts with an environment. The environment consists of different states and allows the agent to take certain actions in those states.
- When the agent performs an action in a particular state, it receives a numerical reward (r) from the environment. The reward provides immediate feedback to the agent about the goodness of its action.
- The Q-value $Q(s,a)$ of a state-action pair is an estimate of the expected cumulative reward the agent can obtain, starting from state s , taking action a , and then acting optimally thereafter. Initially, these Q-values are usually set arbitrarily.
- The Q-table is a data structure used to store and update the Q-values for each state-action pair. It starts with initial values and is updated through learning.
- The Q-learning algorithm iteratively updates the Q-values based on the rewards obtained and the transitions between states. The updates are typically done using the Q-learning update rule mentioned in the previous response.

- The agent faces a trade-off between exploration (trying new actions to discover their effects) and exploitation (choosing actions based on current knowledge to maximize rewards). Properly balancing this trade-off is crucial for effective learning.
- The learning rate (α) determines the extent to which new information replaces old information in the Q-table during updates.
- The discount factor (γ) indicates the importance of future rewards in the agent's decision-making.
- The Q-learning process continues until the Q-values converge to optimal values, ensuring that the agent has learned the best policy to maximize rewards.

Q Table

- A Q-table, short for **Quality table**, is a fundamental data structure used in Q-learning, a popular reinforcement learning technique.
- The Q-table helps an agent learn and make decisions in a discrete **state-action space**.
- It is a matrix where each row represents a state, each column represents a possible action, and each cell stores the corresponding Q-value.
- The Q-value $Q(s,a)$ in the Q-table for a specific state-action pair (s,a) represents the expected cumulative reward the agent can achieve by starting in state s , taking action a , and then following an optimal policy.
- Initially, the Q-table is usually initialized with **arbitrary or zero values**.

- Here's a simple example of a Q-table for a hypothetical problem with two states (State 1 and State 2) and three possible actions (Action 1, Action 2, Action 3):

- | State | Action 1 | Action 2 | Action 3 |
|-------|----------|----------|----------|
| 1 | 0.5 | 0.2 | 0.1 |
| 2 | 0.3 | 0.7 | 0.5 |

- The row "State 1" represents Q-values for each action in State 1.
- The row "State 2" represents Q-values for each action in State 2.
- Q-learning aims to find an optimal policy, which is a mapping of states to actions that maximizes the cumulative reward.
- The Q-table is a crucial tool in this process, enabling the agent to learn and apply the best actions for each state it encounters.

Q-Learning Algorithms

- **Initialize Q-table:** Create a Q-table with dimensions for each state-action pair and initialize the Q-values arbitrarily or to zero.
- **Set hyperparameters:**
- Learning rate (α): Determines how much the Q-values are updated in each iteration.
- Discount factor (γ): Represents the importance of future rewards in the learning process.
- Exploration probability (ϵ): Controls the balance between exploration and exploitation.
- **For each episode:**
- Start in an initial state s .
- Repeat until the episode terminates:
 - Choose an action a for the current state s using an exploration strategy (e.g., epsilon-greedy strategy).
 - Execute the action a , observe the reward r and the next state s' .
 - Update the Q-value for the current state-action pair using the Q-learning update rule:
 - $Q(s,a) \leftarrow (1-\alpha) \cdot Q(s,a) + \alpha \cdot (r + \gamma \cdot \max_{a'} Q(s',a'))$
 - Update the current state s to the next state s' .
- **Convergence:** Continue updating the Q-values over episodes until the Q-values converge or a predefined stopping criterion is met.

- The Q-learning update rule combines the current Q-value $Q(s,a)$ with the observed reward (r) and the maximum Q-value of the next state $\max_{a'} Q(s',a')$, scaled by the learning rate (α) and discount factor (γ).
- This encourages the agent to learn optimal Q-values that lead to higher cumulative rewards.
- The exploration strategy is crucial for the agent to explore the environment effectively.
- The epsilon-greedy strategy is often used, where with probability ϵ a random action is chosen (exploration), and with probability $1-\epsilon$ the action with the maximum Q-value for the current state is chosen (exploitation).

Numerical Examples

Let us set the value of learning parameter $\gamma = 0.8$ and initial state as room B.
the instant reward matrix **R** that represents the environment

$$\mathbf{R} = \begin{array}{c|cccccc} \text{state} \backslash \text{action} & A & B & C & D & E & F \\ \hline A & - & - & - & - & 0 & - \\ B & - & - & - & 0 & - & 100 \\ C & - & - & - & 0 & - & - \\ D & - & 0 & 0 & - & 0 & - \\ E & 0 & - & - & 0 & - & 100 \\ F & - & 0 & - & - & 0 & 100 \end{array}$$

First we set matrix **Q** as a zero matrix.

Look at the second row (state B) of matrix **R**. There are two possible actions for the current state B, that is to go to state D, or go to state F. By random selection, we select to go to F as our action.

Now we consider that suppose we are in state F. Look at the sixth row of reward matrix **R** (i.e. state F). It has 3 possible actions to go to state B, E or F.

$$\mathbf{Q}(\text{state}, \text{action}) = \mathbf{R}(\text{state}, \text{action}) + \gamma \cdot \text{Max}[\mathbf{Q}(\text{next state}, \text{all actions})]$$

$$\mathbf{Q}(B, F) = \mathbf{R}(B, F) + 0.8 \cdot \text{Max}\{\mathbf{Q}(F, B), \mathbf{Q}(F, E), \mathbf{Q}(F, F)\} = 100 + 0.8 \cdot 0 = 100$$

Since matrix Q that is still zero, $Q(F,B), Q(F,E), Q(F,F)$ are all zero. The result of computation $Q(B,F)$ is also 100 because of the instant reward.

The next state is F, now become the current state. Because F is the goal state, we finish one episode. Our agent's brain now contain updated matrix Q as

$$Q = \begin{matrix} & \begin{matrix} A & B & C & D & E & F \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \\ F \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

For the next episode, we start with initial random state. This time for instance we have state D as our initial state.

Look at the fourth row of matrix R ; it has 3 possible actions, that is to go to state B, C and E. By random selection, we select to go to state B as our action.

Now we imagine that we are in state B. Look at the second row of reward matrix R (i.e. state B). It has 2 possible actions to go to state D or state F. Then, we compute the Q value

$$Q(state, action) = R(state, action) + \gamma \cdot \text{Max}[Q(next\ state, all\ actions)]$$

$$Q(D, B) = R(D, B) + 0.8 \cdot \text{Max}\{Q(B, D), Q(B, F)\} = 0 + 0.8 \cdot \text{Max}\{0, 100\} = 80$$

We use the updated matrix $\mathbf{Q}$ from the last episode. $\mathbf{Q}(B,D) = 0$ and $\mathbf{Q}(B,F) = 100$. The result of computation $\mathbf{Q}(D,B) = 80$ because of the reward is zero. The $\mathbf{Q}$ matrix becomes

$$\mathbf{Q} = \begin{matrix} & \begin{matrix} A & B & C & D & E & F \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \\ F \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 80 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

The next state is B, now become the current state. We repeat the inner loop in Q learning algorithm because state B is not the goal state.

There are two possible actions from the current state B, that is to go to state D, or go to state F. By lucky draw, our action selected is state F.

Now we think of state F that has 3 possible actions to go to state B, E or F. We compute the Q value using the maximum value of these possible actions.

$$Q(state, action) = R(state, action) + \gamma \cdot \text{Max}[Q(next\ state, all\ actions)]$$

$$\begin{aligned} Q(B, F) &= R(B, F) + 0.8 \cdot \text{Max}\{Q(F, B), Q(F, E), Q(F, F)\} \\ &= 100 + 0.8 \cdot \text{Max}\{0, 0, 0\} = 100 \end{aligned}$$

The entries of updated Q matrix contain $Q(F, B), Q(F, E), Q(F, F)$ are all zero. The result of computation $Q(B, F)$ is also 100 because of the instant reward. This result does not change the Q matrix.

Because F is the goal state, we finish this episode. Our agent's brain now contain updated matrix **Q** as

$$\mathbf{Q} = \begin{matrix} & \begin{matrix} A & B & C & D & E & F \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \\ F \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 80 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

If our agent learns more and more experience through many episodes, it will finally reach convergence values of Q matrix as

$$\mathbf{Q} = \begin{matrix} & \begin{matrix} state \backslash action & A & B & C & D & E & F \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \\ F \end{matrix} & \begin{bmatrix} - & - & - & - & 400 & - \\ - & - & - & 320 & - & 500 \\ - & - & - & 320 & - & - \\ - & 400 & 256 & - & 400 & - \\ 320 & - & - & 320 & - & 500 \\ - & 400 & - & - & 400 & 500 \end{bmatrix} \end{matrix}$$

This **Q** matrix, then can be normalized into a percentage by dividing all valid entries with the highest number (divided by 500 in this case) becomes

$$\hat{\mathbf{Q}} = \begin{array}{c|cccccc} \text{state} \backslash \text{action} & A & B & C & D & E & F \\ \hline A & - & - & - & - & 80 & - \\ B & - & - & - & 64 & - & 100 \\ C & - & - & - & 64 & - & - \\ D & - & 80 & 51 & - & 80 & - \\ E & 64 & - & - & 64 & - & 100 \\ F & - & 80 & - & - & 80 & 100 \end{array}$$

Once the **Q** matrix reaches almost the convergence value, our agent can reach the goal in an optimum way. To trace the sequence of states, it can easily compute by finding action that makes maximum **Q** for this state.

For example from initial State C, it can use the **Q** matrix as follow:

From State C the maximum **Q** produces action to go to state D

From State D the maximum **Q** has two alternatives to go to state B or E. Suppose we choose arbitrary to go to B

From State B the maximum value produces action to go to state F

Thus the sequence is C -> D -> B -> F

Limitations of Q-Learning

- **High-Dimensional State Spaces:**
- Q-learning struggles to handle high-dimensional state spaces effectively, such as those encountered in computer vision or natural language processing tasks. The state-action space can become excessively large, making it challenging to store and update Q-values efficiently.
- **Curse of Dimensionality:**
- Q-learning's reliance on a table to store Q-values becomes impractical as the state space grows exponentially with the number of dimensions. This curse of dimensionality leads to a combinatorial explosion in the memory and computational requirements, rendering the approach unfeasible for many real-world problems.
- **Continuous Action Spaces:**
- When dealing with continuous action spaces, discretizing the actions to fit the Q-table becomes problematic. The Q-table would require an enormous number of entries, making it impractical and computationally inefficient.
- **Exploration-Exploitation Tradeoff:**
- Balancing exploration (to discover new actions) and exploitation (to choose the best-known actions) is a challenge in Q-learning. Traditional ϵ -greedy strategies for exploration can be inefficient, leading to suboptimal policies.

- **Convergence and Learning Rate Sensitivity:**
- The convergence of Q-learning is not guaranteed, especially in non-stationary environments. The choice of learning rate is crucial, and an inappropriate learning rate can lead to instability or slow convergence.
- **Delayed Rewards and Non-Markovian Rewards:**
- Q-learning assumes that the current state fully encapsulates all information needed to make decisions. However, in some environments, rewards may be delayed or non-Markovian, violating this assumption and making it difficult for Q-learning to learn an optimal policy.
- **Overestimation of Q-values:**
- Q-learning, especially in conjunction with function approximation like neural networks, can lead to overestimation of Q-values, which affects the quality of the learned policy. Techniques like Double Q-learning have been introduced to address this issue.
- **Sample Inefficiency:**
- Q-learning often requires a large number of samples to converge to an optimal policy. This sample inefficiency is a significant drawback, especially in domains where obtaining real-world samples is costly or time-consuming.

- **Inability to Handle Stochastic Environments Effectively:**
- In stochastic environments, where actions do not deterministically lead to specific states or rewards, Q-learning's deterministic nature can lead to suboptimal policies.
- **Knowledge of Environment Dynamics:**
- Q-learning typically assumes knowledge of the environment's dynamics, including transition probabilities and rewards. In practice, these dynamics may be unknown or difficult to model accurately.

Algorithm Refinement- Improved Neural Network Architecture

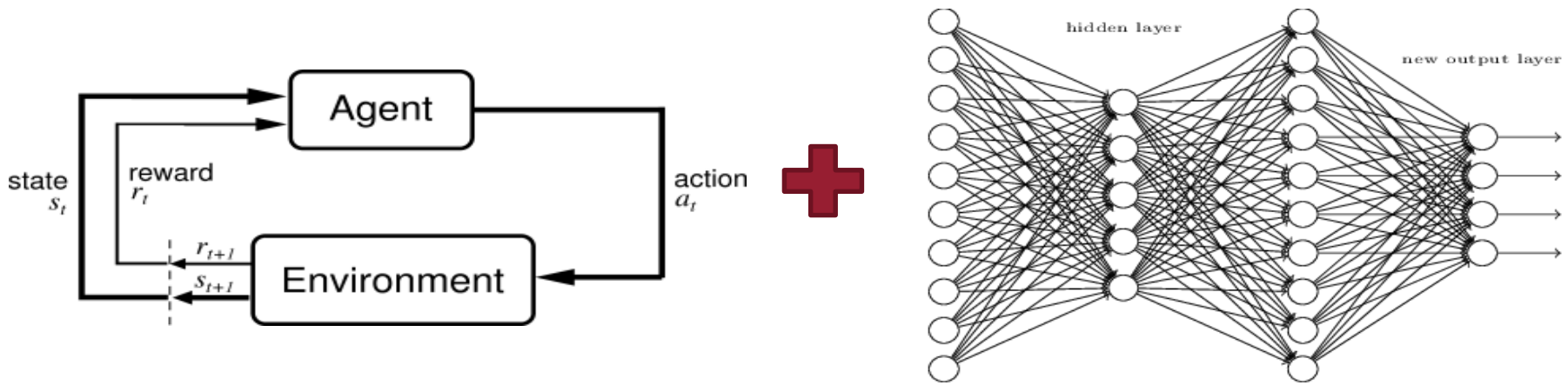
- In Q-learning, using an improved neural network architecture can enhance the algorithm's performance and learning efficiency.
- A well-designed neural network can help approximate the Q-value function accurately, leading to more effective and faster convergence.
- Below are steps to incorporate an improved neural network architecture into the Q-learning algorithm:
- **Deep Q-Network (DQN) Architecture:**
- Utilize a deep neural network, often with convolutional layers, to approximate the Q-value function. The architecture should be designed to process state representations efficiently, especially for image-based input.
- **Convolutional Layers:**
- Incorporate convolutional layers to capture spatial patterns and features from the input states effectively, especially when dealing with image-based inputs. Convolutional layers can significantly improve the representation of visual information.

- **Dueling Network Architecture:**
- Implement the dueling network architecture, which separates the value function and advantage function. This architecture allows for more stable and efficient learning by independently estimating the value of each state and the advantage of each action.
- **Advanced Activation Functions:**
- Experiment with advanced activation functions, such as Rectified Linear Unit (ReLU) variants, Leaky ReLU, Parametric ReLU (PReLU), or Exponential Linear Unit (ELU). These activations can help with faster convergence and mitigate vanishing gradient problems.
- **Batch Normalization:**
- Incorporate batch normalization layers to stabilize and accelerate the training process by normalizing the activations within each mini-batch. This helps improve convergence and generalization.
- **Dropout or Regularization:**
- Apply dropout or other regularization techniques to prevent overfitting and encourage the network to learn more robust features. Dropout can be applied to hidden layers to randomly drop out a fraction of neurons during training.

- **Residual Connections:**
- Implement residual connections to facilitate the training of very deep networks, ensuring effective gradient flow and easing the optimization process.
- **Use of Memory Buffers:**
- Integrate experience replay and target network updates, as commonly seen in DQN. These techniques help stabilize learning and make the neural network training more efficient.
- **Learning Rate Scheduling:**
- Employ learning rate schedules, such as exponential decay or step decay, to adjust the learning rate during training. This can help in better convergence and prevent overshooting.

Deep Q Networks (DQN)

- Deep Q-Networks (DQNs) are a class of deep neural network architectures used in reinforcement learning (RL), specifically in the context of Q-learning.
- DQNs combine Q-learning, which is a value-based RL method, with deep neural networks to handle high-dimensional state spaces and complex tasks.
- **Neural Network Architecture:**
- DQNs utilize a deep neural network, often with convolutional layers, to approximate the Q-value function. This allows them to handle high-dimensional inputs, such as images, effectively.



- **Target Network:**
- DQNs use two separate networks: the **Q-network (online network)** and a **target network**. The target network is a delayed copy of the Q-network, and its parameters are periodically updated to stabilize training and improve convergence.
- **Loss Function:**
- The loss function used in DQNs is often the **mean squared error between the predicted Q-values and the target Q-values**. The target Q-values are computed using the Bellman equation and the target network.
- **ϵ -Greedy Exploration:**
- DQNs typically use an ϵ -greedy strategy for exploration, where with probability ϵ , a random action is chosen, and with probability $1-\epsilon$, the action with the highest Q-value is selected.
- **Target Q-Value Computation:**
- The target Q-value for a state-action pair is calculated as the immediate reward plus the discounted maximum Q-value for the next state:

$$Q_{\text{target}}(s,a) = R + \gamma \cdot \max_{a'} Q(s',a'),$$
- where R is the immediate reward, γ is the discount factor, ' s ' is the next state, and ' a ' is the action in the next state.

Algorithm Refinement – ϵ greedy policy

- The ϵ -greedy policy is a crucial component in Q-learning that balances exploration (trying new actions) and exploitation (choosing actions with the highest estimated value).
- Refining the ϵ -greedy policy can significantly impact the learning process and the efficiency of the Q-learning algorithm.
- Here are some refinements and strategies to improve the ϵ -greedy policy in Q-learning:
- **Decaying ϵ :**
- Start with a high exploration rate (ϵ) and gradually decrease it over time. This approach allows for more exploration in the early stages of learning and shifts towards more exploitation as the agent gains experience.
- **Annealing ϵ :**
- Instead of a predefined decay schedule, dynamically adjust ϵ based on a specific annealing function or schedule. Common annealing functions include linear annealing, exponential decay, or logarithmic decay.
- **Softmax Action Selection:**
- Use a softmax function to select actions based on the Q-values, rather than a deterministic greedy strategy. This provides a soft selection of actions, allowing for a smooth transition from exploration to exploitation.

- **Boltzmann Exploration:**
- Use a variant of softmax action selection, known as Boltzmann exploration. It introduces a temperature parameter that controls the level of exploration. Higher temperature values allow for more exploration, while lower values lead to more exploitation.
- **ϵ -First Strategy:**
 - Initially, use a high ϵ to encourage exploration for a certain number of episodes or steps. After this initial phase, switch to a lower ϵ to prioritize exploitation.
- **Varying ϵ for Different States or Actions:**
 - Instead of using a constant ϵ , vary ϵ based on the state or action being considered. For example, you could have different ϵ values for different regions of the state space or for different actions.
- **Adaptive ϵ Based on Uncertainty:**
 - Adjust ϵ based on the uncertainty or confidence in the Q-value estimates. Higher uncertainty might lead to more exploration to gather additional information.

- **Differential ϵ for Important States:**
 - Assign a higher ϵ for states or regions of the state space that are considered important or challenging, encouraging the agent to focus exploration in those areas.
- **Parameter Tuning for ϵ :**
 - Experiment with different ϵ values and decay schedules to find the optimal combination for your specific problem. Parameter tuning (e.g., using grid search or automated approaches) can help identify the most effective ϵ strategy.
- **Adaptive ϵ for Episodic Tasks:**
 - Modify ϵ at the beginning of each episode based on the episode number or any other relevant metric. This can allow for more exploration in early episodes.

Q learning refinement- mini batches and soft updates

- Refining Q-learning by incorporating mini-batches and soft updates can significantly enhance learning stability, efficiency, and convergence.
- **Mini-Batch Training:**
- Instead of updating the Q-value estimates using a single sample (as in standard Q-learning), mini-batch training involves updating the Q-network parameters based on a small random subset (mini-batch) of experiences sampled from the replay buffer. This approach improves stability and efficiency by reducing the variance in the updates and allowing for more effective weight adjustments.
- **Implementation Steps for Mini-Batch Training:**
- Sample a mini-batch of experiences (state, action, reward, next state) from the replay buffer.
- Compute the target Q-values for each experience in the mini-batch using the Bellman equation.
- Compute the loss between the predicted Q-values and the target Q-values for the mini-batch.
- Update the Q-network parameters by minimizing this loss using backpropagation.

- **Soft Target Network Updates:**
- In addition to mini-batch training, introducing a target network with soft parameter updates enhances Q-learning stability. Rather than copying the parameters directly from the Q-network to the target network periodically (as in standard Q-learning), soft updates involve blending the target network's parameters slowly over time.
- **Implementation Steps for Soft Target Network Updates:**
- Instead of copying the Q-network parameters to the target network, update the target network's parameters gradually using a fraction τ of the Q-network's parameters at each step.
- Mathematically, for each weight w in the target network and Q-network, perform:
- $w_{\text{target}} \leftarrow \tau \cdot w_{\text{q}} + (1 - \tau) \cdot w_{\text{target}}$, where τ is a small value (e.g., 0.001 to 0.01).

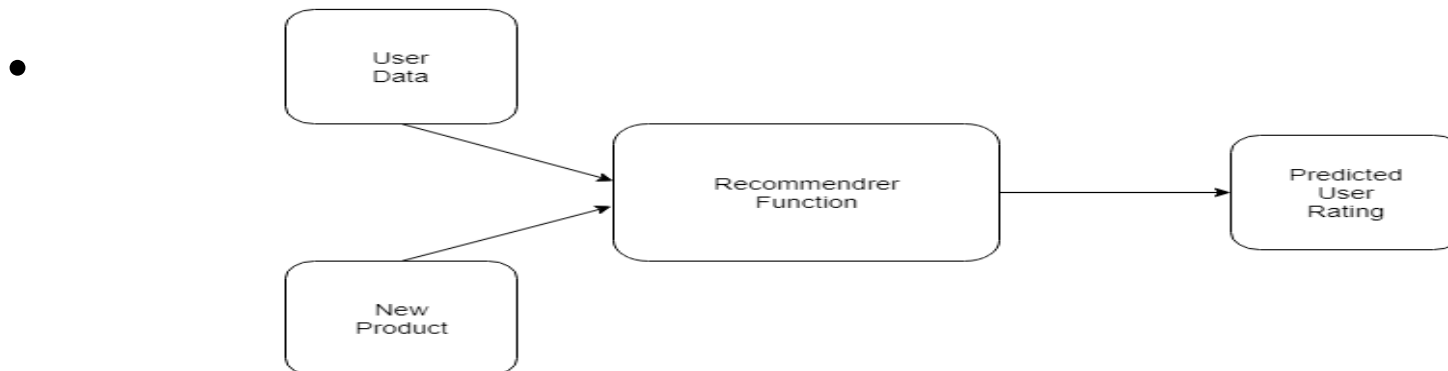
- **Benefits and Advantages:**
- Mini-batch training reduces the noise in the parameter updates and leads to more stable learning by providing a smoother update trajectory.
- Soft target network updates mitigate issues related to target value oscillations and divergence, leading to a more stable and efficient learning process.
- **Parameter Tuning:**
- Tune hyperparameters such as mini-batch size, learning rate, and soft update rate (τ) to achieve optimal performance and convergence speed.

Recommender System

Introduction

- Recommender systems are software tools and algorithms designed to suggest relevant items or content to users based on their preferences, behavior, and interactions with a platform.
- A recommendation system is a subset of machine learning that uses data to help users find products and content.
- Websites and streaming services use recommender systems to generate “for you” or “you might also like” pages and content.
- Recommender systems are an essential feature in our digital world, as users are often overwhelmed by choice and need help finding what they're looking for.
- This leads to happier customers and, of course, more sales.
- Recommender systems are like salesmen who know, based on your history and preferences, what you like.

- Recommender systems are so commonplace now that many of us use them without even knowing it.
- Because we can't possibly look through all the products or content on a website, a recommendation system plays an important role in helping us have a better user experience, while also exposing us to more inventory we might not discover otherwise.
- Some examples of recommender systems in action include product recommendations on Amazon, Netflix suggestions for movies and TV shows in your feed, recommended videos on YouTube, music on Spotify, the Facebook newsfeed and Google Ads.



- An important component of any of these systems is the recommender function, which takes information about the user and predicts the rating that user might assign to a product, for example.
- Predicting user ratings, even before the user has actually provided one, makes recommender systems a powerful tool.

How Do Recommender Systems Work?

- **UNDERSTANDING RELATIONSHIPS**

- Relationships provide recommender systems with tremendous insight, as well as an understanding of customers. There are three main types that occur:
- **User-Product Relationship**
- The user-product relationship occurs when some users have an affinity or preference towards specific products that they need. For example, a **cricket player** might have a preference for cricket-related items, thus the e-commerce website will build a user-product relation of player->cricket.
- **Product-Product Relationship**
- Product-product relationships occur when **items are similar** in nature, either by appearance or description. Some examples include books or music of the same genre, dishes from the same cuisine, or news articles from a particular event.
- **User-User Relationship**
- User-user relationships occur when some **customers have similar taste with respect to a particular product or service**. Examples include mutual friends, similar backgrounds, similar age, etc.

DATA & RECOMMENDER SYSTEMS

- **User Behavior Data**

- Users behavior data is useful information about the engagement of the user on the product. It can be collected from ratings, clicks and purchase history.

- **User Demographic Data**

- User demographic information is related to the user's personal information such as age, education, income and location.

- **Product Attribute Data**

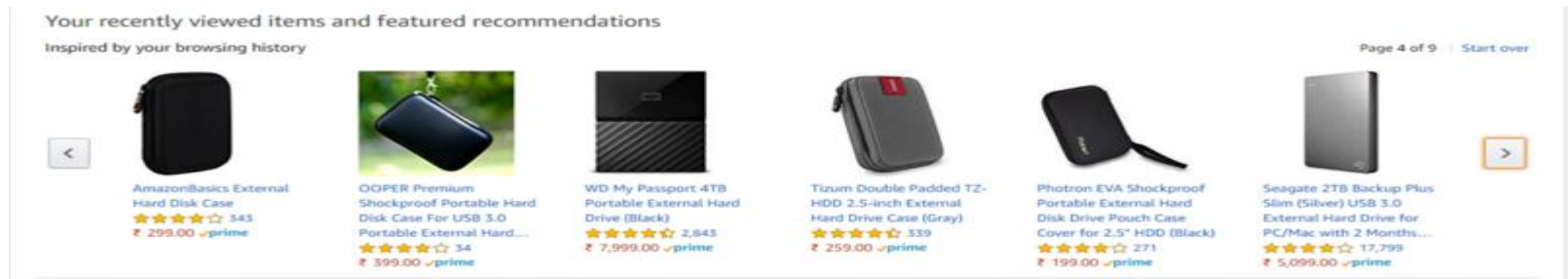
- Product attribute data is information related to the product itself such as genre in case of books, cast in case of movies, cuisine in case of food.

HOW DO WE PROVIDE DATA FOR RECOMMENDER SYSTEMS?

- Data can be provided in a variety of ways. There are two particularly important methods, explicit and implicit rating.
- **Explicit Ratings**
 - Explicit ratings are provided by the user.
 - They infer the user's preference.
 - Examples include star ratings, reviews, feedback, likes and following.
 - Since users don't always rate products, explicit ratings can be hard to get.
- **Implicit Ratings**
 - Implicit ratings are provided when users interact with the item.
 - They infer a user's behavior and are easy to get as users are subconsciously clicking.
 - Examples include clicks, views and purchases. (Note: Views and purchases can be a better entity to recommend as users will have spent time and money on what is most crucial for them.)

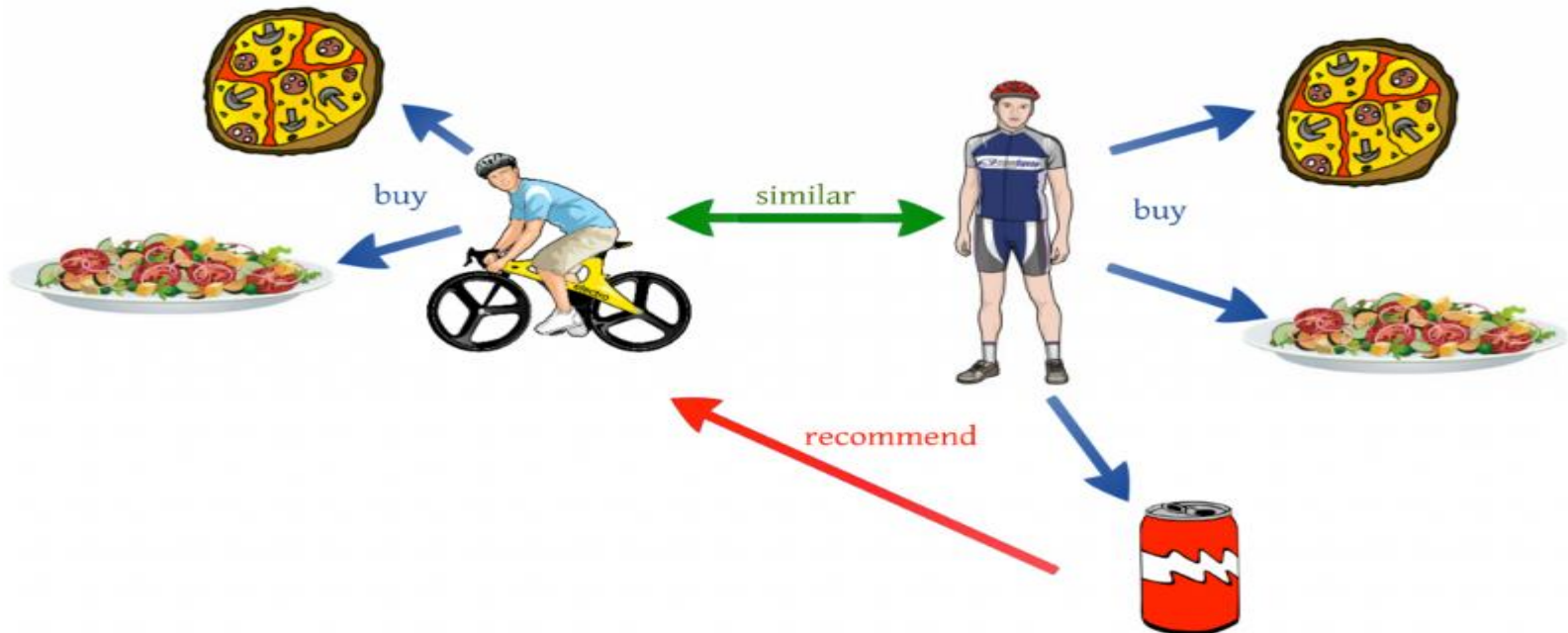
Product Similarity (Item-Item Filtering)

- Product similarity is the most useful system for suggesting products based on **how much the user would like the product.**
- If the user is browsing or searching for a particular product, they can be shown similar products.
- Users often expect to find products they want quickly and move on if they have a hard time finding the relevant product.
- When the user clicks on one product we can show another similar product, or if the user buys the product we can email the user advertisements or coupons based on a similar product.
- Product similarity is particularly useful when we don't know much about the user yet, but we do know what products they're viewing.



User Similarity (User-User Filtering)

- User similarity is for checking the difference between the similarity of two users.
- If two users have similar preferences for a product we can assume **they have similar interests.**
- It's like a friend recommending a product.



- One shortcoming of user similarity, however, is that it requires all the user data to suggest products.
- It's called a **cold start problem** because beginning the recommendation process requires previous data from users.
- A newly launched e-commerce website, for example, suffers from the cold start problem because it doesn't have a large number of users.
- Product similarity doesn't have this problem because it just requires product information and the user's preference.
- Netflix, for example, avoids this issue by asking users their likes when starting a new subscription.

Similarity Measures

- Similarity is measured using the distance metric.
- Nearest points are the most similar and farthest points are the least relevant.
- The similarity is subjective and is highly dependent on the domain and application.
- For example, two movies are similar because of genre or length or cast.
- Care should be taken when calculating distance across dimensions/features that are unrelated.
- The relative values of each element must be normalized, or one feature could end up dominating the distance calculation.

Minkowski Distance

- When the dimension of a data point is numeric, the general form is called the **Minkowski distance**.

$$d(x, y) = \left(\sum_i^n (|x_i - y_i|)^q \right)^{\frac{1}{q}}$$

- It is a generic distance metric where Manhattan($r=1$) or Euclidean($r=2$) distance measures are generalizations of it.

The diagram shows the Minkowski Distance formula in a hand-drawn style. The title "MINKOWSKI DISTANCE" is at the top in large, blocky letters. Below it, the formula is written as $d(p, q) = \left(\sum_{i=1}^n |p_i - q_i|^C \right)^{\frac{1}{C}}$. Annotations include: a red arrow pointing to p_i labeled "ith element of vector p", a blue arrow pointing to q_i labeled "ith element of vector q", and a green arrow pointing to the exponent C with text explaining its values. The signature "ChrisAlbon" is in the bottom left.

MINKOWSKI DISTANCE

$$d(p, q) = \left(\sum_{i=1}^n |p_i - q_i|^C \right)^{\frac{1}{C}}$$

When $C=1$
Minkowski is Manhattan distance.

When $C=2$
Minkowski is Euclidean distance.

ith element of vector p

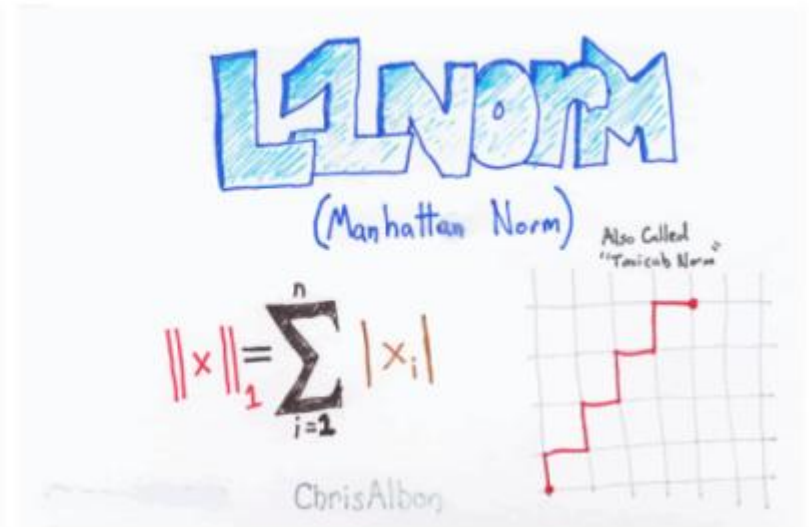
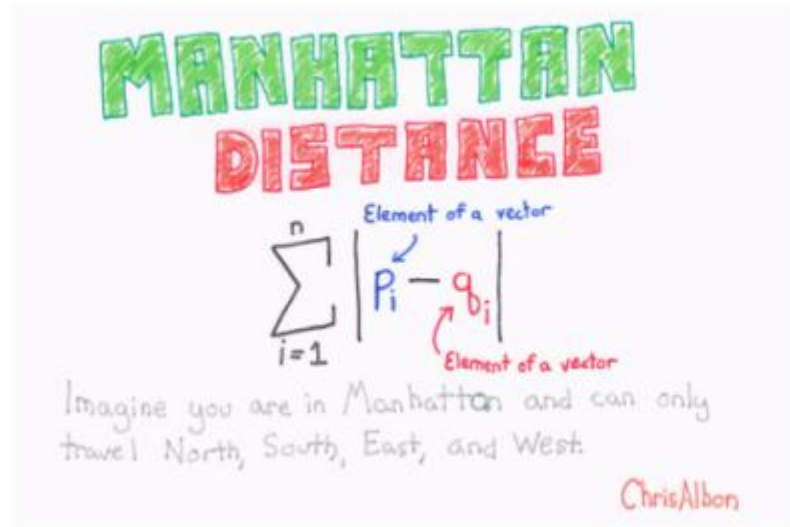
ith element of vector q

ChrisAlbon

Manhattan Distance

- The distance between two points measured along axes at right angles.

$$d(x, y) = \sum_i^n |x_i - y_i|$$



- It is also called rectilinear distance, L1-distance/L1-norm,
- Minkowski's L1- distance, city block distance and taxi cab distance.

Euclidean Distance

- The square root of the sum of squares of the difference between the coordinates and is given by Pythagorean theorem.

$$d(x, y) = \sqrt{\sum_i^n (x_i - y_i)^2}$$

L2 NORM
(Euclidean Norm)

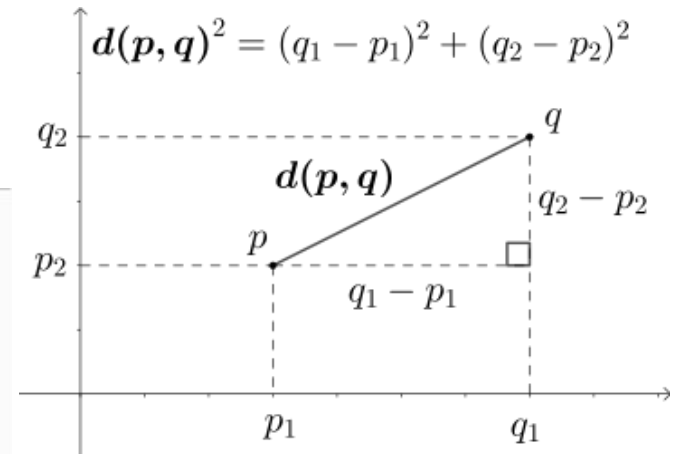
$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

L2 norm is used in many places in machine learning such as normalizing observations and regularization.

↳ Like in NLP.

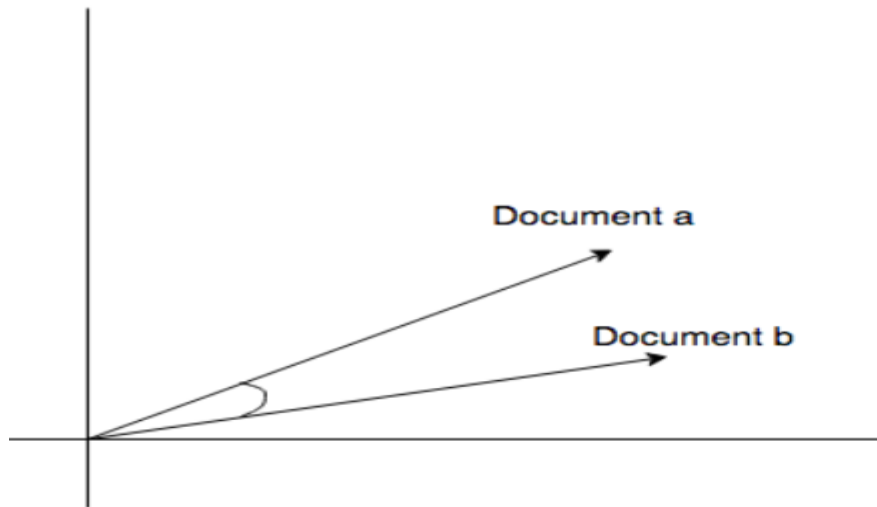
↳ Example: Ridge Regression.

Chris Albon



Cosine Similarity

- Measures the **cosine of the angle** between two vectors.
- It is a judgment of orientation rather than magnitude between two vectors with respect to the origin.
- The cosine of 0 degrees is 1 which means the data points are similar and the cosine of 90 degrees is 0 which means data points are dissimilar.
- Cosine similarity is subjective to the domain and application and is not an actual distance metric.



$$\text{sim}(a, b) = \cos\theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$

Pearson Coefficient

- It is a measure of correlation between two random variables and ranges between $[-1, 1]$.

$$r = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sqrt{\sum (x - \bar{x})^2 \sum (y - \bar{y})^2}}$$

CORRELATION
(Pearson's R)

Handwritten formula with annotations:

Annotations: "value of x_i " points to x_i , "mean of x " points to $\bar{x}$, "value of y_i " points to y_i , "mean of y " points to $\bar{y}$.

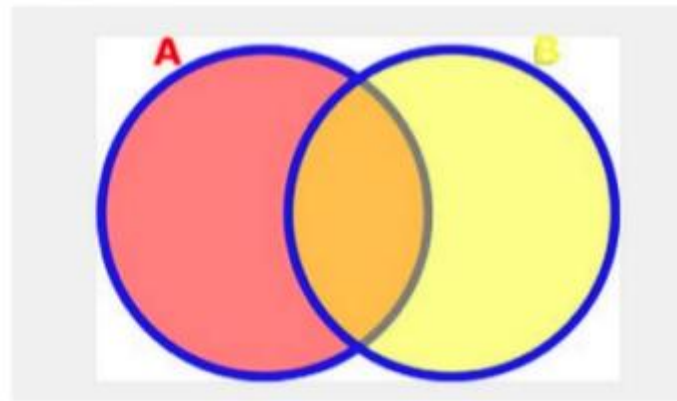
$$\text{Cor}(X, Y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2} \sqrt{\sum (y_i - \bar{y})^2}}$$

Ranges between -1.0 and $+1.0$. The closer to 0.0 the less linear dependence between variables.

Jaccard Similarity

- In the other similarity metrics, we discussed some ways to find the similarity between objects, where the objects are points or vectors.
- We use Jaccard similarity to find similarities between finite sets.
- It is defined as the cardinality of the intersection of sets divided by the cardinality of the union of the sample sets.

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$



Hamming Distance

- If the similarities we discussed were distance measures for continuous variables.
- In the case of categorical variables, Hamming distance must be used.

Hamming Distance

$$D_H = \sum_{i=1}^k |x_i - y_i|$$

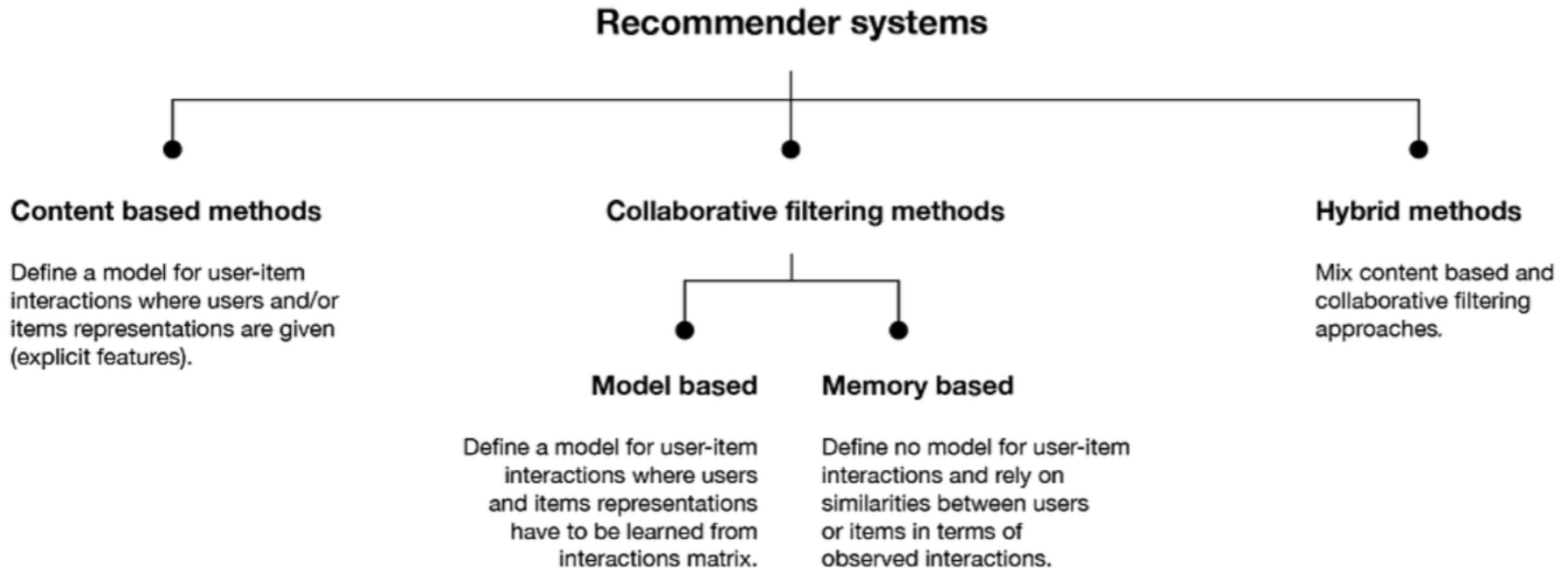
$$x = y \Rightarrow D = 0$$

$$x \neq y \Rightarrow D = 1$$

X	Y	Distance
Male	Male	0
Male	Female	1

The diagram shows the formula for Hamming Loss with handwritten annotations. The title 'HAMMING LOSS' is written in large, colorful block letters. Below it, the formula is
$$L_{\text{Hamming}} = \frac{\sum^n I(\hat{y}_i \neq y_i)}{n}$$
 The numerator is annotated with 'Predicted class' pointing to $\hat{y}_i$ and 'true class' pointing to y_i . The denominator n is annotated with 'number of observations' in red. The signature 'Chris Albon' is at the bottom.

Types of Recommender Systems



Content-Based Recommender Systems

- Content-based recommendation systems use their knowledge about each product to recommend new ones.
- Recommendations are based on attributes of the item.
- Content-based recommender systems work well when [descriptive data](#) on the content is provided beforehand.
- “Similarity” is measured against product attributes.
- Suppose I watch a movie in a particular genre, then I will be recommended movies within that specific genre.
- The movie's attributes, like title, year of release, director and cast, are also helpful in identifying similar movie content.

APPROACH 1: USING RATED CONTENT TO RECOMMEND

- In this approach contents of the product are already rated and based on the user's preference, then a rating is predicted for a similar product.
- We'll use movie recommendations as an example.
- **User Ratings**

User\Movies	Interstellar	Inception	The Shining	Alien
Clark	5 stars			
Bruce			5 stars	
Tony				
Steve		2 stars		1 star

- In the example above, Clark and Bruce have given five-star ratings to the movies *Interstellar* and *The Shining*, clearly indicating a preference for these films.
- For Tony, who has rated nothing, and Steve who has provided only low ratings, it's more difficult to discern their preferences.

- **Movie Attributes**

Movie\Attributes	Science	Adventure	Horror	Mystery
Interstellar	5	4	1	2
Inception	5	3	1	2
The Shining	1	2	5	4
Alien	2	3	5	3

- In the table above, note that *Interstellar* and *Inception* received 5s in the science category, whereas *The Shining* and *Alien* get the highest marks under the horror genre

- **Predicted User Rating**

User\Movies	Interstellar	Inception	The Shining	Alien
Clark	5 stars	5 stars		
Bruce			5 stars	5 stars
Tony				
Steve		2 stars		1 star

- *Inception* is suggested for Clark because he liked *Interstellar* and the movies share similar attributes.
- *Alien* is suggested to Bruce because he liked *The Shining*, which is in the horror genre.
- **Advantages:** Works even when a product has no user reviews.
- **Disadvantages:** Requires descriptive data of all content to recommend, which is time consuming. It's also difficult to implement on large product databases as user's have different opinions about each item.

APPROACH 2: RECOMMENDATION THROUGH DESCRIPTION OF THE CONTENT

- This approach uses the description of the item to make recommendations.
- The description goes deeper into the product details, like title, summary, tag lines, genre, etc., and it provides much more information about the item.
- The format of these details are in text format(string) and it's important to convert this into numbers to easily calculate for similarity.

Term Frequency-Inverse Document Frequency (TF-IDF)

- TF-IDF is used in information retrieval for feature extraction purposes and it is a sub-area of natural language processing (NLP).
- **Term Frequency:** Frequency of the word in the current document to the total number of words in the document.
- It signifies the occurrence of the word in a document and gives higher weight when the frequency is more, so it is divided by document length to normalize.

$$\text{Tf}(t) = \frac{\text{Frequency occurrence of term } t \text{ in document}}{\text{Total number of terms in document}}$$

- **Inverse Document Frequency:** Total number of documents to the frequency occurrence of documents containing the word.
- It signifies the rarity of the word — the less the word occurs in the document, the IDF increases.
- It helps in giving a higher score to rare terms in the documents.

$$\text{Idf}(t) = \log_{10}\left(\frac{\text{Total Number of documents}}{\text{Number of documents containing term } t}\right)$$

TF-IDF

TF-IDF is a measure of originality of a word by comparing the number of times a word appears in a doc with the number of docs the word appears in.

$$\text{TF-IDF} = \text{TF}(t, d) \times \text{IDF}(t)$$

Term frequency

Number of times term t appears in a doc, d

Inverse document frequency

$$\log \frac{1 + n}{1 + \text{df}(d, t)}$$

of documents n

Document frequency of the term t $\text{df}(d, t)$

- In the end, TF-IDF is a measure used to evaluate how important a word is to a document in a document corpus. T
- The importance of the word increases proportionally to the number of times a word appears in the document but is offset by the frequency of the word in the corpus.
- **Example for TF-IDF:** Consider a document containing 100 words wherein the word "odyssey" appears three times. The term frequency for odyssey is then $(3 / 100) = 0.03$. Now, assume we have 10 lakh documents and the word "odyssey" appears in 1,000 of these. The inverse document frequency is calculated as $\log(10,00,000 / 1,000) = 3$. Thus, the TF-IDF weight for Odyssey is $0.03 * 3 = 0.09$.

Collaborative Filtering Recommender Systems

- [Collaborative filtering](#) recommenders make suggestions based on how users rated in the past and not based on the product themselves.
- It only knows how other customers rated the product. “Similarity” is measured against the similarity of users.

Josie		 	Josie bought a toaster and a mixer
Suzie			Suize bought a cheese grader
Steve			Steve bought a toaster What else might Steve buy?

- **USER RATING**

User\Movies	Interstellar	Inception	The Shining	Alien
Clark	5 stars	2 stars	1 star	
Bruce	1 star	5 stars		1 star
Tony	5 star	3 stars	1 star	5 stars
Steve	1 star	5 stars	5 stars	1 star

- As we can see from above Clark and Tony have similar tastes as they rated movies similarly.

- **PREDICTED USER RATING**

User\Movies	Interstellar	Inception	The Shining	Alien
Clark	5 stars	2 stars	1 star	5 stars
Bruce	1 star	5 stars	5 stars	1 star
Tony	5 stars	3 stars	1 star	5 stars
Steve	1 star	5 stars	5 stars	1 star

- Clark is recommended "Alien" from Tony because of their similarity in rating.
- Bruce was suggested "The Shining" because Steve rated it highly.
- **Advantages:**
- No requirement for product descriptions.

- **Disadvantages:**

- Can't recommend items if no user reviews exist (suffers from the cold start problem).
- Difficult to recommend new users and is inclined to favor popular products with lots of reviews.
- Suffers from a sparsity problem as the user will review only selected items.
- Faces the "gray sheep problem" (i.e., useful predictions cannot be made due to sparsity).
- Difficult to recommend new releases since they have less reviews.

Memory Based Recommender System

- Memory-based recommender systems, also known as neighborhood-based recommender systems, rely on the idea of finding similar users or items to generate recommendations.
- These systems make predictions based on the preferences or interactions of users or items without employing any complex mathematical models.
- There are two main types of memory-based recommender systems:
 - **User-Based Collaborative Filtering**
 - **Item-Based Collaborative Filtering**

User-Based Collaborative Filtering

- User-Based Collaborative Filtering (UBCF) is a popular approach in recommender systems where recommendations are generated based on the behaviors and preferences of similar users.
- The basic idea is to find users who are similar to the target user and recommend items liked or rated highly by those similar users.
- UBCF relies on the assumption that users who have similar preferences in the past will have similar preferences in the future.

- **Here's a step-by-step explanation of how to implement a simple User-Based Collaborative Filtering recommendation system:**
- **User-Item Interaction Matrix:**
 - Represent the user-item interactions in a matrix where rows represent users, columns represent items, and the values represent user interactions (e.g., ratings, likes, purchase history).
- **Similarity Calculation:**
 - Compute similarity between users based on their interactions using a similarity metric such as cosine similarity or Pearson correlation.
- **Identify Similar Users:**
 - Identify the most similar users to the target user based on the computed similarity scores.
- **Generate Recommendations:**
 - Recommend items that the target user has not interacted with, based on the interactions of similar users. Sort the recommended items by their predicted ratings or interactions.
- **Present Recommendations:**
 - Present the top-N recommended items to the target user.

Item based collaborative filtering

- Item-Based Collaborative Filtering (IBCF) is a popular technique used in recommender systems to provide recommendations based on similarities between items rather than users.
- It recommends items to a user based on their interactions with similar items.
- The core idea is that if a user likes or interacts with a certain item, they are likely to like other items that are similar to it.

Model Based Collaborative Filtering

- Model-Based Collaborative Filtering is an approach used in recommender systems that involves building predictive models based on the user-item interaction data to make recommendations.
- These models use machine learning or matrix factorization techniques to learn patterns, preferences, and relationships from the data to generate personalized recommendations.

- **Here's an overview of the steps to implement Model-Based Collaborative Filtering:**
- **User-Item Interaction Matrix:**
 - Create a matrix where rows represent users, columns represent items, and the values represent user interactions (e.g., ratings, likes, purchase history).
- **Model Training:**
 - Train a machine learning model (e.g., matrix factorization, neural networks) on the interaction data to learn the underlying patterns and preferences.
- **Model Evaluation:**
 - Evaluate the model's performance using appropriate evaluation metrics (e.g., mean squared error, accuracy, etc.).
- **Generate Recommendations:**
 - Use the trained model to generate recommendations for each user based on their interactions and predicted preferences.
- **Present Recommendations:**
 - Present the top-N recommended items to the users.

- Matrix factorization methods like Singular Value Decomposition (SVD), Alternating Least Squares (ALS), and deep learning-based approaches like neural collaborative filtering (NCF) are commonly used in model-based collaborative filtering.

Examples of Recommender System

- **AMAZON**
- **IMDB**
- **FACEBOOK & INSTAGRAM**
- **YOUTUBE**
- **GOOGLE**
- **ANOTHER GOOGLE SUBSIDIARY- GMAIL**

Ethical Use of Recommender System

- The ethical use of recommender systems is of paramount importance to ensure that users are provided with fair, transparent, and helpful recommendations while respecting their privacy and well-being.
- **Transparency and Explanation:**
 - Provide clear explanations of how the recommender system works, what data is used, and how recommendations are generated. Users should understand why certain recommendations are being made.
- **User Control and Customization:**
 - Allow users to have control over the recommendations they receive. Offer options for customization, preferences, and the ability to filter or adjust recommendations according to their needs and preferences.
- **Avoid Discrimination and Bias:**
 - Strive to eliminate biases related to race, gender, age, ethnicity, or any other characteristic. Ensure that the recommender system provides fair and unbiased recommendations to all users.
- **Diversity and Serendipity:**
 - Aim to offer a diverse set of recommendations, avoiding filter bubbles where users are only exposed to content similar to their previous choices. Encourage users to explore a variety of content.

- **Informed Consent and Privacy:**
 - Obtain informed consent from users regarding data collection, usage, and sharing. Clearly explain the privacy policy and how user data is handled, stored, and protected.
- **Data Collection and Storage:**
 - Collect and store only the necessary data for providing recommendations. Implement strong security measures to protect user data from unauthorized access or misuse.
- **Prevent Over-Personalization:**
 - Avoid excessive personalization that may lead to a "creepy" feeling for users. Strive for a balance between personalization and maintaining user privacy.
- **Avoid Manipulation and Exploitation:**
 - Do not manipulate users or exploit their vulnerabilities to increase engagement or interactions. Prioritize the user's best interests and well-being.

- **Feedback Mechanism:**
 - Implement a feedback mechanism where users can report issues, biases, or concerns related to recommendations. Use this feedback to improve the recommender system.
- **Regular Audits and Assessments:**
 - Conduct regular audits and assessments of the recommender system to ensure compliance with ethical guidelines and standards. Address any identified issues promptly.
- **Stakeholder Involvement:**
 - Involve relevant stakeholders, including users, domain experts, ethicists, and diverse communities, in the design, development, and evaluation of the recommender system to incorporate different perspectives.
- **Education and Awareness:**
 - Educate users about how recommender systems function and their ethical implications. Promote digital literacy and awareness regarding online content and recommendations.

Thanks

Recommender System - II

Binary Labels in Recommender System

- Binary labels in a recommender system refer to the binary, or two-valued, feedback or interactions provided by users regarding items.
- In recommender systems, binary labels like "favs," "likes," and "clicks" are often used to capture user preferences or interactions with items.
- These labels are used to understand and predict user behavior and preferences for recommending items to users.
- Binary labels serve as signals or feedback data used by recommendation algorithms to understand user preferences.
- They help in building recommendation models by predicting or inferring user preferences for items.
- Algorithms can utilize these signals to create models that predict which items a user might prefer based on their history of marking items as favorites, liking items, or clicking on them.

Favorites (Favs)

- "Favs" typically represent items that users explicitly mark or save as their favorites.
- This action implies a higher level of preference or special interest in the item.
- Users might designate certain items (movies, products, songs, etc.) as favorites, indicating a strong positive affinity towards those particular items.

Likes

- "Likes" usually signify a positive interaction or approval of an item, but not necessarily as strong as marking it as a favorite.
- Users might click a "like" button or perform an action indicating they have a positive sentiment or preference for the item.

Clicks

- "Clicks" refer to the user interactions such as clicking on an item, visiting a page, or opening a link.
- In recommender systems, clicks can be used as **implicit feedback** indicating user interest in an item, although it might not explicitly imply preference or satisfaction.

Example

- Suppose we have a user-item interaction matrix where users provide binary feedback for items.
- User-Item Interaction Matrix:

	Item1	Item2	Item3	Item 4
User 1	1	0	1	0
User 2	0	1	1	0
User 3	1	0	0	1

- Users are represented by rows (User 1, User 2, User 3).
- Items are represented by columns (Item 1, Item 2, Item 3, Item 4).
- Each cell in the matrix contains a binary label (1 or 0), indicating user interaction with an item. For instance:
- '1' represents a positive interaction (like, click, purchase, etc.).
- '0' represents no interaction or a lack of interest.
- Interpretation:
- User 1 interacted with Item 1 and Item 3 but did not interact with Item 2 or Item 4.
- User 2 interacted with Item 2 and Item 3 but not with Item 1 or Item 4.
- User 3 interacted with Item 1 and Item 4 but did not interact with Item 2 or Item 3.
- These binary labels could represent various interactions:
- User liking or favoriting an item (indicated by '1').
- Lack of interest or no interaction with an item (indicated by '0').

Mean Normalization

- Mean normalization is a technique used in recommender systems and various data analysis contexts to preprocess data, specifically to normalize and scale the values within a certain range.
- It involves adjusting values in a dataset by subtracting the mean and dividing by the range or standard deviation.
- In the context of recommender systems, mean normalization might be applied to user-item interaction data in a user-item matrix.

Significance of Mean Normalization

- Standardize the data and bring it to a common scale.
- Make data centered around zero, which helps algorithms **converge faster** and prevents bias in models.
- Reduce the impact of different users having varied rating scales (some users might tend to rate higher than others).